

TDD 완전정복

VSCode Java & C++ 개발자를 위한 실전 테스트 주도 개발



SW 테스트
& 테스트 레벨



JUnit 5 &
GoogleTest API



Test Double
& Mockito



TDD 사이클
& 실습



Test Smell &
정적 분석



Java: JUnit 5 · Mockito · Hamcrest



C++: GoogleTest · GMock · gcov

전체 강의 구성 (8시간)

파트	시간	주제	핵심 내용
1부	60분	SW 테스트 개론	테스팅 정의·종류·레벨·비용·테스트 자동화
2부	60분	테스트 케이스 설계	블랙/화이트박스, CORRECT, Right-BICEP, AAA
3부	70분	xUnit 5 완전 정복	어노테이션·Lifecycle·Parameterized·Assertions·Assumptions
4부	60분	Hamcrest & Test Double	Hamcrest 매처, Dummy/Fake/Stub/Spy/Mock
5부	60분	Mockito 실전	verify·when·ArgumentCaptor·spy vs mock
6부	60분	TDD 사이클 & 실습	3법칙·Ask-Respond-Refine·StringCalculator 실습
7부	40분	Test Smell & 정적 분석	Write better tests
8부	20분	마무리 & 실천 로드맵	Java/C++ 비교 정리, 단계별 적용 전략

1

SW 테스트팅 개론

테스팅 정의 · 종류 · 레벨 · 자동화 필요성

🕒 00:00 ~ 01:00 (60분)

SW Testing이란 무엇인가?

SW Testing은 SW의 품질을 조사하는 과정 — 결함을 찾기 위해 SW를 실행하는 과정 (Myers)

테스팅 대상	목적	수행 주체	Java 도구	C++ 도구
Unit Test	개별 함수/클래스 검증	개발자	JUnit 5	GoogleTest
Integration Test	모듈 간 연동 검증	개발자/QA	Spring Test	CTest
System Test	전체 시스템 기능 검증	QA	Selenium	Robot Framework
Acceptance Test	사용자 요구 충족 확인	고객/QA	Cucumber	—

구분	정의
다익스트라	Test로 버그 존재를 증명할 수는 있지만, 버그가 없음을 증명하는 데는 사용할 수 없다
Myers	테스팅은 결함을 찾으려는 의도로 SW를 실행하는 과정이다

Testing의 4 영역



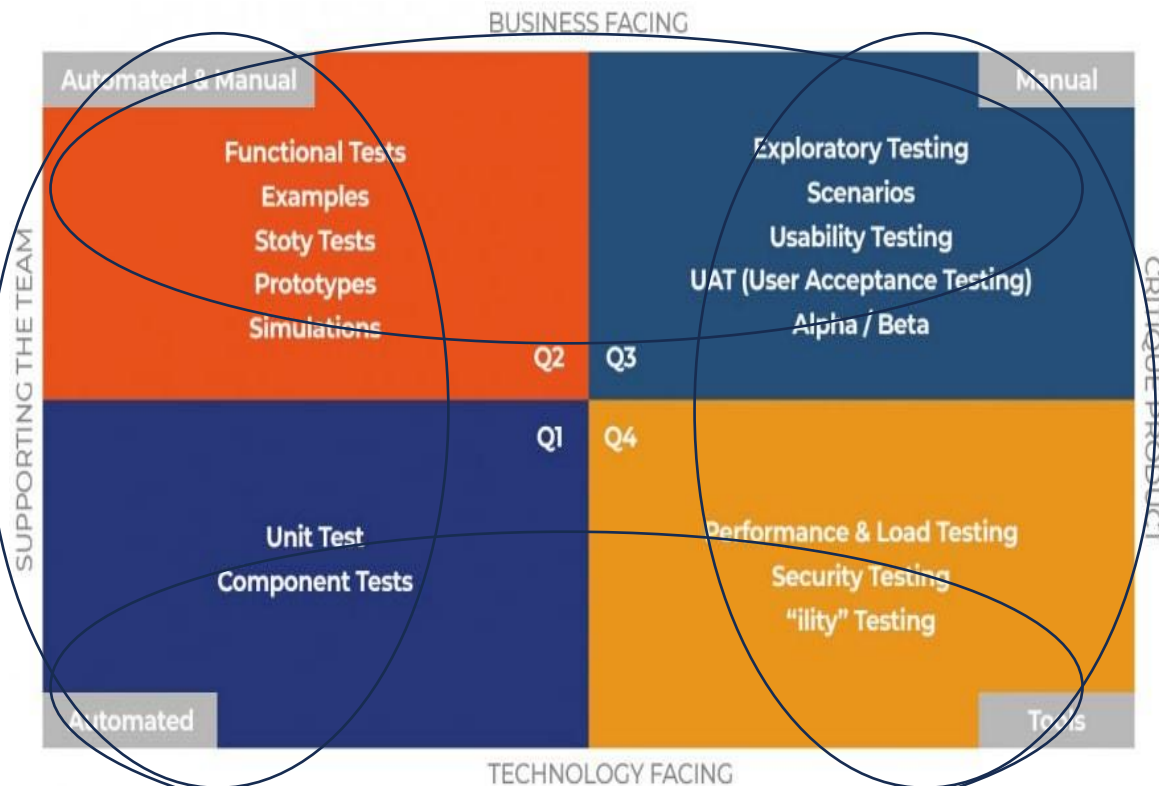
다음 중 개발팀을 지원하는 테스트는?

1. Q1, Q2
2. Q3, Q4
3. Q1, Q4
4. Q2, Q3

개발팀을 지원하는 테스트

Testing의 4 영역

제품 품질을
검토하는 테스트



- Q1: Test Automation to improve the code.
- Q2: Test Automation + Manual to improve business outcomes.
- Q3: Manual tests to provide detailed feedback on product performance, functionality, and user experience. (often it is rechecking things done in Q1 and Q2)
- Q4: Tools to ensure security and compatibility

변경 후 기도하기(Edit and Pray)

- 신중한 검토 -> 변경 대상 이해 -> 변경 -> 정상 동작 확인-> 의도하지 않은 변경 여부 확인(Side effect)
- 전문적인 방법처럼 보이지만, 신중 ≠ 안정

체계적인 보호 없이 변경을 시도하는 방식으로,
위험성이 높음

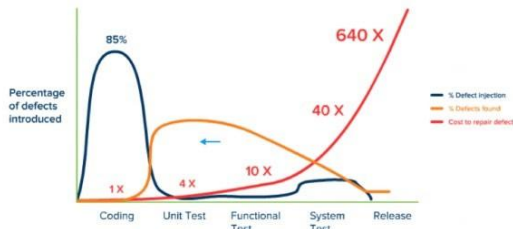
보호 후 수정하기(Cover and Modify)

- 테스트 루틴으로 코드를 보호
- 문제 발생시 빠르게 찾을 수 있음
- 테스트 루틴 = 피드백

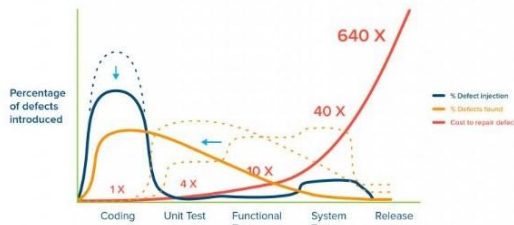
코드 변경 후 문제를 사전에 감지하여, Side Effect를 방지하고 안전하게 유지보수하는 방식

테스팅의 비용 — 왜 일찍 해야 하는가? Shift-Left Testing

결함은 늦게 발견할수록 수정 비용이 기하급수적으로 증가한다 (Cost of Defect)



Jones, Capers. Applied Software Measurement: Global Analysis of Productivity and Quality.



Jones, Capers. Applied Software Measurement: Global Analysis of Productivity and Quality.

x1
요구사항

x5
설계

구현

단위테스트

통합테스트

인수테스트

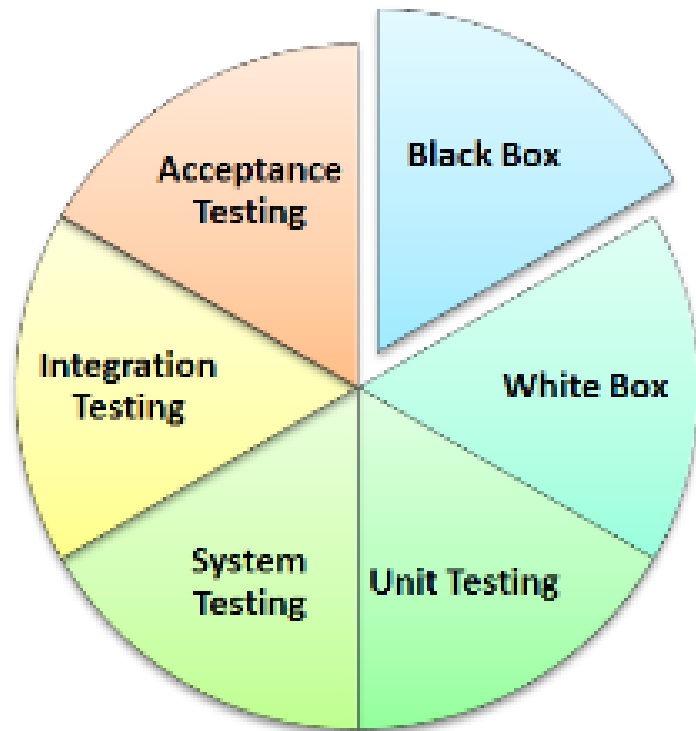
운영

상대적 수정 비용 (요구사항 단계 = x1)

💡 TDD: 개발시간 15~35% 증가 → 결함률 40~90% 감소 (Microsoft/IBM 연구)

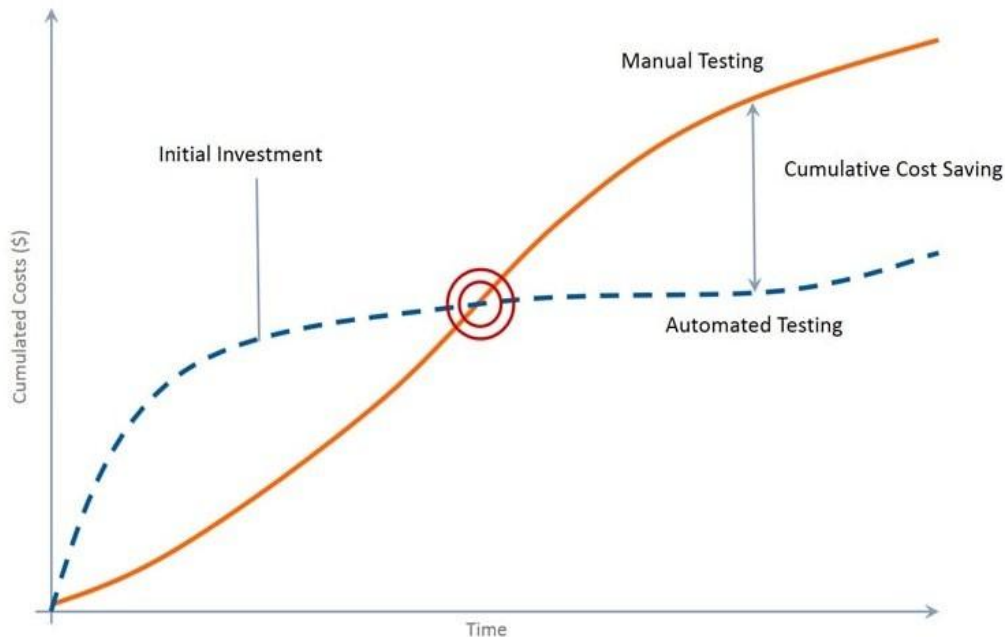
수동 테스트(Manual Testing)란?

수동 테스트는 테스터가 자동화 도구를 사용하지 않고 수동으로 테스트 케이스를 실행하는 일종의 소프트웨어 테스트입니다.



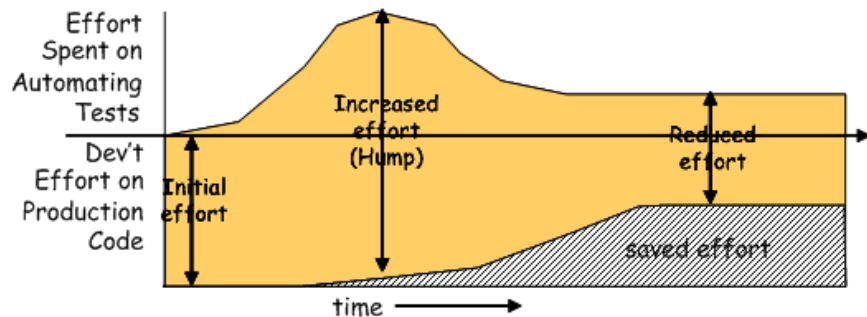
테스트 자동화

- 테스트는 자동화되어야 한다.
- 테스트는 빨라야 한다.
- 테스트는 어떤 경우에도 수행되어야 한다.
- 수동 테스트를 하는 시간
.VS.
자동 테스트를 작성하고 테스트 하는 시간



테스트 자동화

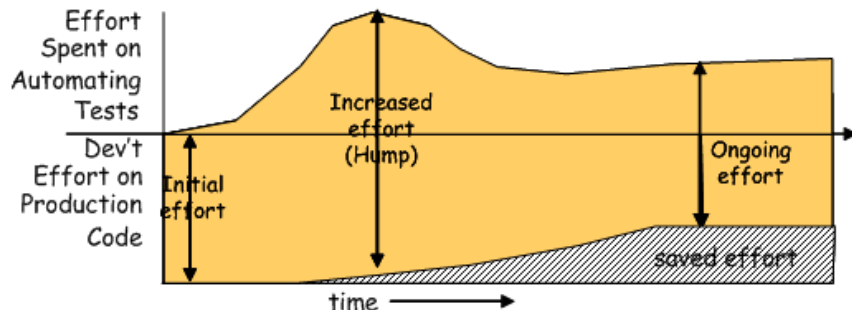
- 단순히 자동화 테스트를 도입하는 것이 중요한 것이 아니라, "올바르게 자동화하는 것"이 ROI를 높이는 핵심이다!



Sketch Economics-Good embedded from Economics-Good.gif

Fig. X: An automated unit test project with a good ROI.

The cost-benefit trade off where the total cost is reduced by good test practices.



Sketch Economics-Bad embedded from Economics-Bad.gif

Fig. X: An automated unit test project with a poor ROI.

The cost-benefit trade off where the total cost is increased by poor test practices.

테스트 자동화 — 언제, 무엇을 자동화할 것인가?

JUnit 5

Java 단위 테스트 표준

Mockito

Java 모킹 프레임워크

JaCoCo

Java 커버리지 측정

GoogleTest

C++ 단위 테스트

GMock

C++ 모킹 프레임워크

gcov/lcov

C++ 커버리지 측정

Test Automation Manifesto	설명
Necessary	각 테스트는 원하는 동작 명세에 기여해야 한다
Sufficient	테스트는 실패 원인을 정확히 드러내야 한다
Unambiguous	성공/실패 기준이 명확해야 한다
Self-checking	사람의 판단 없이 결과를 자동 검증해야 한다
Repeatable	언제 어디서나 동일한 결과여야 한다

Test Automation Manifesto

Automated test should be:

- Concise** – As simple as possible and no simpler.
- SelfChecking** – Test reports its own results ; needs no human interpretation.
- Repeatable** – Test can be run many times in a row without human intervention.
- Robust** – Test produces same result now and forever.
Tests are not affected by changes in the external environment.
- Sufficient** – Tests verify all the requirements of the software being tested.

Test Automation Manifesto

Necessary – Everything in each test contributes to the specification of desired behavior.

Clear – Every statement is easy to understand.

Efficient – Tests run in a reasonable amount of time.

Specific – Each test failure points to a specific piece of broken functionality; unit test failures provide “defect triangulation”.

Independent – Each test can be run by itself or in a suite with an arbitrary set of other tests in any order.

Maintainable – Tests should be easy to understand and modify and extend. **Traceable** – To and from the code it tests and to and from the requirements.

출처 : xUnitPatterns.com

2

테스트 케이스 설계

블랙박스 · 화이트박스 · CORRECT · Right-BICEP · AAA

🕒 01:00 ~ 02:00 (60분)

블랙박스 vs 화이트박스 테스트

구분	블랙박스 (명세 기반)	화이트박스 (구조 기반)
관점	외부 행동·기능 기준	내부 코드·경로 기준
주요 기법	동치 분할, 경계값 분석, 결정 테이블, 시나리오	문장/결정/조건/경로 커버리지
적합 대상	API 계층, 비즈니스 로직 검증	복잡 조건문, 분기 로직 검증
커버리지 측정	기능 커버리지	코드 커버리지 (JaCoCo / gcov)
Java 예시	@Test assertEquals(...)	JaCoCo: mvn test → 커버리지 리포트
C++ 예시	TEST(Suite, Name) EXPECT_EQ	gcov + lcov → HTML 리포트

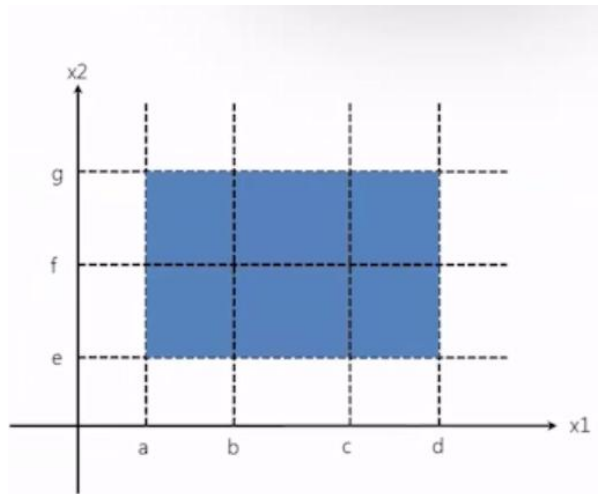


VSCode: Java → JaCoCo 커버리지 + Coverage Gutters 확장 | C++ → gcov + llvm-cov 으로 라인별 커버리지 시각화

블랙박스 테스트(명세 기반 테스트) 주요 방법

동등 분할(equivalence partitioning)

- 기본 아이디어
 - 입력 데이터를 동일한 동작이 예상되는 클래스로 분할한다.
 - 각 클래스의 대표값을 선택하여 테스트 데이터를 설계한다.
 - 단, 분할된 클래스에 속한 값은 모두 같다고 가정한다.
- 테스트 데이터
 - 입력값을 여러 개의 동등 클래스로 분할한다.
 - 각 클래스마다 대표값을 추출한다.
- 이점
 - 누락없이 완전하게 테스트할 수 있다.
 - 중복없이 테스트할 수 있다.



블랙박스 테스트(명세 기반 테스트) 주요 방법

동등 분할(equivalencepartitioning)

- Example 1 : 성적 판정
- 동등 분할을 이용하여 성적(0~100) 등급을 판정하는 테스트 데이터를 작성하시오.

동등 클래스	등급
90 ~ 100	A
80 ~ 89	B
70 ~ 79	C
0 ~ 69	F

입력 값	기대 등급
95	A
85	B
75	C
35	F

블랙박스 테스트(명세 기반 테스트) 주요 방법

동등 분할(equivalence partitioning)

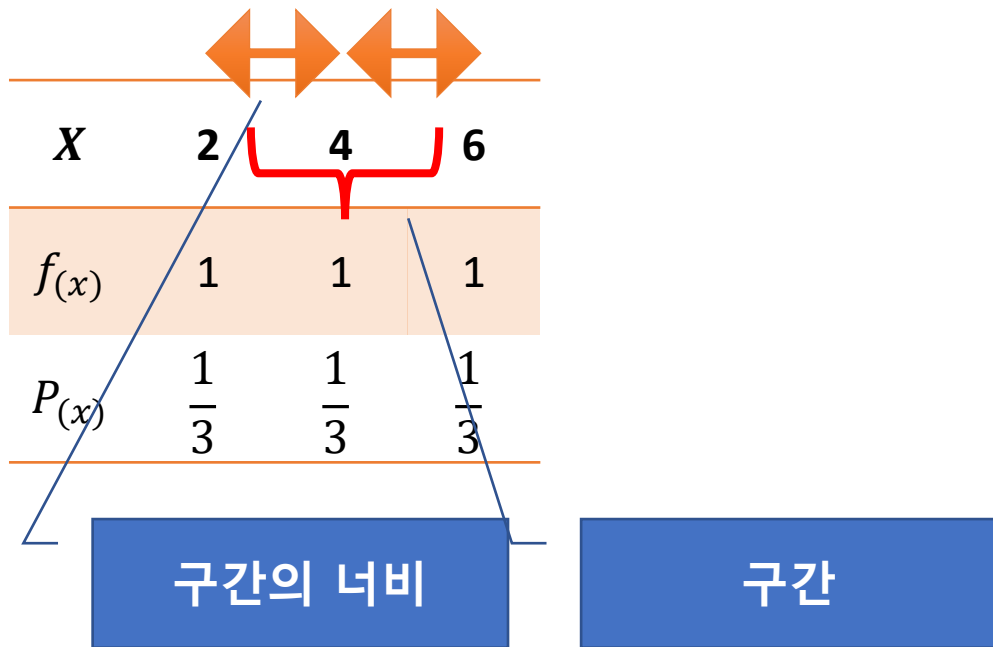
- Example 2 : 무게 판정
- 동등 분할을 이용하여 택배 무게 측정에 대한 테스트 데이터를 작성하시오.

동등 클래스	등급
$0 < \text{무게} < 3$	가
$3 \leq \text{무게} < 7$	나
$7 \leq \text{무게} < 15$	다
$15 \leq \text{무게} < 25$	라
$25 \leq \text{무게} < 30$	마

입력 값	기대 등급
1.5	가
6.3	나
8,9	다
21.7	라
27.2	마

블랙박스 테스트(명세 기반 테스트) 주요 방법

구간



블랙박스 테스트(명세 기반 테스트) 주요 방법

구간

- 일반적으로, 각 구간의 너비는 다음과 같이 한다.
 - (최대값 - 최소값) / 구간의 개수
 - 첫 번째 구간에는 양적 자료의 최소값이 포함되도록 하고, 제일 마지막 구간에는 최대값이 포함되도록 구간을 만들면 된다. 이렇게 만들어진 구간을 이용하여 각 구간에 있는 양적 자료의 빈도와 백분율을 구해서 표를 작성하면 된다. 작성된 표를 이용하여 양적 자료에 있는 특징을 파악하면 된다.
- 그러나, 구간을 만들 때에 구간을 몇 개로 할 지에 대해서는 다양한 방법이 있습니다. 여기서는 Herbert A. Sturges(1926)가 「The Choice of a Class Interval」라는 제목으로 발표한 논문에서 제시한 공식은 다음과 같다. 참고로 n 은 데이터의 개수를 의미한다.

$$1 + 3.3\log_{10}(n)$$

- 예 : 100개의 자료일 경우 $1 + 3.3\log_{10}(100) = 7.3 \rightarrow 7 \sim 8$ 개

블랙박스 테스트(명세 기반 테스트) 주요 방법

구간

- 아래의 예제 데이터를 가지고 구간을 만들고 빈도와 백분율을 구해보도록 한다.

54 56 57 59 73 74 76 78 78 82 82 85 86 91 94 98 116

- Sturges 공식을 따르면, 구간의 개수는 다음과 같다.

$$1 + 3.3\log_{10}(17) = 5.06$$

구분	빈도	백분율(%)
50이상 ~ 65이하	4	23.5
65초과 ~ 80이하	5	29.4
80초과 ~ 95이하	6	35.3
95초과 ~ 110이하	1	5.9
110초과~125이하	1	5.9
합계	17	100.0



밑수가 10인 상용로그

밑수가 n인 자연로그

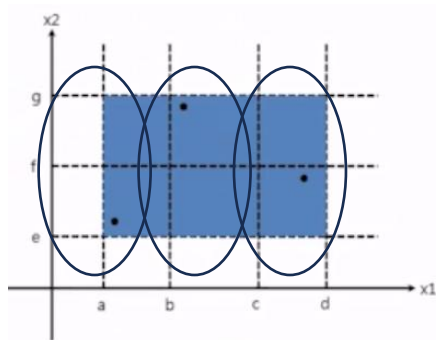
- 구간의 개수는 반올림해서 5개로 한다. 구간의 너비는 $(116-54)/5$ 로 하면 대략적으로 15가 된다.

블랙박스 테스트(명세 기반 테스트) 주요 방법

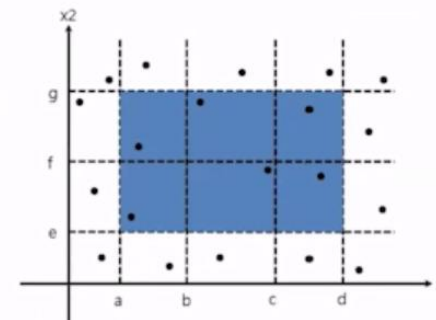
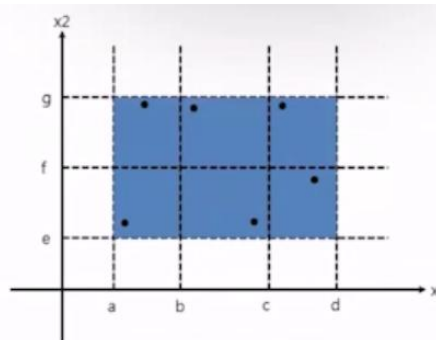
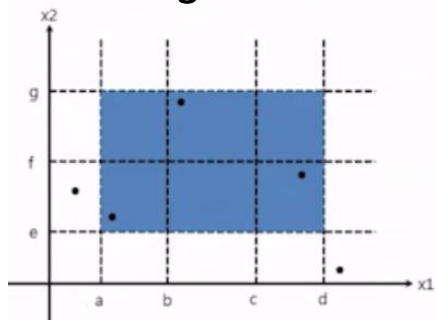
동등 분할(equivalence partitioning)의 다른 예

- Weak normal
 - Single fault assumption 가정\한 변수의 동치 클래스 구간만 이용
 - 가장 많은 변수 선택
- Strong normal
 - Multiple fault assumption 가정
 - 모든 변수간의 곱 집합을 이용
- Weak robust
 - Weak normal + 범위 밖
- Strong robust
 - Strong normal + 범위 밖

2



3



블랙박스 테스트(명세 기반 테스트) 주요 방법

동등 분할(equivalence partitioning)의 다른 예

- Example 3 : 우편물 발송
- 동치 클래스가 주어졌을 때 Weak normal 동등 분할로 테스트 데이터를 작성하시오.

무게
$0 < \text{무게} < 3$
$3 \leq \text{무게} < 7$
$7 \leq \text{무게} < 15$
$15 \leq \text{무게} < 25$
$25 \leq \text{무게} < 30$

5

지역
A
B
C

3

유형
일반
익일

2

가장 많은 변수
선택

무게	지역	유형
1.3	A	일반
5.7	B	익일
7.2	C	일반
23.8	A	일반
26	B	익일

블랙박스 테스트(명세 기반 테스트) 주요 방법

동등 분할(equivalencepartitioning)의 다른 예

- Example 4 : 날짜 계산
- Weak normal, Strong normal 동등 분할로 테스트 데이터를 작성하시오.

입력 값	결과
$1 \leq \text{month} \leq 12$ $1 \leq \text{day} \leq 31$ $1900 \leq \text{year} \leq 2100$	내일 날짜

동치 클래스

- 3 M1 = {month : month has 30 days}
 M2 = {month : month has 31 days}
 M3 = {month : month is February}
- 4 D1 = {1 <= day <= 28}
 D2 = {day = 29}
 D3 = {day = 30}
 D4 = {day = 31}
- 3 Y1 = {year = 2000}
 Y2 = {year : year is a non-century leap year}
 Y3 = {year : year is a common year}

Weak Normal

month	day	year	Expected Output
6	14	2000	6/15/2000
7	29	1996	7/30/1996
2	30	2002	Invalid Input Date
6	31	2000	Invalid Input Date

가장 많은 변수
선택

블랙박스 테스트(명세 기반 테스트) 주요 방법

동등 분할(equivalence partitioning)의 다른 예

- Example 4 : 날짜 계산
- Weak normal, Strong normal 동등 분할로 테스트 데이터를 작성하시오.

Strong Normal

month	day	year	Expected Output
6	14	2000	6/15/2000
6	14	1996	6/15/1996
6	14	2002	6/15/2002
6	29	2000	6/30/2000
6	29	1996	6/30/1996
6	29	2002	6/30/2002
6	30	2000	Invalid Input Date
6	30	1996	Invalid Input Date
6	30	2002	Invalid Input Date
6	31	2000	Invalid Input Date
6	31	1996	Invalid Input Date
6	31	2002	Invalid Input Date
7	14	2000	7/15/2000
7	14	1996	7/15/1996
7	14	2002	7/15/2002
7	29	2000	7/30/2000
7	29	1996	7/30/1996
7	29	2002	7/30/2002
7	30	2000	7/31/2000
7	30	1996	7/31/1996
-	-	-	-

month	day	year	Expected Output
7	31	2000	8/1/2000
7	31	1996	8/1/1996
7	31	2002	8/1/2002
2	14	2000	2/15/2000
2	14	1996	2/15/1996
2	14	2002	2/15/2002
2	29	2000	Invalid Input Date
2	29	1996	3/1/1996
2	29	2002	Invalid Input Date
2	30	2000	Invalid Input Date
2	30	1996	Invalid Input Date
2	30	2002	Invalid Input Date
2	31	2000	Invalid Input Date
2	31	1996	Invalid Input Date
2	31	2002	Invalid Input Date

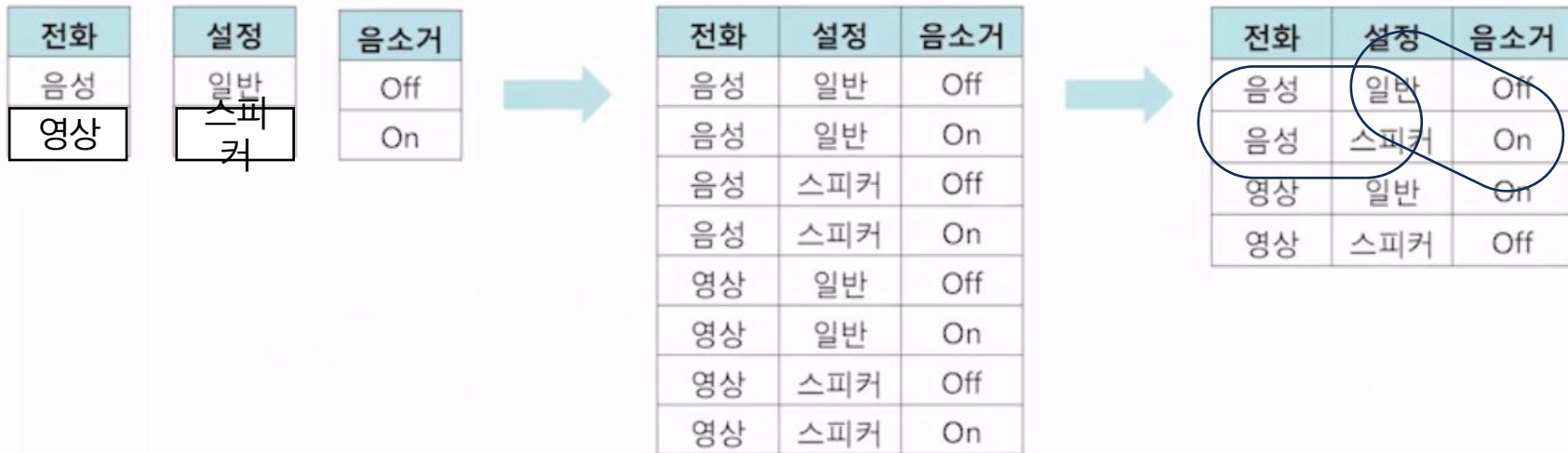
3x4x3

모든 변수간의
곱 집합

블랙박스 테스트(명세 기반 테스트) 주요 방법

페어와이즈(Pairwise) 테스트

- 기본 아이디어
 - Strong normal, All combinations 경우처럼 모든 가능한 입력 값들의 조합들을 테스트하는 대신 모든 짝(pair)들의 조합을 테스트하는 방법



블랙박스 테스트(명세 기반 테스트) 주요 방법

페어와이즈(Pairwise) 테스트

- Example : 피자주문
- 도우:곡물,나폴리,썬 크리스트
- 토핑:기본,프리미엄
- 주문:전화,온라인

테스트 케이스	입력			기대 출력
	도우	토핑	주문	
1	곡물	기본	전화	13000
2	곡물	기본	온라인	12000
3	곡물	프리미엄	전화	15000
4	곡물	프리미엄	온라인	14000
5	나폴리	기본	전화	15000
6	나폴리	기본	온라인	14000
7	나폴리	프리미엄	기본	17000
8	나폴리	프리미엄	온라인	16000
9	썬 크리스트	기본	전화	17000
10	썬 크리스트	기본	온라인	16000
11	썬 크리스트	프리미엄	전화	19000
12	썬 크리스트	프리미엄	온라인	18000

입력 인자 값

입력 인자	도우	토핑	주문
동등 클래스	곡물	기본	전화
	나폴리	프리미엄	온라인
	썬 크리스트		

테스트 케이스	입력			기대 출력
	도우	토핑	주문	
1	곡물	기본	전화	13000
2	곡물	프리미엄	온라인	12000
3	나폴리	기본	전화	15000
4	나폴리	프리미엄	온라인	14000
5	썬 크리스트	기본	온라인	18000
6	썬 크리스트	프리미엄	전화	19000

All combinations 테스트케이스

Pairwise
테스트케이스

블랙박스 테스트(명세 기반 테스트) 주요 방법

경계값 분석(Boundary Value Analysis)

- 기본 아이디어
 - 에러는 경계 값 근처에서 빈번히 발생한다.
 - 그러므로, 경계 값 주위를 꼼꼼히 조사하자.
 - 각 클래스의 하한 및 상한에서 데이터를 도출한다.
- 테스트 데이터
 - 1-value 경계값: min, max
 - 2-value 경계값: min-, min, max, max+
 - 하한 경계값과 하한 경계를 벗어난 가장 가까운 값
 - 상한 경계값과 상한 경계를 벗어난 가장 가까운 값

블랙박스 테스트(명세 기반 테스트) 주요 방법

경계값 분석(Boundary Value Analysis)

- Example 1 : 성적 판정
 - 1-value, 2-value 경계값 분석으로 성적 등급에 대한 테스트 데이터를 작성하시오.

명세

점수	등급
90 ~ 100	A
80 ~ 89	B
70 ~ 79	C
0 ~ 69	F

1-value 경계값 분석

하한경계	상한경계	기대 등급
90	100	A
80	89	B
70	79	C
0	69	F

2-value 경계값 분석

하한경계	상한경계	기대 등급
89, 90	100, 101	
79, 80	89, 90	
69, 70	79, 80	
-1, 0	69, 70	

블랙박스 테스트(명세 기반 테스트) 주요 방법

경계값 분석(Boundary Value Analysis)

- Example 2 : 무게 판정
 - 2-value 경계값 분석으로 택배 무게 측정에 대한 테스트 데이터를 작성하시오.

명세

무게	등급
$0 < \text{무게} < 3$	가
$3 \leq \text{무게} < 7$	나
$7 \leq \text{무게} < 15$	다
$15 \leq \text{무게} < 25$	라
$25 \leq \text{무게} < 30$	마

2-value 경계값 분석

무게	등급	하한 2-Value 경계값	상한 2-Value 경계값
$0 < \text{무게} < 3$	가	0, 0.1	2.9, 3
$3 \leq \text{무게} < 7$	나	2.9, 3	6.9, 7
$7 \leq \text{무게} < 15$	다	6.9, 7	14.9, 15
$15 \leq \text{무게} < 25$	라	14.9, 15	24.9, 25
$25 \leq \text{무게} < 30$	마	24.9, 25	29.9, 30

블랙박스 테스트(명세 기반 테스트) 주요 방법

경계값 분석(Boundary Value Analysis)

- Example 3 : 소수 판정
 - 0~1000 값을 받아 들어 소수를 판정하는 프로그램의 테스트 데이터를 작성하시오.

입력 값	기대 결과
0	No
1000	No

1-value 경계값

입력 값	기대 결과
-1	Invalid
0	No
1000	No
1001	Invalid

2-value 경계값

번호	입력 값	기대 결과
1	-1	음수이어서 도움 메시지 출력
2	0	No
4	3	Yes
5	1000	No
6	1001	범위 넘어선 값이어서 도움 메시지 출력
7	27	두 개 이상 입력이어서 도움 메시지 출력
8	"r"	문자이어서 도움 메시지 출력
9	공백	공백이어서 도움 메시지 출력

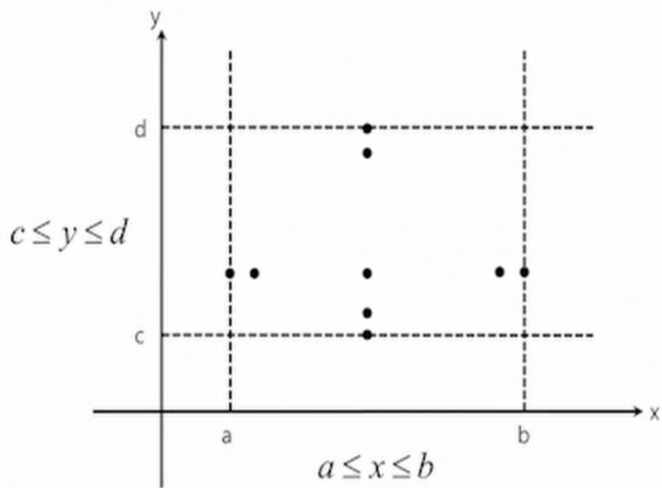
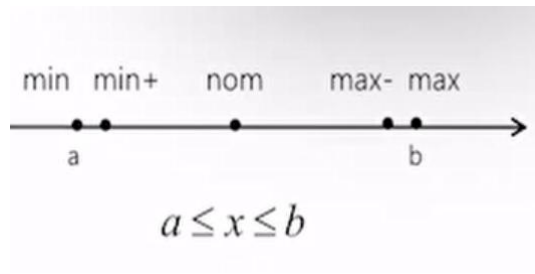
블랙박스 테스트(명세 기반 테스트) 주요 방법

기타 테스트 데이터

- 중앙값(nom)을 테스트해 볼 수도 있음
- 3-value인 min-, min, min+, max-, max, max+ 를 취할 수 있음

기타 사용되는 경계값 분석

- Basic BVA(Boundary Value Analysis)
- Robust BVA
- Worst-case BVA
- Robust Worst-case BVA



블랙박스 테스트(명세 기반 테스트) 주요 방법

Basic BVA(Boundary Value Analysis)

- 기본적인 경계값 고려
- Single fault 채택

Robust BVA

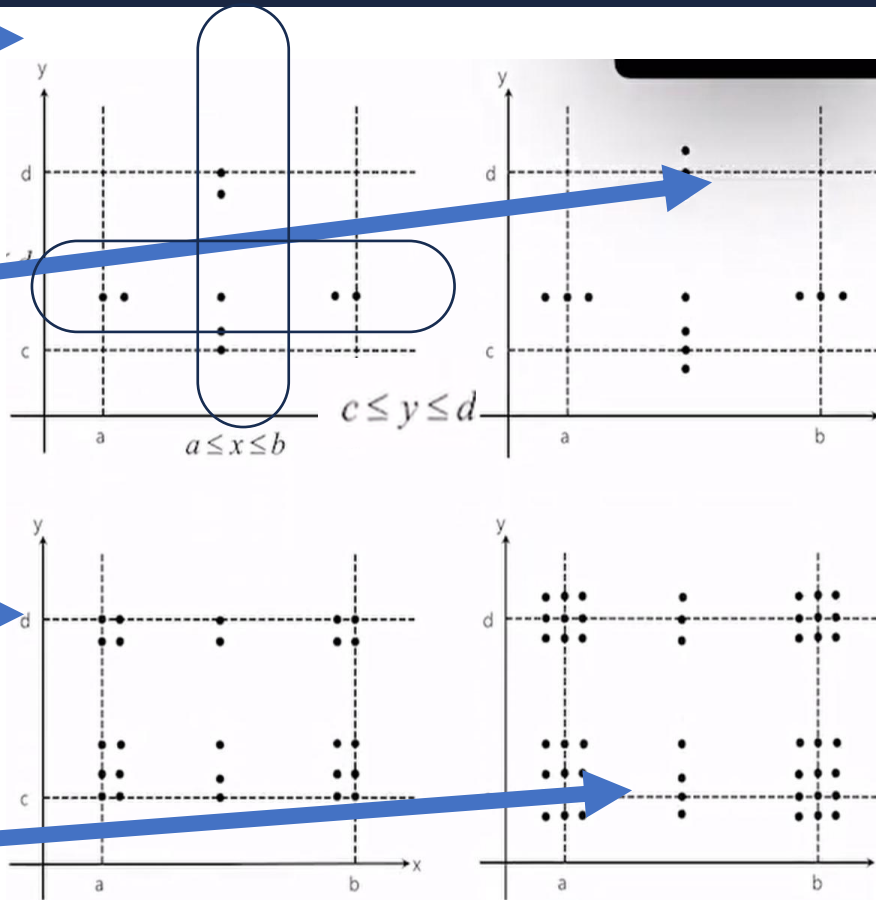
- Basic BVA 확장
- 범위 밖 고려

Worst-case BVA

- 모든 조합을 고려
- Multiple fault 허용

Robust Worst-case BVA

- Robust + Worst-case



블랙박스 테스트(명세 기반 테스트) 주요 방법

의사 결정 테이블 혹은 결정표(decision table)

- 기본 아이디어
 - 입력 조건의 모든 조합을 고려한다.
 - 입력 조건 조합에 따른 출력을 결정한다.
- 테스트 데이터
 - 입력 조건을 모두 나열한다.
 - 조건에 따른 출력을 결정한다.
- 이점
 - 복잡한 논리적 관계를 표현하기 좋음
 - 누락된 요구사항 검사에 유용

테스트 조건		1	2	3	4
입력값	유효한 아이디	T	T	F	F
	유효한 비밀번호	T	F	T	F
예상 출력값	로그인 성공	T	F	F	F
	잘못된 아이디 경고창	F	F	T	T
	잘못된 비밀번호 경고창	F	T	F	F

테스트 케이스	입력값 조합	예상 동작
1번	둘 다 맞음 (T/T)	로그인 성공
2번	아이디 O / 비번 X	비밀번호 경고창
3번	아이디 X / 비번 O	아이디 경고창
4번	둘 다 틀림 (F/F)	아이디 경고창 (우선 순위가 아이디)

블랙박스 테스트(명세 기반 테스트) 주요 방법

의사 결정 테이블 혹은 결정표(decision table)

• Example 1 : 급여 계산

— 근무 시간과 시급

- $0 < \text{hour} < 80$
- $0 < \text{wage} < 100$

— 초과 수당 지급

- $\text{hour} > 40$ 이면 1.5배 지급

— 출력

- 정상급여, 초과수당
- 시간예러
- 시급예러

테스트 조건	1	2	3	4	5	6
$0 < \text{hour}$	T	T	T	F	T	T
$\text{hour} < 80$	T	T	F	T	T	T
$40 < \text{hour}$	F	T	T	F	x	x
$0 < \text{wage}$	T	T	T	T	T	F
$\text{wage} < 100$	T	T	T	T	F	T
출력	정상급여	초과수당	시간예러	시간예러	시급예러	시급예러

hour	wage	expected pay
30	40.00	1200.00
60	50.00	3500.00 (overtime)
90	60.00	invalid hour
50	100.00	invalid wage
...	...	...

블랙박스 테스트(명세 기반 테스트) 주요 방법

의사 결정 테이블 혹은 결정표(decision table)

- Example 2 : 온라인 주문

결재

- 현금 또는 신용카드(둘 다 결재하지 않는다)

우수고객

- 10% 할인

출력

- 주문 접수
- 주문 거부
- 10% 할인
- 정상 가격

		조건의 조합							
조건	현금 결제	Y	Y	Y	Y	N	N	N	N
	신용카드 결제	Y	Y	N	N	Y	Y	N	N
	우수 고객	Y	N	Y	N	Y	N	Y	N
출력	주문 접수			O	O	O	O		
	주문 거부							O	O
	10% 할인			O		O			
	정상 가격				O		O		

해당 사항 없음

		조건의 조합					
조건	현금 결제	Y	Y	N	N	N	
	신용카드 결제	N	N	Y	Y	N	
	우수 고객	Y	N	Y	N	-	
출력	주문 접수	O	O	O	O		
	주문 거부					O	
	10% 할인	O		O			
	정상 가격		O		O		

3 4 5 6 7

입력			예상 출력
현금결제	신용카드	우수고객	
Y	N	Y	10% 할인 현금 주문 접수
Y	N	N	정상 가격 현금 주문 접수
N	Y	Y	10% 할인 신용카드 주문 접수
N	Y	N	정상 가격 신용카드 주문 접수
N	N	-	주문 거부

블랙박스 테스트(명세 기반 테스트) 주요 방법

시나리오 테스트

- 기본 아이디어
 - 실 사용하는 방식에 기반하여 흐름을 테스트함
 - 시스템이 실 사용되는 프로세스 흐름에서 결함을 발견하는 데 유용함
 - End-to-End testing
 - 시스템 테스트, 인수 테스트 단계에서 주로 사용함
- 기본(main, basic) 시나리오
 - 가장 일반적이고 정상적인 상황을 나타내는 흐름
- 대체(alternative)시나리오
 - 기본 시나리오 외 시나리오
 - 사용자 혹은 시스템에 의해 선택적으로 수행하는 흐름
 - 비정상적인 사용, 시스템에서 발생하는 에러 등을 처리하는 흐름

블랙박스 테스트(명세 기반 테스트) 주요 방법

시나리오 테스트

• Example1 : ATM 현금 인출 시나리오

— 기본 시나리오

▪ 계좌에서 성공적으로 현금 인출

— 대체 시나리오

▪ 유효하지 않은 카드

▪ 비밀번호 입력 오류 2회

▪ 비밀번호 입력 오류 3회, 거래 정지

▪ 출금 대신 입금이나 계좌이체를 선택

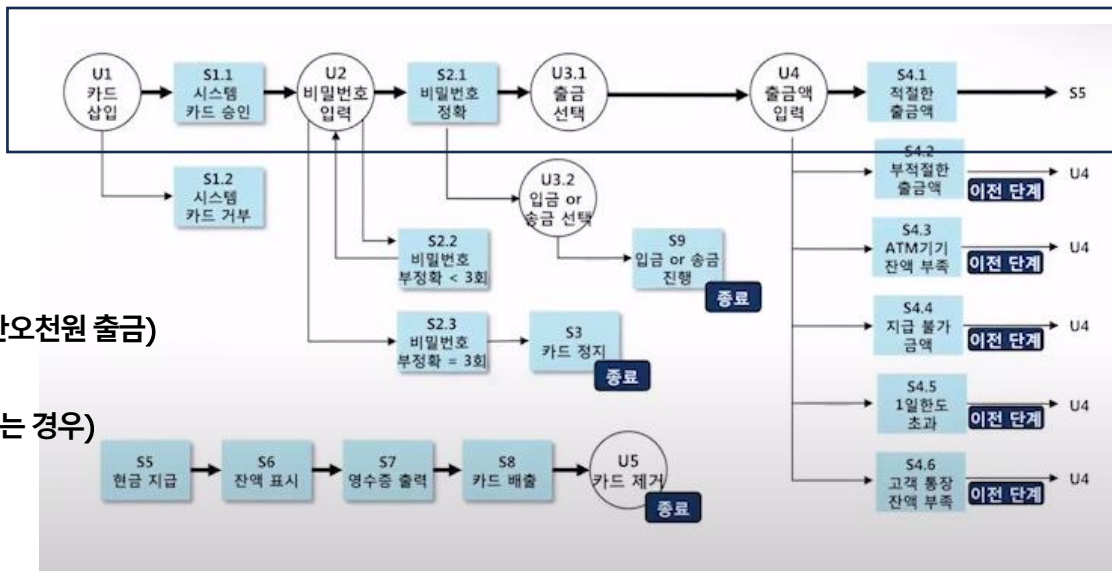
▪ 잘못된 출금 액 선택(예: 만원 단위로 출금이 가능하나, 만오천원 출금)

▪ ATM 기기의 잔액이 충분하지 않음

▪ 지급 권종 부족(예: 3만원 인출하고자 하나, 5만원권만 있는 경우)

▪ 1일 한도 초과

▪ 고객 계좌 잔액 부족



블랙박스 테스트(명세 기반 테스트) 주요 방법

시나리오 테스트

- Example1 : ATM 현금 인출 시나리오
- 기본 시나리오 테스트 케이스

항목	내용
테스트 케이스#	1
테스트 케이스 이름	현금인출 성공
시나리오 실행 경로	U1, S1.1, U2, S2.1, U3.1, U4, S4.1, S5, S6, S7, S8, U5
입력값	- 유효한 카드 - 293910982246 이 올바르다고 가정 - 유효한 비밀번호 - 1234 가 올바르다고 가정 - ATM 잔액 - \$50,000 - 고객 계좌 잔액 - \$100 - 출금액 - \$50
사전 조건	- 유효한 카드이고 올바른 비밀번호이며, ATM 기기가 정상동작 중일 때만 출금이 가능함 - 고객 계좌로부터 성공적으로 인출됨
기대 결과	- ATM 잔액 - \$49,950 - 고객 계좌 잔액 - \$50 - ATM 운영 중, 고객 카드 입력 대기 중

블랙박스 테스트(명세 기반 테스트) 주요 방법

시나리오 테스트

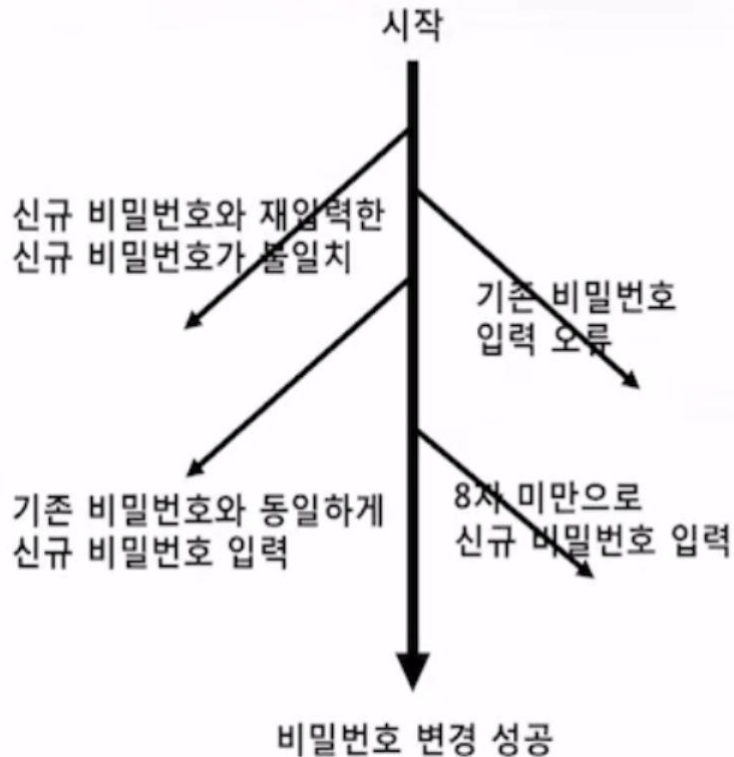
- Example1 : ATM 현금 인출 시나리오
- 대체 시나리오 테스트 케이스

항목	내용
테스트 케이스#	2
테스트 케이스 이름	유효하지 않은 카드
시나리오 실행 경로	U1, S1.2, S8, U5
입력값	유효하지 않은 카드
사전 조건	유효한 카드이고 올바른 비밀번호이며, ATM 기기가 정상동작 중일 때만 출금이 가능함
기대 결과	- 카드는 ATM 기기에서 거부되고 카드가 유효하지 않다는 에러 메시지가 표시됨 - 고객이 유효하지 않은 카드를 제거하도록 ATM 기기는 대기 중상태로 전환됨

블랙박스 테스트(명세 기반 테스트) 주요 방법

시나리오 테스트

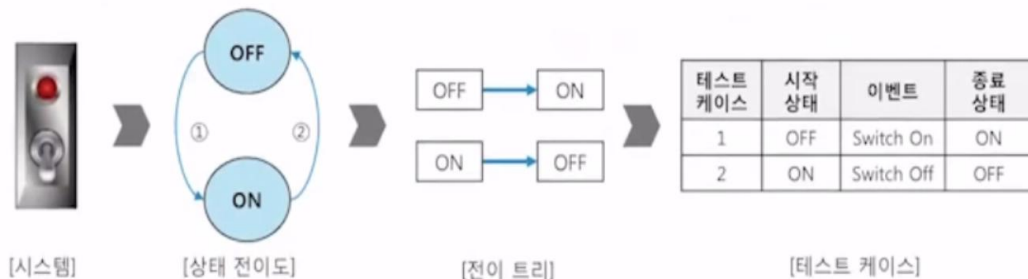
- Example2 : 비밀번호 변경 시나리오
 - 기본 시나리오
 - 비밀번호를 성공적으로 변경함(8자리 이상)
 - 대체 시나리오
 - 기존 비밀번호 입력 오류 발생함
 - 8자 미만으로 신규 비밀번호를 입력함
 - 기존 비밀번호와 동일하게 신규 비밀번호를 입력함
 - 신규 비밀번호와 재입력한 신규 비밀번호가 불일치함



블랙박스 테스트(명세 기반 테스트) 주요 방법

상태 전이도(state-transition diagram)

- 기본 아이디어
 - 시스템의 상태 전이도를 기반으로 테스트 데이터를 도출
- 테스트 데이터
 - 대상 시스템의 상태 전이도를 작성함
 - 상태 전이도를 이용해 전이 트리를 만들
 - 전이 트리로부터 각 전이를 테스트하는 테스트 케이스를 작성함



• 상태 커버리지

$$= \frac{\text{테스트 한 상태 수}}{\text{전체 상태 수}} \times 100\%$$

$$= \frac{2}{2} \times 100\% = 100\%$$

• 전이 커버리지

$$= \frac{\text{테스트 한 전이 수}}{\text{전체 전이 수}} \times 100\%$$

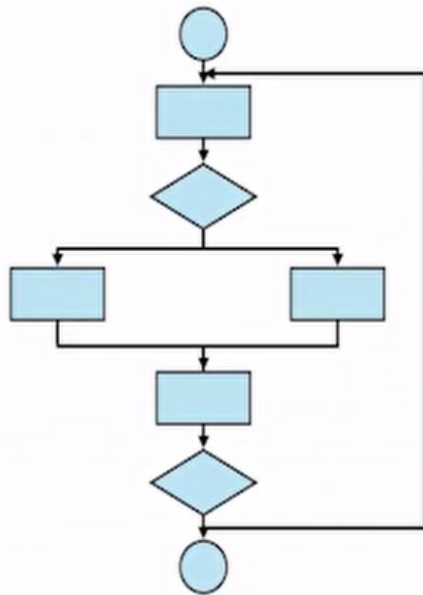
$$= \frac{2}{2} \times 100\% = 100\%$$

화이트박스(구조 기반 테스트) 테스트 주요 방법 분류

- **테스트 커버리지(Test Coverage)**란 소프트웨어의 코드, 기능, 요구 사항 등이 테스트를 통해 얼마나 검증되었는지를 측정하는 지표입니다. 테스트가 얼마나 철저하게 수행되었는지를 나타내는 지표로, 버그를 줄이고 소프트웨어의 품질을 높이는 데 중요한 역할을 합니다.
- 기초 경로(basic path)
 - 순환 복잡도(cydomatic complexity)를 이용하여 테스트하는 방법 · 테스트 대상 경로의 입출력 경로를 선택함으로써 테스트 커버리지를 달성하는 형태로 접근
- 제어 흐름
 - 프로그램의 제어 구조를 그래프 형태로 파악하고 그래프의 구성 요소로 문장(statement), 제어 블록(control block), 분기(branch), 조건(condition), 다중 조건(multiple condition) 등을 인식하여 테스트하는 방법
- 데이터 흐름(data flow)
 - 제어 흐름 그래프에 데이터의 사용 현황을 추가한 그래프 형태로 파악하여 테스트하는 방법

화이트박스(구조 기반 테스트) 테스트 주요 방법 분류

- 1) 문장 커버리지
 - 2) 결정(분기) 커버리지
 - 3) 조건 커버리지
 - 4) 결정/조건 커버리지
 - 5) 다중 조건 커버리지
 - 6) MC/DC
 - 7) 모든 경로 커버리지
- 기본 경로 테스트



구조 기반 테스트

= 화이트 박스 테스트

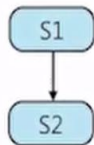
= 코드 기반 테스트

≠ 블랙 박스 테스트

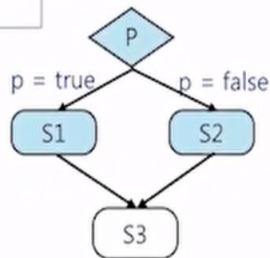
화이트박스(구조 기반 테스트) 테스트 주요 방법 분류

제어 흐름도(CFG, Control Flow Graph)

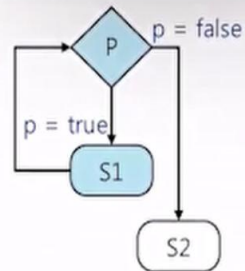
S1
S2



if p S1 else S2



while (p) S1

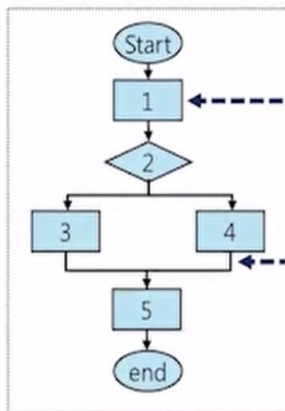


[프로그램 코드]

```

INT X = 1;
IF ( X > 0 ) THEN
    X = X - 1;
ELSE
    X = X + 1;
ENDIF
PRINT(X);
  
```

[제어 흐름도]



노드(Node)

* 노드: 프로그램 실행, 분기,
제어 흐름 시작과 끝

엣지(Edge)

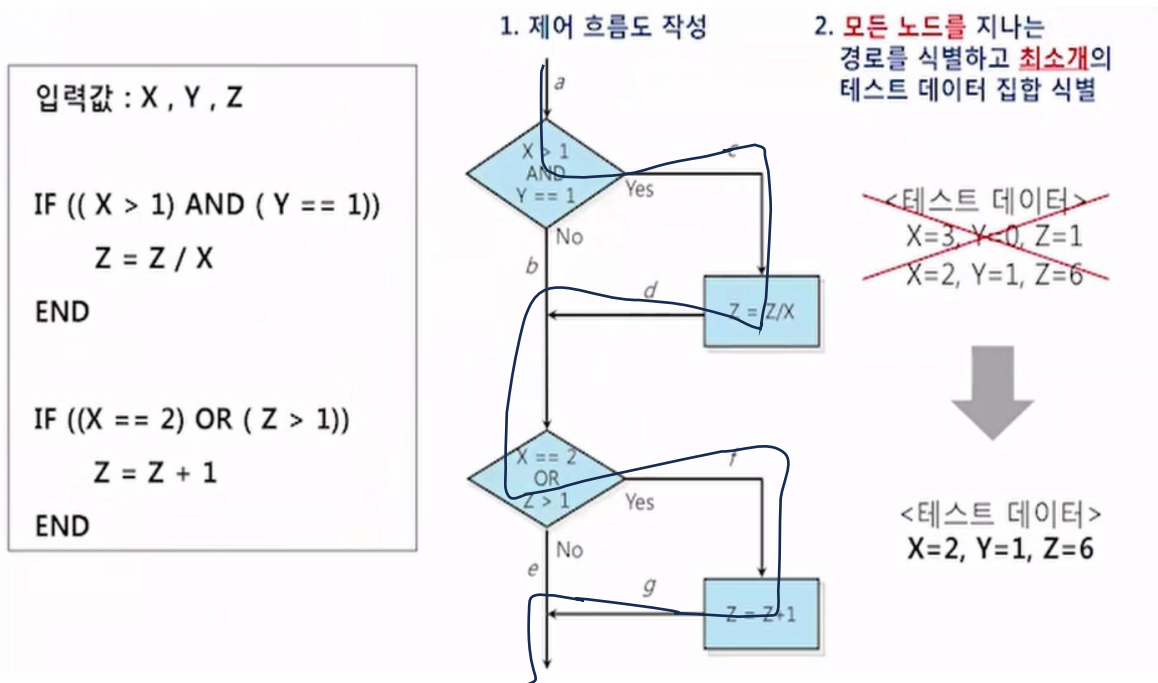
* 엣지: 노드 간 흐름

화이트박스(구조 기반 테스트) 테스트 주요 방법 분류

문장 커버리지

모든 코드의 문장(statement)이 최소 한 번 이상 실행되었는지를 측정하는 테스트 커버리지 지표

- Example1 : 다음 코드의 100% 문장 커버리지를 만족하는 테스트 데이터는?



화이트박스(구조 기반 테스트) 테스트 주요 방법 분류

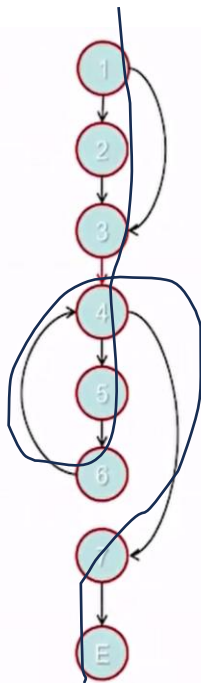
문장 커버리지

- Example2 :
- 다음 코드의 100% 문장 커버리지를 만족하는 테스트 데이터는?

```

if (x < 0) /* 1 */ {
    x := -x /* 2 */
};
z := 1; /* 3 */
while (x > 0) /* 4 */ {
    z := z * y; /* 5 */
    x := x - 1 /* 6 */
};
return z /* 7 */

```



한 경로이면 충분한다

[1,2,3,4,5,6,4,7,E]

이 경로를 지나게 할 최소 개 입력값은?

$x = -1$

$y = \text{어떤 값도 괜찮음}$

$z = \text{어떤 값도 괜찮음}$

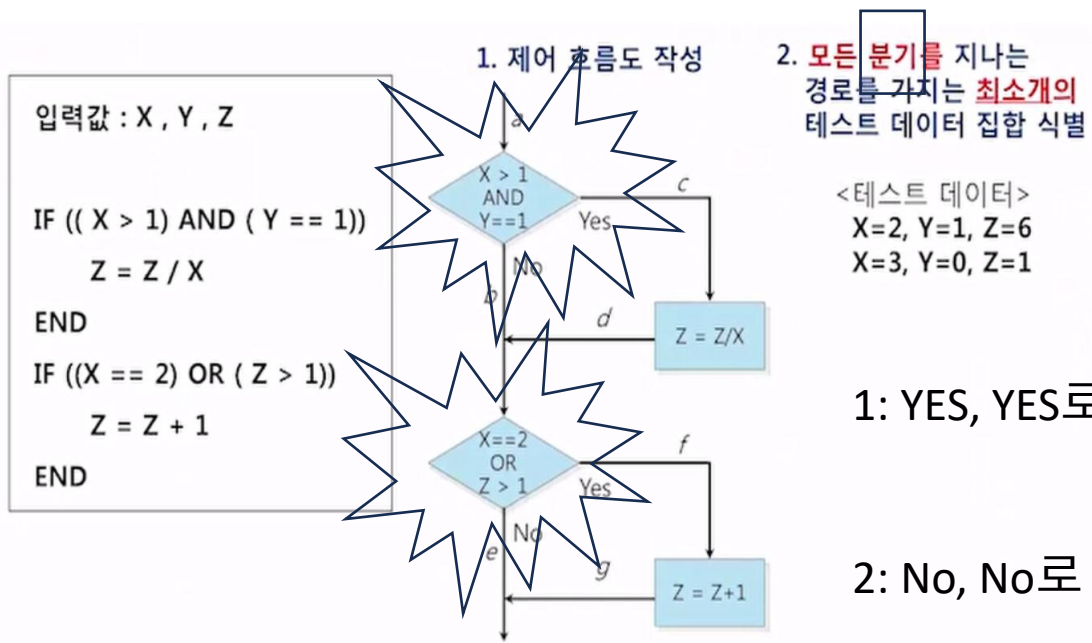
$(x=-1, y=2, z=3)$

화이트박스(구조 기반 테스트) 테스트 주요 방법 분류

결정 커버리지

if, while, for 등 분기 조건이 true/false 두 방향 모두 수행되었는지 확인

- Example1 : 다음 코드의 100% 결정 커버리지를 만족하는 테스트 데이터는?



화이트박스(구조 기반 테스트) 테스트 주요 방법 분류

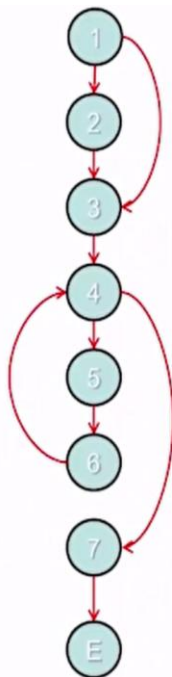
결정 커버리지

- Example2 :
- 다음 코드의 100% 결정커버리지를 만족하는 테스트 데이터는?

```

if (x < 0) /* 1 */ {
    x := -x /* 2 */
};
z := 1; /* 3 */
while (x > 0) /* 4 */ {
    z := z * y; /* 5 */
    x := x - 1 /* 6 */
};
return z /* 7 */

```



최소 두 경로가 있어야 한다.

$p1 = [1, 2, 3, 4, 5, 6, 4, 7, E]$

$p2 = [1, 3, 4, 7, E]$

이들 두 경로를 지나게 할 최 개 입력값은?

$p1 = (x=-1, y=*, z=*)$

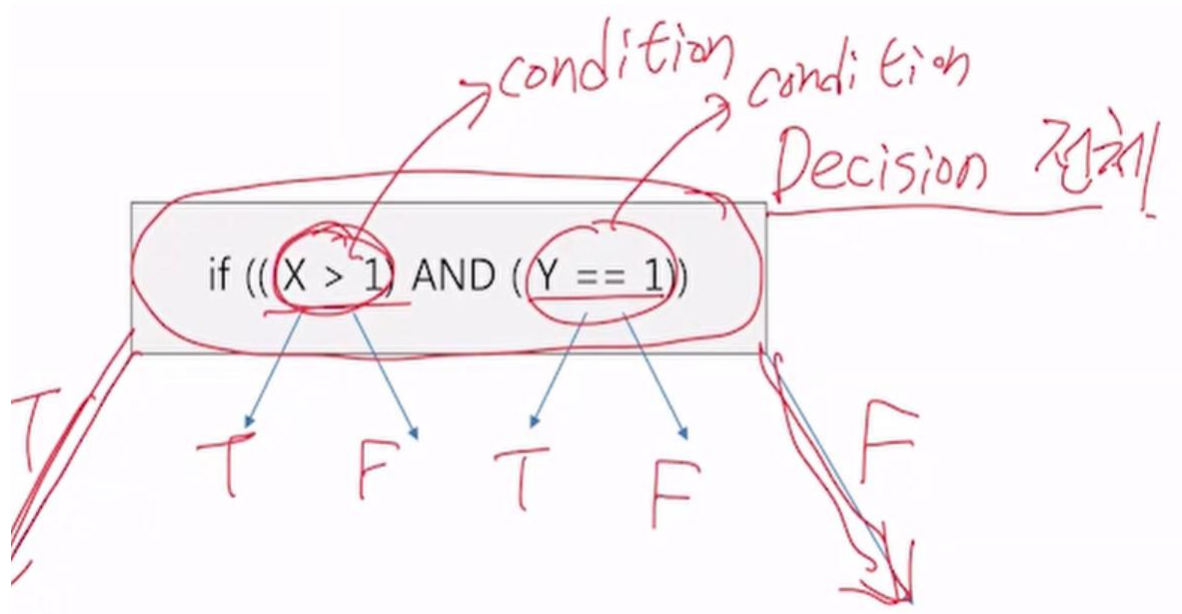
$p2 = (x=0, y=*, z=*)$

이들 데이터로 두 번 실행하면 결정커버리
지 만족한다.

화이트박스(구조 기반 테스트) 테스트 주요 방법 분류

결정과 조건

- 결정(Decision)
- 조건(Condition)



화이트박스(구조 기반 테스트) 테스트 주요 방법 분류

조건 커버리지

각 개별 조건(항)이 true, false가 한 번 이상 되는지 확인

- Example : 다음 코드의 100% 조건 커버리지를 만족하는 테스트 데이터는?

입력값 : X , Y , Z

```
IF (( X > 1) AND ( Y == 1))
```

```
    Z = Z / X
```

```
END
```

```
IF ((X == 2) OR ( Z > 1))
```

```
    Z = Z + 1
```

```
END
```

1. 개별 조건 식별

2. 모든 개별 조건의 T, F를 만족하는 최소 테스트 데이터 집합 식별

테스트 데이터	기본조건			
	X>1	Y==1	X==2	Z>1
X=2, Y=1, Z=6	true	true	true	true
X=0, Y=0, Z=1	false	false	false	false

만족

<테스트 데이터 집합>

X=2, Y=1, Z=6

X=0, Y=0, Z=1

화이트박스(구조 기반 테스트) 테스트 주요 방법 분류

결정/조건 커버리지

- Example :
 - 다음 코드의 100% 조건 커버리지를 만족하는 테스트 데이터는?

입력값 : X , Y , Z

```
IF (( X > 1) AND ( Y == 1))
```

```
    Z = Z / X
```

```
END
```

```
IF ((X == 2) OR ( Z > 1))
```

```
    Z = Z + 1
```

```
END
```

1. 개별 조건과 결정문의 분기를 식별
2. 전체 조건에 유의하여 모든 개별 조건의 T, F를 만족 하는 최소개의 테스트 데이터 집합 식별

테스트 데이터	(X>1) AND (Y==1)		(X==2) OR (Z>1)			
	기본조건		전체 조건	기본조건		전체 조건
	X>1	Y==1		X==2	Z>1	
X=2, Y=1, Z=6	true	true	true	true	true	true
x=0, y=0, z=1	false	false	false	false	false	false

만족

<테스트 데이터 집합>

X=2, Y=1, Z=6

X=0, Y=0, Z=1

화이트박스(구조 기반 테스트) 테스트 주요 방법 분류

다중 조건 커버리지(MCC:Multiple Condition Coverage)

- Example :
- 다음 코드의 100% 조건 커버리지를 만족하는 테스트 데이터는?

if (X>0 AND Y>0 AND Z>0)

TC	테스트 데이터			X>0	Y>0	Z>0
1	X=1	Y=1	Z=1	T	T	T
2	X=1	Y=1	Z=0	T	T	F
3	X=1	Y=0	Z=1	T	F	T
4	X=1	Y=0	Z=0	T	F	F
5	X=0	Y=1	Z=1	F	T	T
6	X=0	Y=1	Z=0	F	T	F
7	X=0	Y=0	Z=1	F	F	T
8	X=0	Y=0	Z=0	F	F	F

n	Multiple Conditions	Test time
1	2	2 ms
4	16	16 ms
8	256	256 ms
16	65,536	65.5 s
32	4,294,967,296	49.5 days

조건에 따라 테스트 할 수 있는 케이스 2^n 과 같다.

화이트박스(구조 기반 테스트) 테스트 주요 방법 분류

MC/DC (Modified Condition/Decision Coverage) 커버리지

- 정의
 - Modified Condition/Decision Coverage
 - 변형된 조건/결정 커버리지
 - 조건의 수 + 1개의 테스트 데이터 요구
 - 실무에서 사용되는 최고 수준의 테스트
- 활용
 - 자동차 분야 ISO 26262 ASIL D 등급 권고
 - 항공 분야 DO-178C 표준 Level A 등급 요구 사항

n	MCC	MC/DC
1	2	2
4	16	5
8	256	9
16	65,536	17
32	4,294,967,296	33

지수 2^n

선형 $n+1$

조건에 따라 테스트 할 수 있는 케이스 $n+1$ 과 같다.

화이트박스(구조 기반 테스트) 테스트 주요 방법 분류

MC/DC (Modified Condition/Decision Coverage) 커버리지

- Example 1:
- AND 연산자 : 조건 2
- T,F가 나오는 경우
- $N + 1 = 3$

한번은 y를 T로 고정

TC	(X>1)	(Y==1)	(X>1) AND (Y==1)
TC1	T	T	T
TC3	F	T	F

한번은 x를 T로 고정

TC	(X>1)	(Y==1)	(X>1) AND (Y==1)
TC1	T	T	T
TC2	T	F	F

IF ((X > 1) AND (Y == 1))			
TC	(X>1)	(Y==1)	(X>1) AND (Y==1)
TC1	T	T	T
TC2	T	F	F
TC3	F	T	F
TC4	F	F	F

TC	(X>1)	(Y==1)	(X>1) AND (Y==1)
TC1	T	T	T
TC2	T	F	F
TC3	F	T	F

화이트박스(구조 기반 테스트) 테스트 주요 방법 분류

MC/DC (Modified Condition/Decision Coverage) 커버리지

- Example 2:
 - OR 연산자 : 조건 2
 - T,F가 나오는 경우
 - $N + 1 = 3$

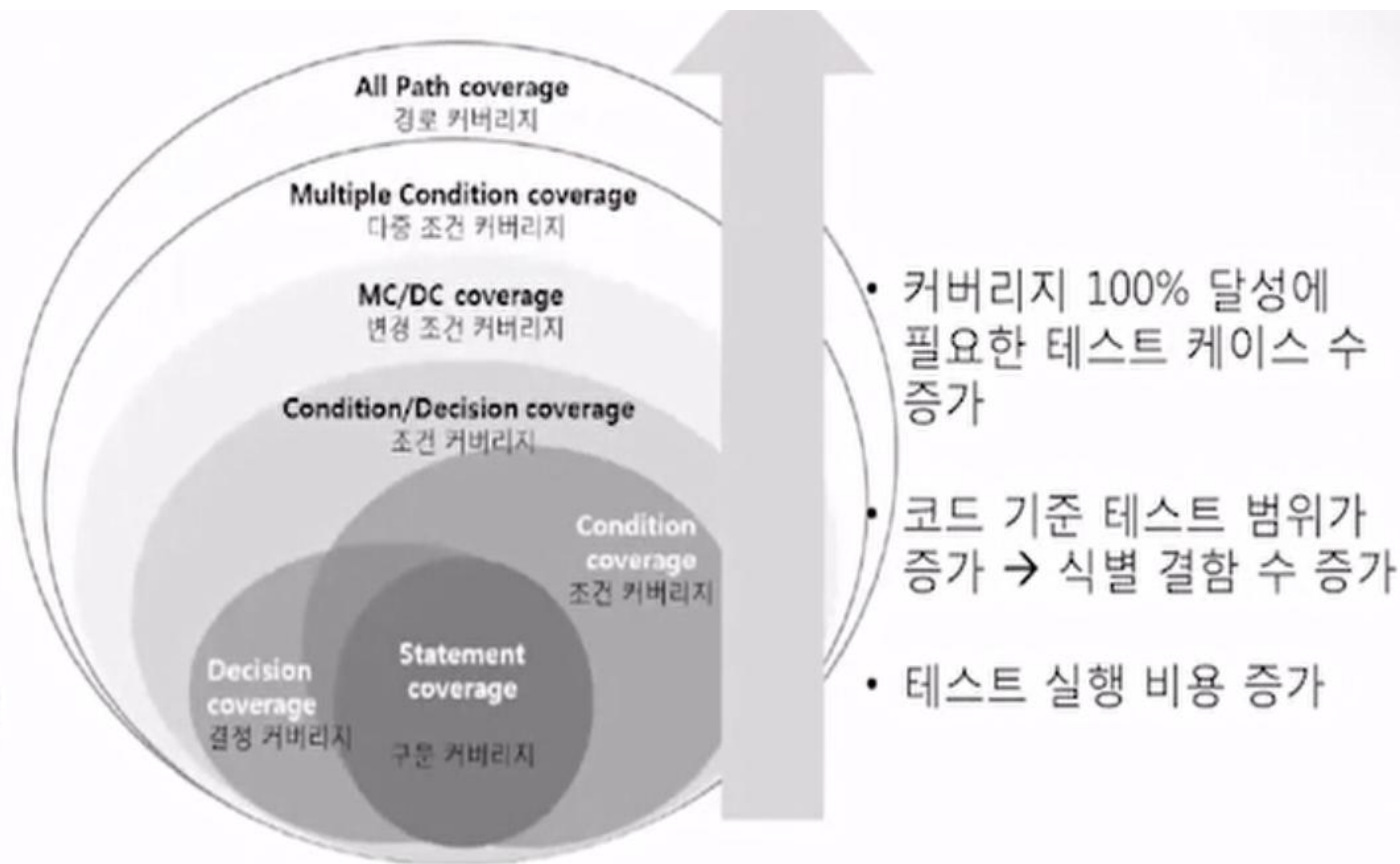
한번은 z를 F로
고정

IF ((X == 2) OR (Z > 1))

TC	(X==2)	(Z>1)	(X==1) OR (Z>1)
TC1	T	T	T
TC2	T	F	T
TC3	F	T	T
TC4	F	F	F

한번은 x를 F로
고정

커버리지간 포함 관계



소프트웨어 품질 관련 표준

표준구분	표준	표준 설명
소프트웨어 제품 품질	ISO/IEC 9126	품질 특성 및 측정 기준 제시, 계량화
	ISO/IEC 14598	ISO 9126에 따른 제품평가 표준: 반복성, 공정성, 객관성, 재생산성
	ISO/IEC 12119	패키지 소프트웨어 품질 요구사항 및 시험
	ISO/IEC 25000 (square)	소프트웨어 제품 평가를 위한 표준
	Good SW	정보통신기술협회 산하 소프트웨어 시험 센터 주관
소프트웨어 품질 일반	ISO/IEC 9000 Series	품질 경영과 품질 보증에 관한 국제 규격, ISO/IEC 9000패밀리의 선택과 사용지침
	6시그마	백만 개의 제품 중에서 10-6개의 오류를 목표로 하는 모토로라의 경영 혁신 기법

소프트웨어 품질 관련 표준

표준구분	표준	표준 설명
소프트웨어 프로세스 품질	ISO/IEC 12207	소프트웨어 생명주기 프로세스(Life Cycle Process, SDLC)를 기본, 지원, 조직 SDLC로 구분, 각각 수행 프로세스를 규정
	ISO/IEC 15504 (SPICE)	프로세스 표준 모델 소프트웨어 프로세스 요소(개발/지원/조직 등)의 계획, 통제, 개선 역량(capability)과 프로세스(process)로와 규명
	ISO/IEC 15288	시스템 생명주기 프로세서 (참조 모델 :PRM)
	CMMi	프로세스 표준 모델 (de facto standard / 사실 표준)
소프트웨어 서비스	ITIL	IT 서비스 관리 베스트 프랙티스(best practice)
	eSCM	IT 아웃소싱 품질 개선 모델

테스트 효과성 분석

APFD (Average Percentage of Faults Detected)

테스트 효과성은 어떠한 테스트 활동을 통해 목표했던 품질 수준에 도달하였는지의 여부를 나타낸다. 테스트 효과성은 테스트의 계획 및 설계, 테스트 활동을 위하여 사용된 데이터, 테스트 케이스, 측정 기법이 목표를 만족하였음을 의미한다.

- 발견된 결함의 평균율을 나타내며 회귀 테스트 효율성을 평가하기 위해 사용하는 기법이다. APFD 평가 기법은 우선순위화한 테스트 케이스 순서에서 결함이 발견되는 비율을 측정하는 방법이다. 즉, 결함이 누적적으로 발견되는 비율을 평균으로 나타내는 값이다. 테스트 케이스 수행 시 결함 발견율이 높을수록 해당 테스트 케이스의 효과성이 높다는 것을 의미한다. APFD는 “테스트를 얼마나 똑똑하게 순서화했는지”를 측정하는 지표

APFD 값이 높을수록 좋음
범위:

$$0 \leq APFD \leq 1$$

- T: N개의 테스트 케이스로 이루어진 테스트 스위트
- m: 테스트 하는 프로그램에 포함된 결함의 수
- n : 테스트 케이스의 전체 수
- F: m개의 결함을 포함한 결함의 집합
- TP_i : T에서 테스트 케이스 순서 중에서 결함 I를 노출시키는 첫 번째 테스트 케이스

$$APFD = 1 - \frac{\sum_{i=1}^m \frac{TF_i}{n \cdot m}}{2n}$$

AAA 패턴 — 좋은 테스트의 구조

모든 테스트는 Arrange(준비) → Act(실행) → Assert(검증) 3 단계로 명확히 구분해야 한다

Java — JUnit 5

Java

```
@Test
@DisplayName("점수 평균이 올바르게 계산된다")
void meanScore_shouldCalculateCorrectly() {
    // Arrange - 준비
    ScoreCollection collection = new
    ScoreCollection();
    collection.add() -> 5;
    collection.add() -> 7;
}
```

// Act — 실행

C++ — GoogleTest

C++

```
TEST(ScoreCollectionTest, MeanScore) {
    // Arrange
    ScoreCollection collection;
    collection.add([]{ return 5; });
    collection.add([]{ return 7; });

    // Act
    int result = collection.arithmeticMean();
}
```

// Assert

AAA 단계	Java (JUnit 5)	C++ (GoogleTest)	핵심 원칙
Arrange (준비)	@BeforeEach void setUp()	void SetUp() override	테스트 픽스처 분리
Act (실행)	메서드 호출 1줄	함수 호출 1줄	단 하나의 동작
Assert (검증)	assertEquals / assertThat	EXPECT_EQ / ASSERT_EQ	명확한 성공/실패

FIRST — 좋은 테스트의 5가지 조건

F — Fast

밀리초 단위 실행. DB·네트워크 없이 단위 수준에서 격리

@BeforeEach 로 인메모리 객체 초기화 | SetUp() + 인메모리 자료구조 활용

I — Isolated

테스트간 상태 공유 없음. @BeforeEach/@AfterEach 로 초기화

static 변수 공유 금지 | 각 TEST() 는 독립 픽스처

R — Repeatable

환경 무관 동일 결과. 시간·랜덤·네트워크에 의존 금지

@RepeatedTest | INSTANTIATE_TEST_SUITE_P

S — Self-validating

자동 판정. 사람이 눈으로 확인 불필요

assertEquals(expected, actual) |
EXPECT_EQ(expected, actual)

T — Timely

실제 코드 직전 또는 동시 작성 (TDD)

@Test 먼저 작성 → 빨간 막대 확인 | TEST() 먼저 → fail 확인

FIRST — 좋은 테스트의 5가지 조건

F (Fast) - 테스트는 빠르게 실행되어야 한다

- 테스트가 느리면 개발 속도를 저하시킬 수 있음.
- 빠른 실행을 위해 데이터베이스, 네트워크 호출을 최소화해야 함.
- 예제 (빠른 테스트)

```
@Test
```

```
void testAddition() {  
    assertEquals(10, calculator.add(5, 5)); // 단순한 연산 테스트 → 빠름  
}
```

- 안 좋은 예제 (느린 테스트)

```
@Test
```

```
void testWithDatabase() {  
    Database db = new Database();  
    db.connect();  
    assertEquals("Hello", db.getData());  
}
```

FIRST — 좋은 테스트의 5가지 조건

✓ 좋은 예제 (빠른 테스트)

```
#include <gtest/gtest.h>
```

```
// 간단한 Calculator 클래스
```

```
class Calculator {
```

```
public:
```

```
    int add(int a, int b) {
```

```
        return a + b;
```

```
    }
```

```
};
```

```
TEST(CalculatorTest, TestAddition) {
```

```
    Calculator calculator;
```

```
    EXPECT_EQ(10, calculator.add(5, 5)); // 단순한 연산 테스트 → 빠름
```

```
}
```

✗ 안 좋은 예제 (느린 테스트)

```
#include <gtest/gtest.h>
```

```
#include <string>
```

```
#include <thread>
```

```
#include <chrono>
```

```
// 느린 Database 클래스 (예시)
```

```
class Database {
```

```
public:
```

```
    void connect() {
```

```
        // 실제 DB 연결 대신 delay로 시뮬레이션
```

```
        std::this_thread::sleep_for(std::chrono::seconds(2));
```

```
    }
```

```
    std::string getData() {
```

```
        return "Hello";
```

```
    }
```

```
};
```

```
TEST(DatabaseTest, TestWithDatabase) {
```

```
    Database db;
```

```
    db.connect(); // 느린 외부 의존성
```

```
    EXPECT_EQ("Hello", db.getData());
```

```
}
```

• 빠른 테스트 (Good)

- 순수 함수 (Pure Function)
- 외부 의존성 없음
- 실행 속도 빠름 → TDD에 적합

• 느린 테스트 (Bad)

- DB, 네트워크 등 외부 의존성 존재
- 테스트 실행 느림
- 불안정(Flaky) 가능성 증가

FIRST — 좋은 테스트의 5가지 조건

I (Isolated/Independent) - 테스트는 서로 독립적이어야 한다

- 하나의 테스트가 다른 테스트에 영향을 주면 안 됨.
- 독립적인 테스트를 유지하기 위해 전역 변수, 공유 상태 사용을 피해야 함.
- 좋은 예제 (독립적인 테스트)

@Test

```
void testStringLength(){
    String str = "Hello";
    assertEquals(5, str.length()); // 독립적 테스트
}
```

- 안 좋은 예제 (의존성이 있는 테스트)

```
static int counter = 0; // 공유 상태 사용
```

@Test

```
void testIncrement(){
    counter++;
    assertEquals(1, counter);
}
```

@Test

```
void testDecrement(){
    counter--;
    assertEquals(-1, counter); // 이전 테스트에 따라 실패 가능
}
```

- 공유 상태(counter)로 인해 테스트 간 의존성이 생김 → 독립적으로 실행될 수 있도록 개선 필요.

FIRST — 좋은 테스트의 5가지 조건

✓ 좋은 예제 (독립적인 테스트)

```
#include <gtest/gtest.h>
#include <string>

TEST(StringTest, TestStringLength) {
    std::string str = "Hello";
    EXPECT_EQ(5, str.length()); // 독립적인 테스트
}
```

✗ 안 좋은 예제 (의존성 있는 테스트)

```
#include <gtest/gtest.h>

// 공유 상태 (Global 변수)
static int counter = 0;

TEST(CounterTest, TestIncrement) {
    counter++;
    EXPECT_EQ(1, counter);
}

TEST(CounterTest, TestDecrement) {
    counter--;
    EXPECT_EQ(-1, counter); // 실행 순서에 따라 실패 가능
}
```

FIRST — 좋은 테스트의 5가지 조건

개선된 코드 (독립성 확보)

```
#include <gtest/gtest.h>
```

```
class Counter {
```

```
public:
```

```
    int value = 0;
```

```
    void increment() { value++; }
```

```
    void decrement() { value--; }
```

```
};
```

```
TEST(CounterTest, TestIncrement) {
```

```
    Counter counter; // 각 테스트마다 새 인스턴스
```

```
    counter.increment();
```

```
    EXPECT_EQ(1, counter.value);
```

```
}
```

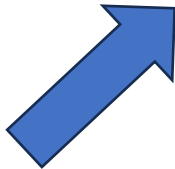
```
TEST(CounterTest, TestDecrement) {
```

```
    Counter counter; // 공유 상태 제거
```

```
    counter.decrement();
```

```
    EXPECT_EQ(-1, counter.value);
```

```
}
```



•문제점

- static 전역 변수 → 테스트 간 상태 공유
- 실행 순서에 따라 결과가 달라짐 (Flaky 테스트)

•해결 방법

- 테스트마다 새 객체 생성
- Shared state 제거
- Test Isolation 확보

FIRST — 좋은 테스트의 5가지 조건

R (Repeatable) - 테스트는 반복 실행해도 일관된 결과를 보장해야 한다

- 같은 입력이면 언제 어디서 실행하더라도 동일한 결과가 나와야 함.
- 외부 환경(네트워크, 시간, 랜덤 값 등)에 영향을 받지 않도록 설계해야 함.
- 좋은 예제 (일관된 결과 보장)

```
@Test
```

```
void testMultiplication() {
```

```
    assertEquals(20, calculator.multiply(4, 5)); // 동일한 결과 보장
```

```
}
```

- 안 좋은 예제 (환경에 따라 결과가 달라짐)

```
@Test
```

```
void testWithCurrentTime() {
```

```
    long timestamp = System.currentTimeMillis();
```

```
    assertEquals(1700000000000L, timestamp); // 실행할 때마다 값이 다름
```

```
}
```

- 현재 시간(System.currentTimeMillis())을 사용하여 테스트가 실패할 가능성이 있음.

FIRST — 좋은 테스트의 5가지 조건

✓ 좋은 예제 (결과가 항상 동일한 테스트)

```
#include <gtest/gtest.h>
```

```
// Calculator 클래스
```

```
class Calculator {
```

```
public:
```

```
    int multiply(int a, int b) {
```

```
        return a * b;
```

```
    }
```

```
};
```

```
TEST(CalculatorTest, TestMultiplication) {
```

```
    Calculator calculator;
```

```
    EXPECT_EQ(20, calculator.multiply(4, 5)); // 항상 동일한 결과
```

```
}
```

✗ 안 좋은 예제 (환경에 따라 결과가 달라지는 테스트)

```
#include <gtest/gtest.h>
```

```
#include <chrono>
```

```
TEST(TimeTest, TestWithCurrentTime) {
```

```
    auto now = std::chrono::system_clock::now();
```

```
    auto timestamp = std::chrono::duration_cast<std::chrono::milliseconds>(
```

```
        now.time_since_epoch()
```

```
    ).count());
```

```
    EXPECT_EQ(1700000000000LL, timestamp); // 실행할 때마다 값이 달라짐 → 실패 가능
```

```
}
```

FIRST — 좋은 테스트의 5가지 조건

개선 방법 (결정론적 테스트로 변경)

```
#include <gtest/gtest.h>
```

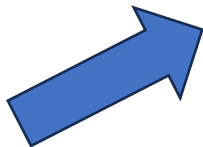
```
// 시간 제공자 인터페이스
```

```
class TimeProvider {
```

```
public:
```

```
    virtual long long now() = 0;
```

```
};
```



```
// Mock/ Fake 구현
```

```
class FakeTimeProvider : public TimeProvider {
```

```
public:
```

```
    long long fixedTime;
```

```
    FakeTimeProvider(long long t) : fixedTime(t) {}
```

```
    long long now() override {
```

```
        return fixedTime;
```

```
    }
```

```
};
```

```
TEST(TimeTest, TestWithFixedTime) {
```

```
    FakeTimeProvider timeProvider(1700000000000LL);
```

```
    EXPECT_EQ(1700000000000LL, timeProvider.now()); // 항상 동일
```

```
}
```

- ❌ 문제
- 현재 시간, 랜덤, 외부 환경 → 테스트 결과 변함
- ✅ 해결
- 시간/랜덤을 주입(Dependency Injection)
- Mock/Fake 사용
- 항상 같은 입력 → 같은 결과 (Deterministic)

FIRST — 좋은 테스트의 5가지 조건

S (Self-validating) - 테스트 결과는 명확한 성공/실패 기준을 가져야 한다

- 테스트는 자동으로 결과를 검증해야 하며, 사람이 직접 확인하지 않아도 됨.
- 테스트 실패 시 원인을 명확히 알 수 있어야 함.
- 좋은 예제 (명확한 검증)

```
@Test
```

```
void testDivision() {
```

```
    assertEquals(2, calculator.divide(10, 5)); // 예상 결과와 비교
```

```
}
```

- 안 좋은 예제 (수동 확인 필요)

```
@Test
```

```
void testPrintMessage() {
```

```
    System.out.println("Hello, World!"); // 출력 확인이 필요함 → 자동 검증 없음
```

```
}
```

- 출력 결과를 사람이 직접 확인해야 하는 테스트는 자동화되지 않은 테스트.

FIRST — 좋은 테스트의 5가지 조건

✓ 좋은 예제 (자동 검증 가능한 테스트)

```
#include <gtest/gtest.h>
```

```
// Calculator 클래스
```

```
class Calculator {
```

```
public:
```

```
    int divide(int a, int b) {
```

```
        return a / b;
```

```
    }
```

```
};
```

```
TEST(CalculatorTest, TestDivision) {
```

```
    Calculator calculator;
```

```
    EXPECT_EQ(2, calculator.divide(10, 5)); // 기대값과 비교 (자동 검증)
```

```
}
```

✗ 안 좋은 예제 (수동 확인 필요)

```
#include <gtest/gtest.h>
```

```
#include <iostream>
```

```
TEST(OutputTest, TestPrintMessage) {
```

```
    std::cout << "Hello, World!" << std::endl;
```

```
    // 출력만 있음 → 검증 없음 → 테스트로서 의미 없음
```

```
}
```

FIRST — 좋은 테스트의 5가지 조건

개선된 코드 (출력도 테스트 가능하게)

```
#include <gtest/gtest.h>
```

```
#include <sstream>
```

```
#include <iostream>
```

```
TEST(OutputTest, TestPrintMessage) {
```

```
    std::ostringstream buffer;
```

```
    std::streambuf* old = std::cout.rdbuf(buffer.rdbuf()); // cout  
리다이렉트
```

```
    std::cout << "Hello, World!";
```

```
    std::cout.rdbuf(old); // 복구
```

```
    EXPECT_EQ("Hello, World!", buffer.str()); // 자동 검증
```

```
}
```

- ❌ 문제
 - cout, print만 있는 테스트 → 검증(assert)이 없음
 - 사람이 직접 확인해야 함 → 자동화 실패
- ✅ 해결
 - 출력도 캡처해서 비교
 - 반드시 EXPECT_EQ, ASSERT_*로 검증

FIRST — 좋은 테스트의 5가지 조건

T (Timely) - 테스트는 개발 과정에서 적절한 시점에 작성되어야 한다

- 테스트는 개발이 끝난 후가 아니라, 코드 작성과 동시에 또는 그 이전에 작성해야 함.
- TDD(Test-Driven Development, 테스트 주도 개발) 방식이 이상적.
- 좋은 예제 (테스트 선 작성)

@Test

```
void testSquare() {
```

```
    assertEquals(25, Math.pow(5, 2)); // 기능 구현 전 테스트 작성
```

```
}
```

- 안 좋은 예제 (기능이 완성된 후 테스트 작성)

// 이미 개발 완료된 후 테스트 추가

```
public int square(int x) {
```

```
    return x * x;
```

```
}
```

- 기능 개발 후 테스트 추가하면, 설계 결함을 사전에 발견하기 어려움.

FIRST — 좋은 테스트의 5가지 조건

✓ 좋은 예제 (TDD: 테스트 먼저 작성)

```
#include <gtest/gtest.h>
```

```
#include <cmath>
```

```
TEST(MathTest, TestSquare) {
```

```
    EXPECT_EQ(25, std::pow(5, 2)); // 기능 구현 전에 테스트 작성
```

```
}
```

✗ 안 좋은 예제 (구현 후 테스트 작성)

```
#include <gtest/gtest.h>
```

```
// 이미 구현된 코드 (테스트보다 먼저 작성됨)
```

```
int square(int x) {
```

```
    return x * x;
```

```
}
```

```
// 나중에 추가된 테스트
```

```
TEST(MathTest, TestSquare) {
```

```
    EXPECT_EQ(25, square(5));
```

```
}
```


FIRST — 좋은 테스트의 5가지 조건

TDD 방식으로 개선 (RED → GREEN)

```
#include <gtest/gtest.h>
```

```
// 1. RED: 먼저 실패하는 테스트 작성
```

```
TEST(MathTest, TestSquare) {  
    EXPECT_EQ(25, square(5)); // 아직 구현 안됨 → 실패  
}
```

```
// 2. GREEN: 최소 구현
```

```
int square(int x) {  
    return x * x;  
}
```

- ❌ 문제
 - 기능 구현 후 테스트 작성 → 검증이 아닌 사후 확인
 - ✅ TDD 원칙
1. **RED**: 실패하는 테스트 먼저 작성
 2. **GREEN**: 테스트 통과 최소 구현
 3. **REFACTOR**: 구조 개선

Right-BICEP & CORRECT — 무엇을 테스트할 것인가?

Right-BICEP	설명
R — Right	결과가 옳은가? assertEquals(10, add(5,5))
B — Boundary	모든 경계조건이 맞는가? MAX_VALUE, 0, null
I — Inverse	역관계 확인? sort → reverse 후 원본
C — Cross-check	다른 수단으로 교차 검증?
E — Error	에러 조건 처리? assertThrows
P — Performance	성능 조건? assertTimeout

CORRECT	설명
C — Conformance	형식 준수: 이메일·날짜·JSON 형식
O — Ordering	정렬·순서: 오름차순 보장 여부
R — Range	범위: 나이 0~150, 배열 인덱스
R — Reference	참조 유효성: 없는 ID·삭제된 객체
E — Existence	존재 여부: null·빈 문자열·0
C — Cardinality	개수: 최소 1개, 최대 100개
T — Time	시간: 만료·타임아웃·동기화

```

// Java — CORRECT 경계 조건 종합 예시
@Test void testConformance() { assertThrows(IllegalArgumentException.class, () -> new Email("invalid")); } // C
@Test void testRange()      { assertThrows(IllegalArgumentException.class, () -> new Age(-1)); } // R
@Test void testExistence()  { assertTrue(userRepo.findById(999L).isEmpty()); } // E
@Test void testCardinality() { assertEquals(0, cart.getItems().size()); } // C

```

테스트의 6가지 영역: Right-BICEP

1. 정확성 테스트 (Right)

```
#include <gtest/gtest.h>
```

```
class Calculator {
public:
    int add(int a, int b) { return a + b; }
};
```

```
TEST(CalculatorTest, TestAdditionCorrectness) {
    Calculator calculator;
    EXPECT_EQ(10, calculator.add(5, 5)); // 5 + 5 = 10 검증
}
```

2. 경계값 테스트 (Boundary / Overflow)

```
#include <limits>
```

```
TEST(CalculatorTest, TestOverflow) {
    Calculator calculator;
    int result = calculator.add(std::numeric_limits<int>::max(), 1);

    // C++에서는 overflow가 정의되지 않음 (UB)
    // 실제 테스트에서는 safetype 또는 체크 필요
    EXPECT_LT(result, 0); // overflow 발생 가정 (주의: 환경 의존)
}
```

현실적으로는 int 대신 long long 또는 overflow 체크 로직을 넣는 게 맞습니다.

테스트의 6가지 영역: Right-BICEP

3. 역관계 테스트 (Inverse Relationship)

```
class Calculator {  
    public:  
        int add(int a, int b) { return a + b; }  
        int subtract(int a, int b) { return a - b; }  
};  
  
TEST(CalculatorTest, TestInverseRelationship) {  
    Calculator calculator;  
    int result = calculator.add(5, 5);  
    EXPECT_EQ(5, calculator.subtract(result, 5));  
}
```

4. 교차 검증 (Commutative Law)

```
TEST(CalculatorTest, TestCommutativeProperty) {  
    Calculator calculator;  
    EXPECT_EQ(calculator.add(2, 3), calculator.add(3, 2));  
}
```

테스트의 6가지 영역: Right-BICEP

5. 예외 처리 테스트 (Error Handling)

```
#include <stdexcept>
```

```
class Calculator {
```

```
public:
```

```
    int divide(int a, int b) {
```

```
        if (b == 0) {
```

```
            throw std::invalid_argument("division by zero");
```

```
        }
```

```
        return a / b;
```

```
    }
```

```
};
```

```
TEST(CalculatorTest, TestDivideByZero) {
```

```
    Calculator calculator;
```

```
    EXPECT_THROW(calculator.divide(10, 0), std::invalid_argument);
```

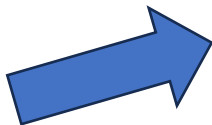
```
}
```

테스트의 6가지 영역: Right-BICEP

6. 성능 테스트 (Performance)

```
#include <chrono>
```

```
class Calculator {  
public:  
    long long factorial(int n) {  
        long long result = 1;  
        for (int i = 1; i <= n; ++i) {  
            result *= i;  
        }  
        return result;  
    }  
};
```



```
TEST(CalculatorTest, TestPerformance) {  
    Calculator calculator;  
  
    auto start = std::chrono::high_resolution_clock::now();  
    calculator.factorial(10000);  
    auto end = std::chrono::high_resolution_clock::now();  
  
    auto duration =  
        std::chrono::duration_cast<std::chrono::nanoseconds>(end - start).count();  
  
    EXPECT_LT(duration, 1000000); // 1ms 이하  
}
```

테스트의 6가지 영역: Right-BICEP

1. Right (정확한 결과 확인)

```
assertEquals(10, calculator.add(5, 5));
```

- $5 + 5 = 10$ 이 정확한 결과인지 검증.

2. Boundary conditions (경계값 테스트)

```
assertEquals(0, calculator.add(Integer.MAX_VALUE, 1)); // Overflow 테스트
```

- 최대값 경계 테스트.

3. Inverse relationships (역관계 테스트)

```
int result = calculator.add(5, 5);
```

```
assertEquals(5, calculator.subtract(result, 5));
```

- $5 + 5 = 10$ 이후 $10 - 5 = 5$ 로 원래 값으로 되돌아오는지 확인.

4. Cross-checks (교차 확인)

```
assertEquals(calculator.add(2, 3), calculator.add(3, 2));
```

- 순서에 관계없이 결과가 동일한지 확인 (덧셈의 교환법칙).

테스트의 6가지 영역: Right-BICEP

5. Error conditions (에러 핸들링 테스트)

```
assertThrows(IllegalArgumentException.class, () -> calculator.divide(10, 0));
```

- 0으로 나누었을 때 예외 발생을 확인.

6. Performance (성능 테스트)

```
long startTime = System.nanoTime();
```

```
calculator.factorial(10000);
```

```
long endTime = System.nanoTime();
```

```
assertTrue((endTime - startTime) < 1000000); // 1ms 이하로 실행되어야 함
```

- 특정 연산이 정해진 시간 내 실행되는지 테스트.

CORRECT 경계 조건 검증

C (Conformance, 형식 준수)

- 데이터가 올바른 형식을 따르는지 검증
- 문자열, 이메일, 날짜 형식, JSON/XML 스키마 등 확인
- 예제 (이메일 형식 검증)

@Test

```
void testEmailFormat() {  
    assertTrue(UserValidator.isValidEmail("test@example.com")); // 유효한 이메일  
    assertFalse(UserValidator.isValidEmail("invalid-email")); // 잘못된 형식  
}
```

CORRECT 경계 조건 검증

✓ C++ 코드 (이메일 검증 테스트)

```
#include <gtest/gtest.h>
```

```
#include <regex>
```

```
#include <string>
```

```
class UserValidator {
```

```
public:
```

```
    static bool isValidEmail(const std::string& email) {
```

```
        // 간단한 이메일 정규식 (기본 검증용)
```

```
        const std::regex pattern(R"((\w+)(\.?w+)*@(\w+)(\.\w+)+)");
```

```
        return std::regex_match(email, pattern);
```

```
    }
```

```
};
```

```
TEST(UserValidatorTest, TestEmailFormat) {
```

```
    EXPECT_TRUE(UserValidator::isValidEmail("test@example.com")); // 정상 이메일
```

```
    EXPECT_FALSE(UserValidator::isValidEmail("invalid-email")); // 잘못된 형식
```

```
}
```

- assertTrue → EXPECT_TRUE
- assertFalse → EXPECT_FALSE
- Java static 메서드 → C++ static 함수
- 이메일 검증은 std::regex 사용

CORRECT 경계 조건 검증

O (Ordering, 정렬/순서)

- 데이터가 올바르게 정렬되었는지 확인 (예: 오름차순/내림차순 정렬이 유지되는지)
- 정렬이 필요한 리스트, 날짜, ID 값 등의 순서 검증
- 예제 (날짜 정렬 검증)

@Test

```
void testDateOrdering() {  
    LocalDate startDate = LocalDate.of(2024, 1, 1);  
    LocalDate endDate = LocalDate.of(2024, 12, 31);  
    assertTrue(startDate.isBefore(endDate)); // 올바른 순서 -> startDate 가 endDate 보다 앞인지 확인  
}
```

CORRECT 경계 조건 검증

✓ C++ 코드 (날짜 순서 검증)

```
#include <gtest/gtest.h>
```

```
#include <time>
```

```
std::time_t makeTime(int y, int m, int d) {
```

```
    std::tm t = {};
```

```
    t.tm_year = y - 1900;
```

```
    t.tm_mon = m - 1;
```

```
    t.tm_mday = d;
```

```
    return std::mktime(&t);
```

```
}
```

```
TEST(DateTest, TestDateOrdering) {
```

```
    std::time_t start = makeTime(2024, 1, 1);
```

```
    std::time_t end = makeTime(2024, 12, 31);
```

```
    EXPECT_LT(start, end);
```

```
}
```

• Java LocalDate → C++ std::chrono::year_month_day

• isBefore() → EXPECT_LT(start, end)

• 비교를 위해 sys_days로 변환 필요

CORRECT 경계 조건 검증

R (Range, 범위 검증)

- 입력 값이 허용된 최소/최대 범위를 벗어나지 않는지 확인
- 음수 입력, 배열 인덱스 초과 등 예외 처리
- 예제 (나이 범위 검증)

```
@Test
```

```
void testAgeRange() {
```

```
    assertTrue(UserValidator.isValidAge(25)); // 0~120 사이 → 유효
```

```
    assertFalse(UserValidator.isValidAge(-1)); // 음수 → 유효하지 않음
```

```
    assertFalse(UserValidator.isValidAge(150)); // 너무 큰 값 → 유효하지 않음
```

```
}
```

CORRECT 경계 조건 검증

✓ C++ 코드 (나이 범위 검증)

```
#include <gtest/gtest.h>
```

```
class UserValidator {
```

```
public:
```

```
    static bool isValidAge(int age) {
```

```
        return age >= 0 && age <= 120; // 0 ~ 120 사이만 유효
```

```
    }
```

```
};
```

```
TEST(UserValidatorTest, TestAgeRange) {
```

```
    EXPECT_TRUE(UserValidator::isValidAge(25)); // 정상 범위
```

```
    EXPECT_FALSE(UserValidator::isValidAge(-1)); // 음수 → invalid
```

```
    EXPECT_FALSE(UserValidator::isValidAge(150)); // 너무 큼 → invalid
```

```
}
```

이 테스트는 Boundary Value Test (경계값 테스트)의 대표적인 예:

정상값: 25

하한 초과: -1

상한 초과: 150

실제로는 아래도 추가하면 더 완벽함:

```
EXPECT_TRUE(UserValidator::isValidAge(0)); // 하한
```

```
EXPECT_TRUE(UserValidator::isValidAge(120)); // 상한  
한 줄 핵심
```

"경계값(0, 120)을 반드시 포함해서 테스트해야 한다"

CORRECT 경계 조건 검증

R (Reference, 참조 유효성)

- 외부 참조가 올바른지 검증 (예: 외부 ID, 데이터베이스 참조값 등)
- 삭제된 객체나 잘못된 참조값을 사용하지 않는지 확인
- 예제 (존재하지 않는 사용자 참조 검증)

@Test

```
void testReferenceValidation() {  
    User user = database.findUserById(9999); // 없는 사용자 ID  
    assertNull(user); // 존재하지 않는 ID → NULL 반환  
}
```

CORRECT 경계 조건 검증

✓ C++ 코드 (참조 무결성 / NULL 검증)

```
#include <gtest/gtest.h>
```

```
// 간단한 User 클래스
```

```
class User {
public:
    int id;
    User(int id) : id(id) {}
};
```

```
// Database 클래스 (예시)
```

```
class Database {
public:
    User* findUserById(int id) {
        // 존재하지 않는 ID라면 nullptr 반환
        return nullptr;
    }
};
```

```
TEST(DatabaseTest, TestReferenceValidation) {
    Database database;
```

```
    User* user = database.findUserById(9999); // 존재하지 않는 ID
    EXPECT_EQ(nullptr, user); // NULL (nullptr) 검증
}
```

- Java null → C++ nullptr
- assertNull → EXPECT_EQ(nullptr, ...)
- 포인터 기반 반환 (User*)으로 NULL 표현
- "존재하지 않는 객체는 nullptr 또는 optional로 명확히 표현하라"

CORRECT 경계 조건 검증

E (Existence, 존재 여부)

- 필수 필드가 누락되지 않았는지 검증 (예: 필수 입력값 체크, NULL 방지)
- 예제 (필수 입력값 검증)

```
@Test
```

```
void testRequiredFields() {
```

```
    User user = new User(null, "password123"); // 이름이 없음
```

```
    assertFalse(UserValidator.isValid(user)); // 유효하지 않음
```

```
}
```

CORRECT 경계 조건 검증

✓ C++ 코드 (필수 필드 검증)

```
#include <gtest/gtest.h>
```

```
#include <string>
```

```
// User 클래스
```

```
class User {
```

```
public:
```

```
    std::string name;
```

```
    std::string password;
```

```
    User(const std::string& name, const std::string& password)
```

```
        : name(name), password(password) {}
```

```
};
```

```
// Validator 클래스
```

```
class UserValidator {
```

```
public:
```

```
    static bool isValid(const User& user) {
```

```
        // 이름이 비어있으면 invalid
```

```
        return !user.name.empty();
```

```
    }
```

```
};
```

```
TEST(UserValidatorTest, TestRequiredFields) {
```

```
    User user("", "password123"); // 이름 없음 (Java null → C++에서는 빈 문자열로 표현)
```

```
    EXPECT_FALSE(UserValidator::isValid(user)); // 유효하지 않음
```

```
}
```

- Java null → C++에서는 보통 빈 문자열 "" 또는 std::optional
- "값 자체가 없음"을 표현하려면 👉 std::optional이 더 정확
- assertFalse → EXPECT_FALSE
- 필수 필드 검증: empty() 체크

CORRECT 경계 조건 검증

C (Cardinality, 데이터 개수)

- 배열, 컬렉션의 최소/최대 크기 확인 (예: 최소 1개, 최대 100개 등 제약 조건 검증)
- 예제 (리스트 크기 검증)

```
@Test
```

```
void testListSize() {
```

```
    List<String> names = Arrays.asList("Alice", "Bob");
```

```
    assertTrue(names.size() >= 1 && names.size() <= 100); // 1~100 사이
```

```
}
```

CORRECT 경계 조건 검증

✓ C++ 코드 (리스트 크기 범위 검증)

```
#include <gtest/gtest.h>
```

```
#include <vector>
```

```
#include <string>
```

```
TEST(ListTest, TestListSize) {
```

```
    std::vector<std::string> names = {"Alice", "Bob"};
```

```
    EXPECT_TRUE(names.size() >= 1 && names.size() <= 100); // 1 ~ 100 사이
```

```
}
```

- Java List<String> → C++ std::vector<std::string>

- Arrays.asList(...) → {...} 초기화 리스트

- assertTrue → EXPECT_TRUE

- size()는 C++에서도 동일하게 사용 가능

- 이 테스트는 Boundary Value Test (경계값 테스트)

- 최소값: 1

- 최대값: 100

- 실제로는 아래도 추가하면 더 완벽:

- EXPECT_EQ(1, names.size()); // 최소

- EXPECT_EQ(100, names.size()); // 최대 케이스도 별도 테스트 추천

CORRECT 경계 조건 검증

T (Time, 시간 관련 검증)

- 시간 제한, 만료 시간, 동기화 이슈 테스트 (예: API 응답 시간, 캐시 만료 등)
- 예제 (응답 시간 검증)

@Test

```
void testResponseTime() {  
    long startTime = System.currentTimeMillis();  
    api.callEndpoint();  
    long elapsedTime = System.currentTimeMillis() - startTime;  
    assertTrue(elapsedTime < 1000); // 응답 시간이 1초 이내여야 함  
}
```

CORRECT 경계 조건 검증

✓ C++ 코드 (응답 시간 테스트)

```
#include <gtest/gtest.h>
```

```
#include <chrono>
```

```
// 예시 API 클래스
```

```
class API {
```

```
public:
```

```
    void callEndpoint() {
```

```
        // 실제 API 호출 대신 더미 처리
```

```
    }
```

```
};
```

```
TEST(APITest, TestResponseTime) {
```

```
    API api;
```

```
    auto startTime = std::chrono::steady_clock::now();
```

```
    api.callEndpoint();
```

```
    auto endTime = std::chrono::steady_clock::now();
```

```
    auto elapsedTime = std::chrono::duration_cast<std::chrono::milliseconds>(
```

```
        endTime - startTime
```

```
    ).count();
```

```
    EXPECT_LT(elapsedTime, 1000); // 1초(1000ms) 이내
```

```
}
```

- System.currentTimeMillis() → std::chrono::steady_clock
- 시간 차이 계산 → duration_cast<std::chrono::milliseconds>
- assertTrue → EXPECT_LT

테스트 케이스?

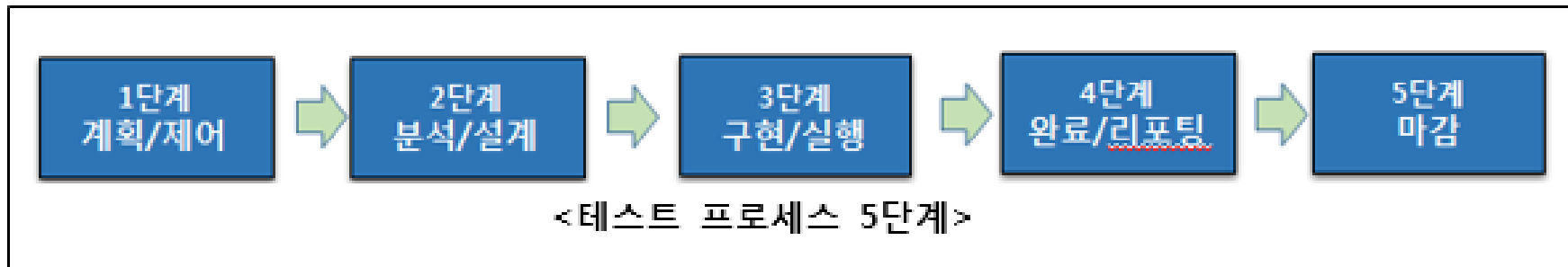
테스트 케이스(Test Case)의 개념

- 테스트 케이스란 요구 사항을 준수하는지 검증하기 위하여 테스트 조건, 입력값, 예상 출력값 및 수행한 결과 등 테스트 조건을 명세한 것이다.
- 테스트 케이스는 테스트 대상인 특정 프로그램 경로의 실행, 요구사항과 일치 여부 확인을 위해 디자인된 입력값의 묶음, 실행 사전 조건, 기대 결과와 실행 사후 조건의 집합이다. 즉 테스트 되어야 하는 특정 조건+입력값+기대 결과를 포함한다.

테스트 케이스?

테스트 프로세스 5단계

- 테스트 프로세스는 테스트와 관련된 다양한 활동이 체계적으로 진행되어 의도된 테스트 목적과 목표를 달성할 수 있도록 테스트의 모든 구성 요소를 엮어주는 역할을 한다.
- 테스트 프로세스는 다음과 같은 단계로 구성되어 있어 있으며, 이에 대하여 파악하고 있어야 한다.
- 테스트 프로세스는 계획/제어, 분석/설계, 구현/실행, 완료/리포팅과 마감의 5단계로 구성.



테스트 케이스?

테스트 프로세스 5단계

1. 계획과 제어

- 테스트 계획 수립은 테스트 목표와 임무를 달성하기 위해 이를 확인하고 필요한 활동을 정의하며, 테스트 제어는 계획 대비 실제 진행 상황을 비교하는 지속적인 활동

2. 분석과 설계

- 테스트 분석과 설계는 일반적이고 추상적인 테스트 목적을 실제적이고 구체적인 테스트 상황과 테스트 케이스로 변환하는 활동.

3. 구현과 실행

- 테스트 구현과 실행은 가장 효율적이고 효과적으로 테스트를 실행하기 위하여 테스트 케이스를 조합하고 테스트 실행에 필요한 다른 정보를 포함하는 테스트 프로시저를 명세화하는 활동

4. 완료조건과 리포팅

- 초기에 정의된 테스트 목표에 비해 어느 정도 실제 테스트가 수행되었는지를 평가하는 활동

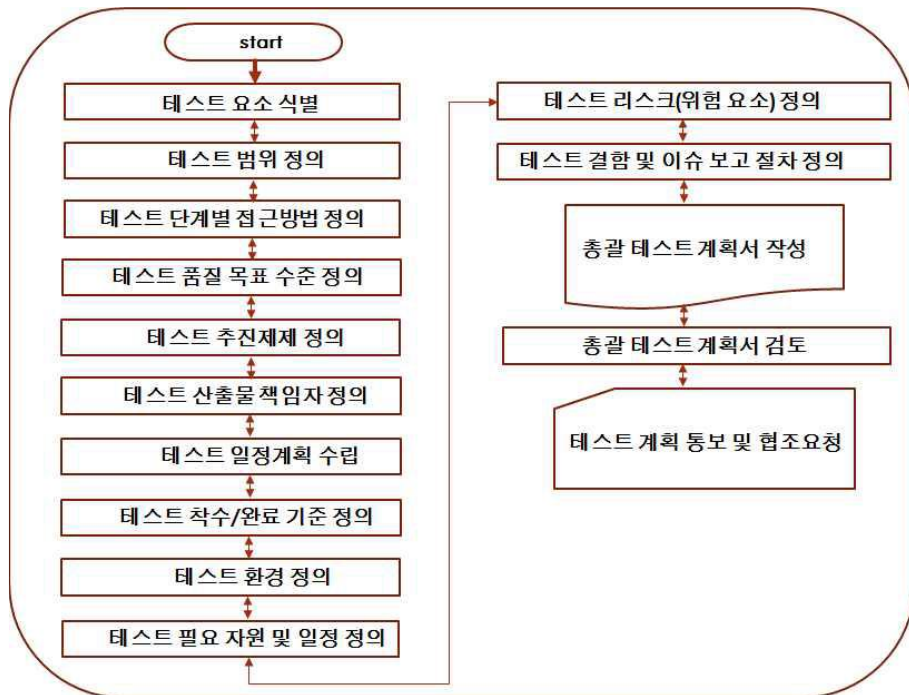
5. 테스트 마감

- 완료된 테스트 활동에서 데이터를 수집하여, 테스트에서 발견된 사실 및 데이터, 경험을 종합하고 축적하는 활동

테스트 케이스?

테스트 범위 적정성 검토와 관리 계획서 작성

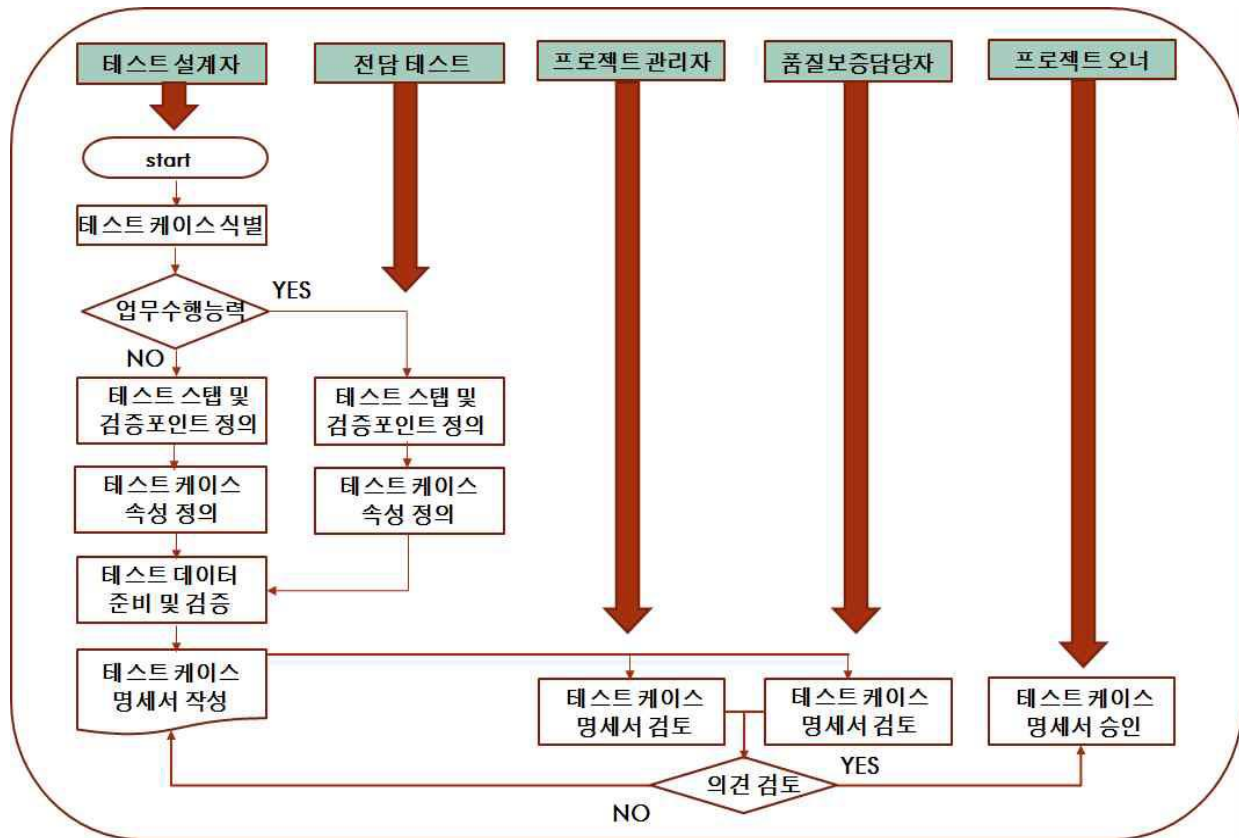
- IT테스트 범위를 정의하는 작업은 전략을 기반으로 한다. 전체적인 테스트를 수행하지 않을경우에 제외해야 할 범위를 산정하고 사유를 기술한다.
- 요구사항 타당성을 검토한다.
- 테스트 전략을 수립한다.
- 테스트 전략 및 계획 활동 작업 흐름도 예시



테스트 케이스?

테스트 범위 적정성 검토와 관리 계획서 작성

- 테스트 케이스 설계 활동
작업 흐름도 예시



테스트 케이스?

테스트 범위 적정성 검토와 관리 계획서 작성

- 테스트 관리 계획서를 작성하기 위해서는 인수 테스트 계획, 시스템 테스트 계획, 통합 테스트 계획, 단위 테스트 계획을 중심으로 관리를 수행하게 되며 이를 중심으로 작성한다.
1. 각 단계별 테스트 계획서를 작성한다.
 - 각 단계별 테스트 계획서를 작성하기 위하여 테스트의 범위, 착수 기준과 완료 기준, 환경, 필요한 자원 및 일정, 테스트 수행 시 위험 요소, 결함이 발생하였을 때 이를 해결하기 위한 절차, 쟁점 발생 시 해결 절차 등을 통하여 각 단계별 테스트 계획서를 작성하고 검토한 후 승인 절차를 거친다.
 2. 인수를 위한 테스트 계획서를 작성한다.
 - 고객의 요구사항이 도출되면 이를 기준으로 착수를 수행하며, 고객의 요구사항 명세서와 프로젝트 개발 계획서를 입력물로 활용하여 인수 테스트 계획서를 작성한다.
 3. 시스템 테스트를 위한 계획서를 작성한다.
 - 업무 설계서가 작성되면 이를 기준으로 착수를 수행하며, 총괄 테스트 계획서, 고객 요구사항 명세서, 설계서 등을 입력물로 활용하여 시스템 테스트 계획서를 작성한다.
 4. 통합 테스트를 위한 계획서를 작성한다.
 - 업무 설계서가 작성되면 이를 기준으로 착수를 수행하며, 총괄 테스트 계획서, 프로젝트 개발 계획서, 고객 요구사항 정의서, 설계서, 프로젝트 표준 등을 입력물로 활용하여 통합테스트 계획서를 작성한다.
 5. 단위 테스트를 위한 계획서를 작성한다.
 - 기술 부분의 설계서가 작성되면 이를 기준으로 착수를 수행하며, 총괄 테스트 계획서, 프로젝트 개발 계획서, 기술 설계서, 프로젝트 표준 등을 입력물로 활용하여 단위 테스트 계획서를 작성한다.

테스트 케이스 작성절차

참조 문서 수집

- 테스트 계획서에 명시된 테스트 케이스 작성지침과 수준을 고려하여 테스트 설계에 필요한 분석 및 설계문서를 수집한다.

테스트 케이스 작성

- 테스트 설계 기법을 이용하여 테스트케이스 작성한다.

내부 검토

- PM, 아키텍처, 디자이너, 기획자, 개발자와 테스트 담당자가 작성된 테스트케이스의 적정성을 검토한다.(상황에 따라 내부 검토자는 변경될 수 있음)

요구사항 대비 커버리지 분석

- 테스트 케이스가 어느 정도 요구사항을 반영하는가에 대한 분석. 기본적으로 테스트 가능한 요구사항은 모두 테스트케이스에 반영이 되었는지 확인한다.

승인

- 작성된 테스트 케이스를 고객(현업), 기획자 및 PM에게 승인을 획득한다.



테스트 케이스 작성절차

테스트 설계

- 개발 산출물을 이용하여 테스트 케이스를 식별하고 정의하는 활동

구분	입력물 작성 항목	입력물
시스템 테스트 설계	✓ 고객 요구 정의서 작성 ✓ 시스템 테스트 계획 수립	- 시스템 테스트 설계 명세 - 테스트 전략 - 고객 요구 정의서 - 업무 설계서 - 시스템 테스트 계획서
통합 테스트 설계	✓ 설계서 작성 ✓ 통합 테스트 계획 수립	- 통합 테스트 설계 명세 - 테스트 전략 - 통합 테스트 계획서 - 테크니컬 설계서 - 설계서 - 프로젝트 표준
단위 테스트 설계	✓ 테크니컬 설계서 ✓ 단위 테스트 계획 수립	- 단위 테스트 설계 명세 - 테스트 전략 - 단위 테스트 계획서 - 테크니컬 설계서 - 프로그램 명세서 - 프로젝트 표준
종료 조건		산출물
테스트 케이스 승인		- 수준별 검증된 테스트 케이스 명세서, 테스트 데이터
작업		
테스트 케이스 식별		
테스트 시나리오 및 검증 포인트 정의		
테스트 케이스 속성 정의		
테스트 데이터 준비 및 검증		
테스트 케이스 명세서 작성		
테스트 케이스 명세서 검토		
테스트 케이스 명세서 승인		

테스트 케이스 작성절차

구성요소

- 테스트 케이스는 테스트 조건을 확인하기 위해 개발한다. 테스트 케이스는 고객의 요구사항과 요구사항과의 일치성을 확인하고, 요구사항과의 관계를 확인하기 위해 연관 관계 정보를 확보하여야 한다.
- 테스트 케이스 구성 요소(IEEE 829)

항목	설명
식별자 (Identifier)	항목 식별자, 일련번호
테스트 항목 (Test Item)	테스트할 모듈 또는 기능
입력 명세 (Input Specification)	입력값 또는 테스트 조건
출력 명세 (Output Specification)	테스트 케이스 실행 시 기대되는 출력값 결과
환경 설정 (Environmental Needs)	테스트 수행 시 필요한 하드웨어나 소프트웨어 환경
특수 절차 요구 (Special Procedure Requirement)	테스트 케이스 수행 시 특별히 요구되는 절차
의존성 기술 (Inter-case Dependencies)	테스트 케이스 간의 의존성

테스트 케이스 예

테스트 케이스			프로젝트명	인사관리 앱 시스템 구축			
			대상 시스템	정산 시스템			
단계	단위 테스트	작성자	홍길동	승인자	박문수	문서 상태	최초 작성
작성일	2020-09-01	버전	v1.0	테스트 범위	공통 모듈	테스트 조직	개발팀
테스트 ID	TC-CMM-001("Common Module" or "Core Mathematical Module") TC-AO-001("Arithmetic Operations")			테스트 일자	2020-09-01		
테스트 목적	사칙연산 정확도 테스트						
테스트 기능	+ 기능 점검						
입력값	1,10 / 0,1/ 0,0/ 0,-1/-1,-10						
테스트단계	테스트 케이스			예상값	중요도	성공 /실패	비고
	1+10			11	중요	성공	
	0+1			1	중요	성공	
	0/0			예외 발생	중요	성공	
	추가된 테스트 사례 1. 예외 처리 확인: • 0 / 0 → ArithmeticException 예외 발생 확인. 다양한 조합:						
테스트환경	자바 JDK 버전: 17 IDE: IntelliJ IDEA 2023.2 운영 체제: 윈도우 10						
전제 조건	프로그램은 오류 없이 성공적으로 컴파일되어야 합니다. 모든 종속성을 올바르게 설치하고 구성해야 합니다.						
성공 /실패 기준	성공: 모든 테스트 사례가 예상한 결과를 생성합니다. 실패: 테스트 케이스가 예상한 결과를 생성하지 않습니다.						
특별하게 필요한 절차	테스트 결과를 기록하고 이에 따라 테스트 사례 문서를 업데이트합니다. 즉각적인 해결을 위해 모든 실패를 개발팀에 전달하세요.						

테스트 케이스 작성절차

테스트 케이스 목록 양식

ID	구분	사전조건	수행절차	기대결과	결과	추적성	중요도	비고
1.1	우편번호 조회	동 이름, 번지 등 5자 이상 입력 후 조회 버튼 클릭	1. 주소 변경 페이지 이동 2. 주소 페이지에서 우편번호 조회 아이콘 클릭	팝업 윈도우 표시 후 조회 데이터 출력 조회된 내용이 없는 경우 우편번호가 없다는 내용의 메시지 제공		요구사항 ID-100	중	우편번호로 주소 조회 기능

테스트 케이스 작성 시 견지해 볼 수 있는 지침

OKR (Objectives and Key Results)

"목표(Objective)는 영감을 주고, 핵심 결과(Key Results)는 실행을 이끈다."

넓게 보는 비전(목표)을 설정하면서도 이를 달성하기 위한 실행 가능한 구체적인 지표(핵심 결과)를 설정하는 방법론.

Google 등에서 성공적으로 활용된 사례.

사례 1: Gmail 출시

Objective: Gmail을 성공적으로 런칭하기.

Key Results:

베타 사용자 100만 명 확보.

평균 이메일 전송 속도를 0.1초 이하로 줄이기.

사용자 피드백 점수 90점 이상 달성.

사례 2: Google Chrome

Objective: 세계에서 가장 빠르고 안정적인 브라우저를 만들기.

Key Results:

브라우저 로딩 속도를 3초 이하로 줄이기.

사용자 만족도 점수 95% 이상 유지.

6개월 내 시장 점유율 10% 도달.

테스트 케이스 작성 시 견지해 볼 수 있는 지침

테스트 목표 설정

1. 비즈니스 모델을 구축한다.
 2. 대상 운영 플랫폼에 대한 품질 차원을 분석하고 테스트 기법을 적용한다.
 3. 품질에 대한 이해를 통하여 목표를 설정한다.
- 테스트 목표 수립에 대한 예

품질목표	평가 항목	목표값	평가 기준
적합성	기능 구현 완전성	100%	전체 기능에 대한 테스트 케이스 시험
정확성	기대 정확도	100%	전체 기능에 대한 테스트 케이스 시험
	정밀도	100%	계산을 수행하는 테스트 케이스 시험
신뢰성	잔존 결함률	낮은 심각도: 20% 이하 높은 심각도: 100%	발견된 결함 중 심각도에 따라 수정된 정도
효율성	응답 시간 적합성	80% 이상	시스템 동시 접속자 1,000명에서 페이지 로딩 속도가 3초 이내
	메모리 오류	0%	메모리 오류가 발생하면 안 됨.

테스트 케이스 작성 시 견지해 볼 수 있는 지침

테스트 목표 설정

- SMART 원칙에 따라 목표를 수립한다.
 - 목표는 다음처럼 구체적, 측정

구분	설명
specific	• 목표는 구체적이어야 한다.
measurable	• 목표는 측정 가능해야 한다.
attainable	• 목표는 달성 가능한 수준이어야 한다.
realistic	• 목표는 현실적이어야 한다.
time based	• 목표는 시간 제한을 두고 설정해야 한다.

테스트 케이스 작성 시 견지해 볼 수 있는 지침

테스트 대상의 품질수준 예시

품질특성	평가항목	목표 값	평가기준(적합기준)	평가결과
기능성	기능구현 완전성	X≈100%	전체기능 TC중 100%성공	최종조치결과 반영
	계산 정확도	X≈95%	계산을 수행하는 TC중 95% 이상이 대략 정확한 값 도출	목표값 만족
	데이터 교환성	X≈95%	다른 소프트웨어나 시스템과 데이터 교환을 시도한 전체 TC 중 95% 이상이 대한 교환 성공	목표값 만족
보안성	접근 제어성	X≈0%	규정된 불법 조작은 성공한 불법조작이나, 규정되지 않은 불법조작이 없음	목표값 만족
	사용자 인증성	X≈100%	전체사용자 중 사전에 허가된 사용자만 통과	목표값 만족
신뢰성	결함밀도	X≤3%	시스템 테스트 시 전체 수행된 TC 중 결함된 결함율이 3% 이하	최종조치결과 반영
사용성	기능이해 가능성	X≈90%	인터뷰스로 가용한 기능 중 모든 기능에 대하여 사용자가 목적을 바르게 기술한 기능이 90% 이상	목표값 만족
효율성	평균 응답시간	X≤1	실제 응답시간의 평균이 규정된 평균응답시간(≤5sec)이하	목표값 만족
	평균처리량	X≥1	실제 작업 처리량의 평균이 규정된 평균 작업 처리량 이상	목표값 만족
유지보수성	결함 분석능력	X≥95%	결함된 전체 TC 중 원인을 파악하지 못한 고장TC가 5% 미만(95%이상의 고장 TC에 대한 원인파악)	목표값 만족
이식성	이식성	X≈100%	상이한 디바이스 운영체제 및 브라우저에서 제대로 동작 가능	목표값 만족
상호운용성	상호운용성	X≈100%	유사App을 같은 기기에 설치하여 충돌이 발생하지 않고 제대로 동작 가능	목표값 만족

테스트 케이스 작성 시 견지해 볼 수 있는 지침

테스트 산출물 관리 방안에 대한 주요 측정 지표

측정 지표	수식
테스트 진척율(%)	$\text{통과된 테스트 케이스 수} \div \text{생성된 테스트 케이스 수}$
단위 결함 수	$\text{발견된 결함} \div \text{수행한 테스트 수}$
결함 조치율(%)	$\text{수정 완료된 결함} \div \text{발견된 결함}$
요구사항 커버리지(%)	$\text{테스트 케이스로 확인된 요구사항 수} \div \text{요구사항 명세에 정의된 요구사항 수}$
프로그램 소스 결함율(%)	$\text{발견된 결함} \div \text{프로그램 소스 라인}$
결함 재발생률(%)	$\text{테스트 실패 결함 수} \div \text{수정된 것으로 보고된 결함 수}$

테스트 케이스 작성 시 견지해 볼 수 있는 지침

테스트 계획서 목차 예시

목차

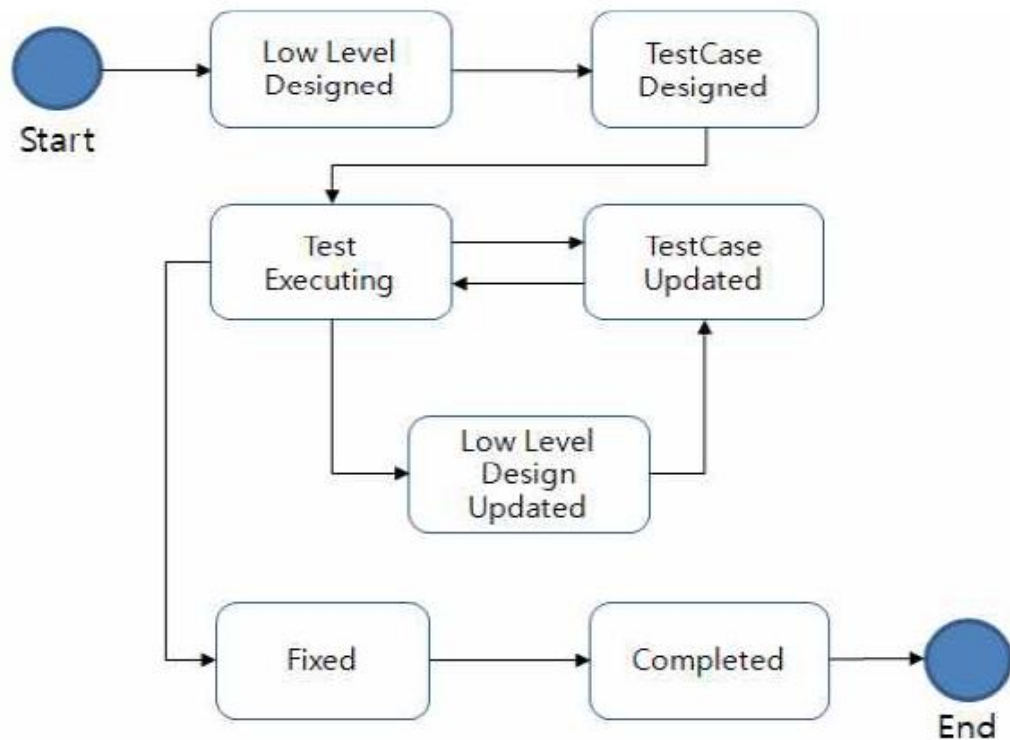
1. 개요
 - 1.1. 목적
 - 1.2. 범위 및 테스트 베이스 도출
2. 테스트 전략
 - 2.1. 기능 테스트 전략
 - 2.2. 비기능 테스트 전략
 - 2.3. 테스트 시작 및 종료 조건 정의
 - 2.4. 테스트 규모 산정
3. 일정 계획
4. 테스트 인프라
 - 4.1. 테스트 환경
 - 4.2. 테스트 도구 및 자동화 접근 방법
5. 테스트 조직
 - 5.1. 테스트 조직
 - 5.2. 테스트 역할
 - 5.3. 교육 훈련 요구사항 및 계획
6. 테스트 산출물
 - 6.1. 테스트 실행 관련 산출물
 - 6.2. 테스트 진행 보고서
 - 6.3. 테스트 프로세스 제어
7. 형상 관리
8. 결함 관리
 - 8.1. 결함 보고서 구성 내용
 - 8.2. 결함 관리 측정 지표

진단 결과 통합 명세화 예시

프로세스 영역	주요 프로세스 목표 및 프랙티스	수행 여부	점수
테스트 계획	SP1.2 제품 리스크 식별	○	FA
	SP3.2 테스트 생명주기 정의	○	
테스트 조직	SP1.1 테스트 조직 정의	○	PA
	SP3.1 테스트 경력 경로 수립	X	
테스트 설계 및 실행	SP1.2 테스트 케이스를 식별하고 우선순위화	○	LA
	SP2.4 테스트 실행 일정 개발	○	
	SP3.1 인테이크 테스트 실행	X	
테스트 생명주기 및 통합	SP1.3 테일러링 기준 및 가이드라인 수립	X	NA
	SP2.1 통합 생명주기 모델 수립	X	

단위 테스트 프로세스 절차 및 활동 예

효과적이고 효율적인 테스트를 수행하기 위해서는 테스트 계획을 수립, 수행, 평가등의 일련의 과정이 체계적으로 정의되어야 한다.

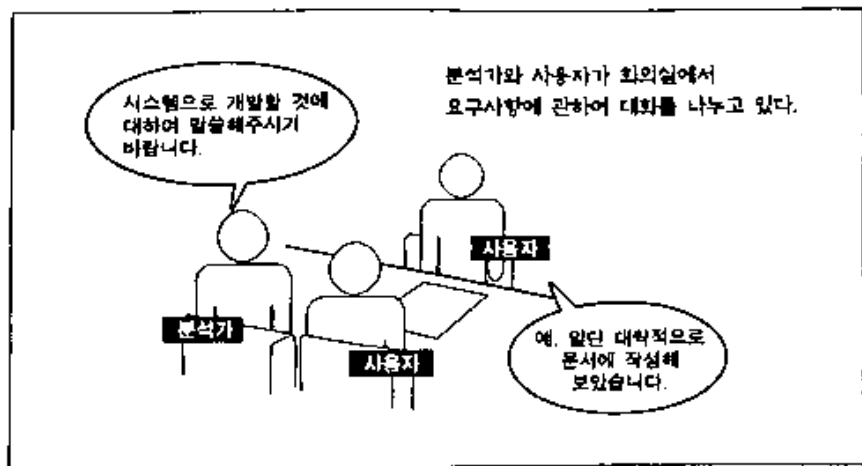


단위 테스트 프로세스 절차 및 활동 예

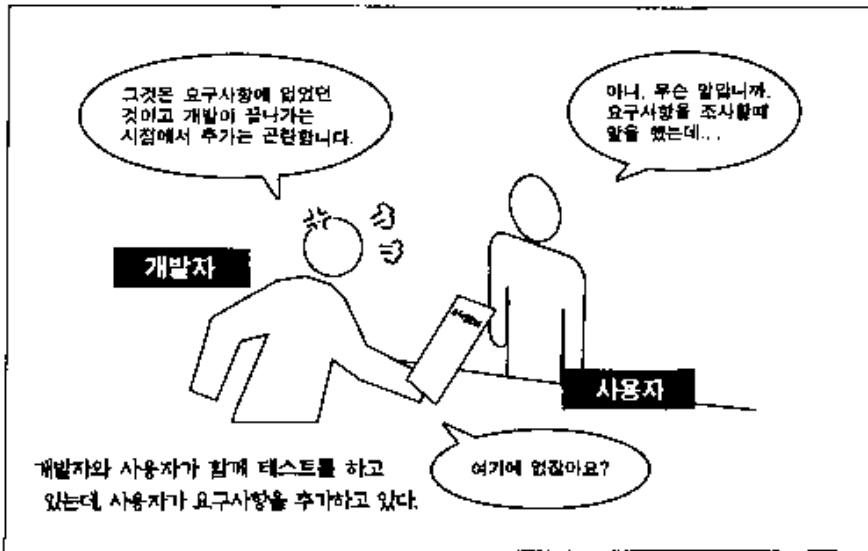
단위 테스트 케이스 작성을 위한 참조 문서를 수집한다.

- 공통 모듈 상세 설계 산출물과 요구 사항 정의서, 요구 사항 명세서 등 분석 및 설계 문서를 수집한다.

【시스템 개발 환경의 모습】

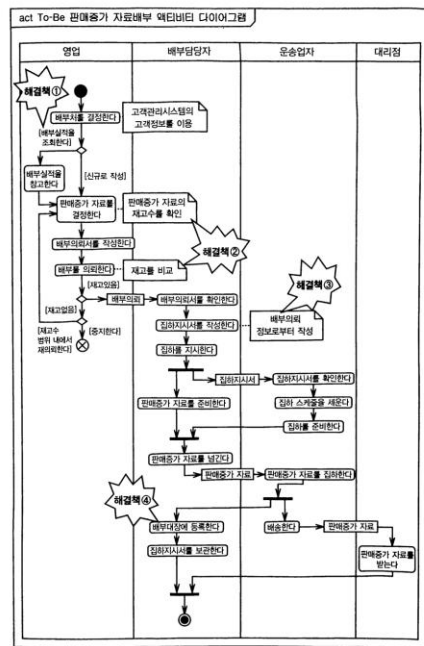
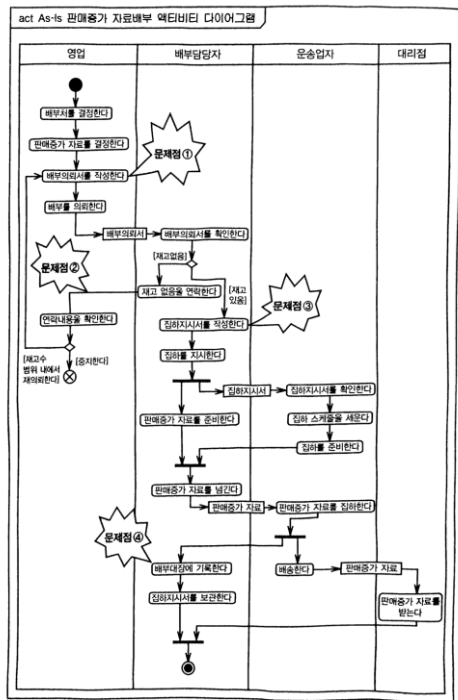


【시스템 개발 환경의 모습】



단위 테스트 케이스 작성을 위한 참조 문서를 수집한다.

- 비즈니스 현황 모델

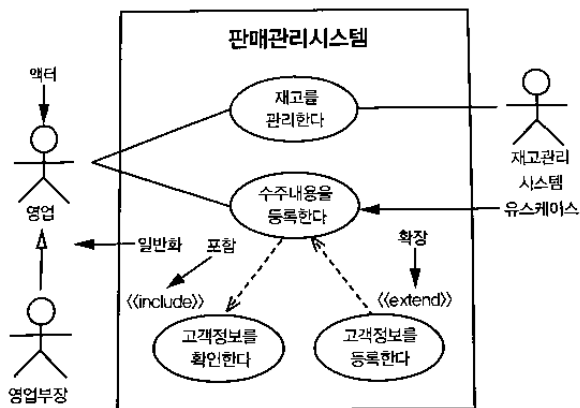


[그림 3-3-2] 해결책을 적용한 액티비티 다이어그램(TO-BE)

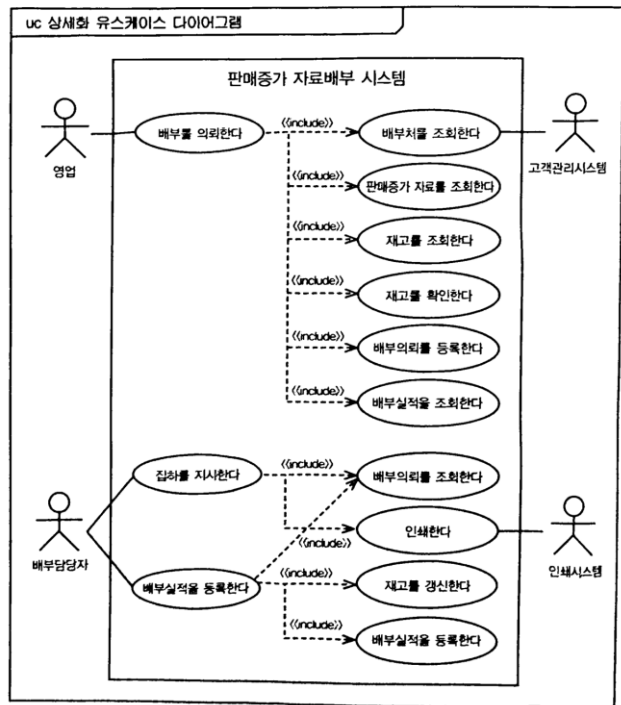
단위 테스트 프로세스 절차 및 활동 예

단위 테스트 케이스 작성을 위한 참조 문서를 수집한다.

- 유스케이스 다이어그램



[그림 1-1-9] 유스케이스 다이어그램의 예



[그림 4-3-4] 유스케이스 다이어그램에 인클루드(include) 유스케이스 반영

단위 테스트 프로세스 절차 및 활동 예

- 유스케이스
- 비즈니스 규칙
 - 부가가치세는 매출액에 10%를 곱하여 산출한다는 것은 잘 알려진 계산법이다. 이와 같이 사전에 정해진 규칙에 의하여 관리되는 것을 총칭하여 비즈니스 규칙이라고 한다.

1. 법령 규칙

- 법령에 관한 부분은 반드시 지켜야 하는 규칙이므로 정확하게 정의해야 한다.
- 부가가치세는 매출금액에 10%를 곱하여 산출하며 원미민은 절삭한다.
- 갑근세는 간이세액 조건표를 기준으로 산출한다.
- 주민세는 갑근세에 10%를 곱하여 10원 단위까지만 산출하며 10원 미만은 절삭한다.

2. 업무처리 규칙

- 업무처리 규칙은 조직에서 업무를 처리함에 있어 준수해야 할 규칙을 포함한다.
- 고객은 가용 포인트 이내에서 사은품을 신청할 수 있다.
- 임직원의 근무시간은 09:00부터 18:00까지이다.
- 출고일은 배송차량이 회사 정문을 통과하는 시점을 기준으로 한다.

3. 계산 방법

- 계산 방법은 계산하는 기준을 설정하는 것이다.
- 판매수량에 표준단가를 곱하여 판매금액을 계산한다. 만약 표준단가가 없으면 $\square$ 판매를 중지하고, 단가 관리자에게 문의한다.
- 포인트를 사용하면 누적 포인트에서 공제한다.
- 포인트는 판매금액의 1%로 계산하며 원단위 이하는 반올림하여 원단위까지 계산한다.

단위 테스트 프로세스 절차 및 활동 예

- 유스케이스
- 비즈니스 규칙 문서 작성 방법

■ 비즈니스 규칙

1. BS0001 : 고객은 가용 포인트 범위에서 사은품을 신청할 수 있으며 가용 포인트는 다음과 같은 방법으로 계산한다.
 - 1.1 사은품 신청 전일까지 발생 포인트 - 사은품 신청 당일까지 사용한 포인트
2. BS0002 : 사은품 신청은 당사 홈페이지에서만 가능하다. 사은품 신청서는 사용하지 않는다.
3. BS0003 : 사은품 신청은 신청마감을 하기 전까지는 언제든지 추가 및 변경이 가능하다.

■ 참고자료

자료번호	명 칭
101	DM 양식
102	사은품 신청서
103	사은품 신청 현황

단위 테스트 프로세스 절차 및 활동 예

요구 사항 추적 성 매트릭스를 만드는 방법

- 테스터는 BRD (Business Requirement Document) 및 TRD (Technical Requirement Document)를 기반으로 테스트 사례를 작성하기 시작합니다.

1단계 : 테스트 케이스

- "로그인 확인, 올바른 아이디와 비밀번호를 입력하면 성공적으로 로그인해야합니다"

2 단계 :이 테스트 사례가 확인하는 기술 요구 사항을 확인.

3 단계 : 테스트 케이스에서이 기술 요구 사항을 기록.

4 단계 : 이 TR (기술 요구 사항)이 정의 된 비즈니스 요구 사항 식별

5 단계 : 테스트 사례에서 BR (비즈니스 요구 사항)

6 단계 : 모든 테스트 케이스에 대해 위의 작업을 수행합니다.

단위 테스트 프로세스 절차 및 활동 예

요구사항 분석표

• 기능 요구사항 분석표

요구사항ID	요구사항 명	요구사항 내용	구현 방안	검증 방안	테스트ID
FR-007	문화xxDB관리 시스템 구축	문화xxDB 서비스 및 콘텐츠 관리를 위한 웹 기반 관리페이지 구축	- 웹 기반 관리 페이지 구축 - 모든 내용을 한눈에 볼 수 있도록 제작	- 웹 기반 관리 페이지가 정상적으로 구축됨을 구현된 기능에 대해 단위 테스트와 통합 테스트를 수행하여 검증 - 콘텐츠 관리 기능 구현 여부 확인	UT-DM-001, TC-DM-001
FR-008	문화데이터 콘텐츠 관리	문화데이터 콘텐츠의 등록/수정/삭제/조회 기능 구현	문화xx시스템에서 유형별로 정리된 DB를 토대로 메뉴 구성	- 문화xx시스템 DB와 구현된 메뉴의 일치성 여부 확인	UT-DM-002, TC-DM-002
FR-011	기관연계 관리	xx시스템에서 정보 변경 시 oo시스템에 변경 내역을 푸시(push)하는 라이브러리 제공	REST 기반으로 변경 내역을 푸시(push)하는 기능을 수행하는 라이브러리 개발	- xx기관의 oo시스템으로 구축 예정	UT-DM-002, TC-DM-002

단위 테스트 프로세스 절차 및 활동 예

요구사항 분석표

• 비기능 요구사항 분석표

요구사항 ID	요구사항 명	요구사항 내용	구현 방안	검증 방안	산출물
xxPER-001	시스템 성능	관련 서버 및 SW의 성능을 테스트하고 결과 보고, CPU사용률은 70% 이하		구축된 시스템의 성능 테스트 결과 제시	시스템 테스트 결과서
xxPER-002	평균응답 시간	정상 상태에서 사용자의 질의요청에 결과 페이지를 4초 이내에 출력		구축된 시스템의 성능 테스트 결과 제시	시스템 테스트 결과서
xxPER-005	시스템 처리량	초당 최소 100건의 사용자 질의 처리, 최대 부하 상태에서 초당 50건의 질의 처리		구축된 시스템의 성능 테스트 결과 제시	시스템 테스트 결과서

단위 테스트 프로세스 절차 및 활동 예

요구사항 명세서

- 요구사항 명세서에 반영된 사용자 요구사항, 시스템 요구사항, 애플리케이션의 기능 요구 사항 및 비기능 요구사항에 대해 분석한다.

요구사항ID	요구사항 명	요구사항 내용	유형	중요도	난이도	제약사항	출처
Ra-xxx-012	관리자	통합게시판 관리 기능 구현	기능	상	5		SFR-001
Ra-xxx-013	쪽지 송수신 기능구현	웹UI를 통한 쪽지... 수신자별, 그룹별, 개인별 쪽지 발송...	기능	상	5	파일첨부 기능 삭제	SFR-001
Ra-xxx-031	평균응답 시간	사용자의 질의요청에 대한 결과 페이지를 4초 이내 출력	비기능(성능)	상	4		PER-002

단위 테스트 프로세스 절차 및 활동 예

단위 테스트 케이스를 작성한다.

- 목록 형식 테스트 케이스의 예

테스트 케이스 ID	테스트 시나리오	테스트 단계	테스트 데이터	예상 결과	실제 결과	통과 실패
TU01	유효한 데이터로 고객 로그인 확인	사이트 http://#으로 이동 하십시오 .사용자 입력암호를 입력제출을 클릭하십시오 .	사용자 ID = id99 암 호 = pass99	사용자가 애플리케이션에 로그인 해야합니다 .	예상대로	패스
TU02	잘못된 데이터로 고객 로그인 확인	사이트 http://#으로 이동 하십시오 .사용자 입력암호를 입력제출을 클릭하십시오 .	사용자 ID = id99 암 호 = pass99	사용자는 애플리케이션에 로그인 해서는 안됩니다 .	예상대로	패스

단위 테스트 프로세스 절차 및 활동 예

단위 테스트 케이스를 작성한다.

- 테이블 형식 테스트 케이스 예

테스트 케이스 No. 001					
목적:화면 시험					
특수 요구사항:유저 아이디와 패스워드 검증					
단계	입력	예상 출력	실행 결과	TR#	비고
1. 화면 로그인 및 팝업(유저 아이디와 패스워드 없이 로그인 시도)					
1-1	파워 온	카피라이트 화면 디스플레이			
1-2	'beep' 버튼 클릭	소리 및 화면 반전			
1-3	'continue' 버튼 클릭	버로그인 화면 보임			
1-4	'로그인' 버튼 클릭	로그인 실패 창 나타남			실패 메시지 확인 버튼 보임
1-5	'확인' 버튼 클릭	로그인 화면으로 리턴			
2. 정상 유저 아이디와 패스워드 없이 로그인 시도					
2-1	유저 아이디 필드 클릭	유저 아이디 필드			

단위 테스트 프로세스 절차 및 활동 예

테스트 케이스를 작성

- 일반 테스트 시나리오를 기술한 테스트 케이스 형태 문서화 예시

테스트 케이스 No. 001		
목적: 화면 시험		
특수 요구사항: 유저 아이디와 패스워드 검증		
단계	설명	예상출력
1	화면 로그인 및 팝업(유저 아이디와 패스워드 없이 로그인 시도)	
2	정상 유저 아이디와 패스워드 없이 로그인 시도	
3	유저 아이디 입력 없이 정상적인 패스워드로만 로그인 시도(로그인 화면에서 나옴)	
4	비정상적인 유저 아이디와 패스워드를 이용하여 다양한 방법으로 로그인을 시도한다.	
5	정상적인 유저 아이디와 패스워드를 이용하여 로그인을 시도한다.	

단위 테스트 프로세스 절차 및 활동 예

단위 테스트 케이스 결과서를 작성한다.

단위 테스트 결과서				
시스템/서브시스템		시험 담당자		시험일자
관련 유스케이스 ID		시험 확인자		
테스트 케이스 ID		프로그램 담당자		완료일자
설명	유스케이스 명세에 정의된 각 유스 케이스의 기능을 참고하여 현 유스 케이스에서 테스트하고자 하는 기능에 대해 간략하게 기술한다.			
기능 유형	예) 조회, 추가, 수정, 삭제			
처리 유형	예) 단일 데이터 처리, 복수 데이터 처리			
테스트 항목 및 항목별 테스트 결과				
기본 기능 수행	유스케이스 명세에 나타난 기능들이 제대로 수행되는지를 확인한다. 여러 건의 데이터의 경우 연속적인 데이터 입력과 여러건의 데이터의 처리가 동시에 일어나는지를 나타낸다.			
수행 속도	조회건수	단일 페이지인 경우, 다음 버튼인 경우에만 사용한다.	수행 속도	작업을 요청한 결과가 나타나기 까지 걸리는 수행 시간을 초단위로 기술한다.
	처리건수	여러건의 데이터를 한번에 처리하는 경우 한번에 처리되는 데이터 건수를 기술한다.	수행 속도	초
화면 일반				
화면 네비게이션				
데이터 처리				
권한 처리				
메시지 처리				
표준 준수				
비고				
최종 평가	합격(), 불합격(), 사유()			

단위 테스트 프로세스 절차 및 활동 예

단위 테스트 결과 등록

스텝 #	활동	입력 데이터	테스트 예상 결과 값	테스트 수행일	테스트 결과	검토자
10	글로벌 로밍 메뉴 선택	글로벌 로밍 메뉴 선택	1."선택된 서비스가 없습니다. 서비스를 선택해 주십시오" 메시지 팝업	2020.09.01	pass	홍길동
20	통합 UI(360) 화면에서 고객정보 조회	전화 번호: 010-xxxx-xxxx	1. 고객 정보 정상 조회	2020.09.01	fail	이순신
30	글로벌 로밍 메뉴 선택	글로벌 로밍 메뉴 선택	1. 글로벌 로밍 화면 정상 로딩 2. 고객 단말기 탭에 스텝 20에서 선택한 상품에 해당하는 단말기 정보 조회 3. 로밍 서비스 누적 이용 이력 영역 정보 조회 4. 로밍 사용 이력 정보 조회	2020.09.01	blocked	황진희

단위 테스트 프로세스 절차 및 활동 예

작성된 단위 테스트 케이스를 내부 검토한다

1. 단위 테스트 케이스의 적정성을 검토한다.

- PM(Project Manager), 아키텍처, 기획자, 개발자, 테스트 담당자 등 내부 이해관계자들이 작성된 테스트 케이스의 적정성을 검토한다.

2. 요구 사항에 충족할 수 있는 테스트 범위인지 검토한다.

- 단위 테스트 케이스가 요구 사항에 충족할 수 있도록 작성되었는가에 대한 테스트 범위를 검토한다.

3. 공통 모듈은 다른 모듈에서 활용되므로 활용성이나 영향도 등을 고려하여 검토한다.

요구사항 대비 커버리지 분석

테스트 커버리지는 테스트 케이스 또는 테스트 스위트(suite)를 통해 테스트된 범위를 의미한다. 일반적으로 퍼센트(%)로 표시되며, 100%는 전체 범위를 테스트했다는 의미로, 값이클수록 테스트를 충분히 수행했음을 나타낸다. 테스트 케이스의 수와 테스트 커버리지를 비교하여 테스트의 효율성을 측정한다.

- 요구사항 커버리지
 - 이해관계자의 요구사항 중에서 테스트된 요구사항의 비율
- 설계 커버리지
 - 클래스, 컴포넌트, 테이블 등과 같이 설계된 개체 중에서 테스트를 수행한 개체의 비율
- 코드 커버리지
 - 프로그램 소스 코드의 라인별 명령어, 조건문, 분기 등의 구문을 통틀어 전체 코드에서 테스트된 코드의 비율
- 위험 커버리지
 - 계획 단계에서 예상된 위험 중에서 테스트된 위험의 비율
- 구문 커버리지(statement coverage)
 - 전체 코드에서 한 번 이상 테스트된 구문의 비율
- 조건 커버리지(condition coverage)
 - 조건문의 내부 조건에 따른 전체 처리 경로 중에서 테스트를 수행한 경로의 비율
- 결정 커버리지(decision coverage)
 - 분기의 내부 조건이 아닌 내부 조건 결과로 인한 전체 결과에 따른 코드 진행 경로 중에서 테스트를 수행한 경로의 비율

요구사항 대비 커버리지 분석

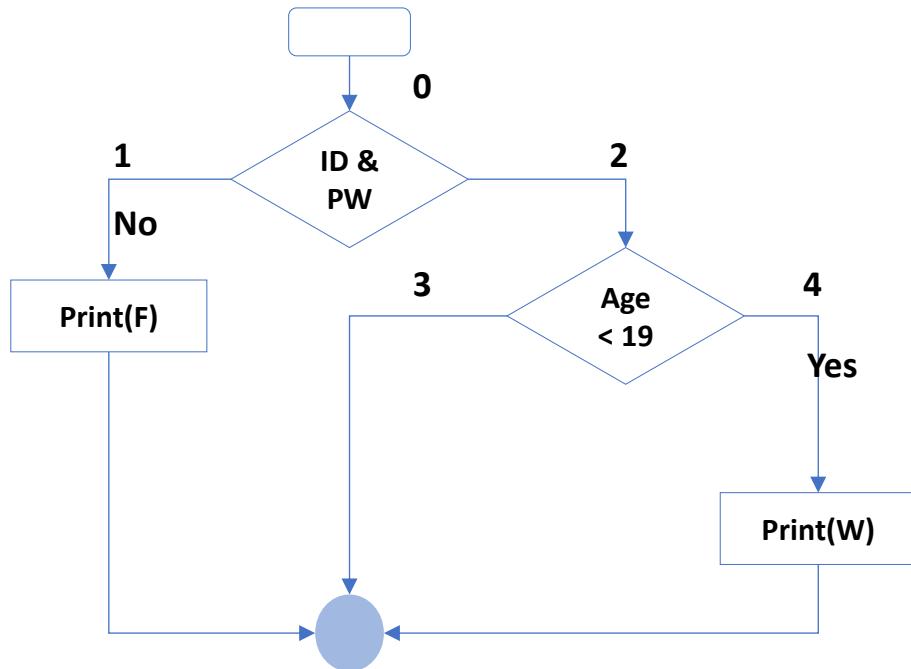
기능에 대한 테스트 커버리지

- 위험 테스트 커버리지 예시

위험 아이템	우선순위	장애 가능성	예상 손실	테스트 케이스	실행 여부
위험 아이템 1	1	9	데이터 손실	TC_PER_01 TC_CM_01 TC_TB_02	O O O
위험 아이템 1	1	9	손해 배상		X
위험 아이템 1	2	7	업무 중단	TC_UI_04 TC_TB_04	O X
...					
위험 아이템 n	5	1	사용자 불편	TC_UI_01 TC_TB_01	O

- 제어 흐름 테스트는 구문 커버리지 또는 결정 커버리지를 달성하기에 유용하다.
- 프로그램 코드 예 제어 흐름도 표기

```
print("Fail")
```



요구사항 대비 커버리지 분석

테스트 케이스 설계

- 구문 커버리지 100% 달성을 위한 테스트 설계
- 다음 테스트 커버리지는 구문을 커버하는 최소의 테스트 케이스이다. 하지만, 모든 결정을 커버하는 테스트는 아니다. 3No 방향 표기에 대한 테스트 케이스가 없으므로 결정문 커버리지는 총 4의 결정 중 3개의 결정만 테스트만 수행하였으므로 75% 결정문 테스트 커버리지가 되는 것이다.

번호	테스트 케이스
TC1(0,1)	0 -> 1
TC2(0,2,4)	0 -> 2 -> 4

- 따라서 추가적인 테스트 케이스 TC3(0,2,3)를 수행하게 되면 결정문 커버리지 100%를 달성하게 된다.

번호	테스트 케이스
TC1(0,1)	0 -> 1
TC2(0,2,4)	0 -> 2 -> 4
TC3(0,2,3)	0 -> 2 -> 3

요구사항 대비 커버리지 분석

테스트 후 활동-지속적 통합

- 기능에 대한 테스트 커버리지
 - 전체 비즈니스 프로세스와 시스템 기능에 대해 수행된 테스트 케이스 수를 기준으로 기능에 대한 테스트 커버리지를 측정한다.
 - 기능에 대한 테스트 커버리지 예시

시스템 기능 테스트 커버리지=(테스트 케이스가 있는 시스템 기능/ 전체 시스템 기능) = $(26/32) \times 100 = 81.25\%$

비즈니스 프로세스			시스템 기능	중요도	테스트케이스 수
L1	L2	L3			
IT테스트	테스트 기획	사업 환경 분석	테스트 기획 관리	1	5
		사업 범위 파악			
		사업 목표 설정			
		사업 정책 수립			
	테스트 분석				21
	...				
	테스트 완료			5	0
...					
N					
총합			32	100	26

- 결함 보고서

[illegible]

1일차 강의 자료 · 8시간

워크 북 실습

Arithmetic, PrimeChecker



TDD with C++ & Java

VSCode 개발자를 위한 실전 테스트 주도 개발

Google Test · Google Mock · CMake | JUnit 5 · Mockito (참고)

C++17 / C++20

Google Test

Google Mock

CMake 3.14+

Java 참고: Java 17+

Java 참고: JUnit 5

Java 참고: Mockito 5

3

강의 목차 — 4시간 커리큘럼 (C++ 주, Java 참고)

00:00–01:00

★ C++ 주

Ch.1 Google Test 기초 & 테스트 설계 원칙

C++ | CMake 설정 · TEST/TEST_F · 테스트 설계(AAA/FIRST/Right-BICEP/CORRECT)

Java 참고 | JUnit 5 아키텍처, @Test, Lifecycle 어노테이션

01:00–02:00

★ C++ 주

Ch.2 Google Test 고급 — 파라미터·Matcher·Approval

C++ | TYPED_TEST · TestWithParam · EXPECT_THAT · Matcher 완전정복 · ApprovalTests C++

Java 참고 | @ParameterizedTest, Hamcrest, ApprovalTests Java

02:00–03:00

★ C++ 주

Ch.3 Google Mock — Test Double 완전정복

C++ | MOCK_METHOD · ON_CALL · EXPECT_CALL · ArgumentCaptor · InSequence

Java 참고 | Mockito mock/stub/spy/verify, ArgumentCaptor, InOrder

03:00–04:00

★ C++ 주

Ch.4 Write Better Tests — TDD 실전

C++ | Red-Green-Refactor · friend 테스트 · 컬렉션 · operator== · Test Smell

Java 참고 | TDD 체크리스트, 닭과달걀, 동치성, 리플렉션

CHAPTER 1

Google Test 기초 & 테스트 설계 원칙

CMake 프로젝트 설정 · TEST/TEST_F · AAA · FIRST · Right-BICEP · CORRECT

Google Test 아키텍처 & 프레임워크 비교

★ Google Test (C++) — 메인

gtest_main / gmock_main

main() 자동 생성, 테스트 실행기

GoogleTest (gtest)

단언(EXPECT_*/ASSERT_*), 테스트 정의

GoogleMock (gmock)

Mock 객체, MOCK_METHOD, EXPECT_CALL

CMake / CTest / CDash

빌드·실행·리포팅 통합

Java 대응 — JUnit 5

JUnit Platform

IDE/빌드도구 런처

JUnit Jupiter

@Test, @ExtendWith

Mockito

mock(), verify()

Maven / Gradle

빌드, 의존성 관리

C++ ASSERT_* = 즉시 중단 (return) | EXPECT_* = 계속 진행 (후속 검사 가능) ↔ Java: assertThrows 즉시 중단, assertAll 전체 수행

VSCode C++ 테스트 환경 구성 — CMake + Google Test

C++ (Google Test / GMock)

```
# CMakeLists.txt (루트)
cmake_minimum_required(VERSION 3.14)
project(MyApp CXX)
set(CMAKE_CXX_STANDARD 17)

include(FetchContent)
FetchContent_Declare(
  googletest
  URL https://github.com/google/googletest/
    archive/refs/tags/v1.14.0.zip
)
set(gtest_force_shared_crt ON CACHE BOOL "" FORCE)
FetchContent_MakeAvailable(googletest)

# 프로덕션 코드
add_library(mylib
  src/score_collection.cpp
  src/calculator.cpp)

# 테스트 바이너리
add_executable(all_tests
  tests/score_collection_test.cpp
  tests/calculator_test.cpp)

target_include_directories(all_tests PRIVATE include)
target_link_libraries(all_tests
  mylib
  GTest::gtest_main
  GTest::gmock_main)
```

VSCode 필수 확장

- ms-vscode.cpptools
- ms-vscode.cmake-tools
- matepek.vscode-catch2-test-adapter
- ms-vscode.cpptools-extension-pack

빌드 & 실행 명령

```
cmake -B build -DCMAKE_BUILD_TYPE=Debug
```

```
cmake --build build -j4
```

```
ctest --test-dir build -V
```

VSCode: Ctrl+Shift+P → CMake: Run Tests

또는 테스트 패널 왼쪽 사이드바 클릭

Java: Extension Pack for Java + pom.xml에 junit-jupiter + mockito-core 의존성 추가

TEST · TEST_F · TEST_P — 테스트 정의 3가지 방법

C++ (Google Test / GMock)

```
// ① TEST — 독립 테스트 (픽스처 없음)
TEST(CalculatorTest, AddsTwoNumbers) {
    Calculator calc;
    EXPECT_EQ(10, calc.add(5, 5));
}

TEST(CalculatorTest, DivideByZeroThrows) {
    Calculator calc;
    EXPECT_THROW(calc.divide(10, 0),
                 std::invalid_argument);
}

// ② TEST_F — 픽스처 공유 (SetUp/TearDown)
class CalculatorTest : public ::testing::Test {
protected:
    Calculator calc;           // 공유 객체
    void SetUp() override {
        calc.reset();          // 각 테스트 전 초기화
    }
    void TearDown() override { /* 정리 */ }
};

TEST_F(CalculatorTest, AddAfterReset) {
    EXPECT_EQ(0, calc.result()); // SetUp 보장
    EXPECT_EQ(7, calc.add(3, 4));
}

TEST_F(CalculatorTest, SubtractAfterReset) {
    EXPECT_EQ(-1, calc.subtract(3, 4));
}
```

Java 대응 (JUnit 5 / Mockito)

```
// Java ① — @Test (독립)
@Test
void addsTwoNumbers() {
    Calculator calc = new Calculator();
    assertEquals(10, calc.add(5, 5));
}

@Test
void divideByZeroThrows() {
    assertThrows(IllegalArgumentException.class,
                 () -> calc.divide(10, 0));
}

// Java ② — @BeforeEach (픽스처)
class CalculatorTest {
    Calculator calc;

    @BeforeEach
    void setUp() {
        calc = new Calculator();
        calc.reset();
    }

    @AfterEach
    void tearDown() { /* 정리 */ }

    @Test
    void addAfterReset() {
        assertEquals(0, calc.result());
        assertEquals(7, calc.add(3, 4));
    }
}
```

TEST = 함수형 / TEST_F = 클래스 픽스처 공유 / TEST_P = 파라미터화 | Java 대응: @Test = TEST, @BeforeEach = SetUp(), @AfterEach = TearDown()

AAA 패턴 (Arrange-Act-Assert) — 테스트의 기본 뼈대

Arrange

객체 생성 · 초기값 설정 · Mock 준비

Act

테스트 대상 메서드 실행

Assert

결과 검증 (EXPECT_*/ASSERT_*)

C++ (Google Test / GMock)

```
// score_collection.h
class ScoreCollection {
    std::vector<std::function<int()>> scores;
public:
    void add(std::function<int()> scoreable) {
        scores.push_back(scoreable);
    }
    int arithmeticMean() {
        if (scores.empty())
            throw std::runtime_error("empty");
        int total = 0;
        for (auto& s : scores) total += s();
        return total / static_cast<int>(scores.size());
    }
};

// score_collection_test.cpp
TEST(ScoreCollectionTest, ArithmeticMeanOfTwo) {
    // Arrange
    ScoreCollection collection;
    collection.add([] { return 5; });
    collection.add([] { return 7; });

    // Act
    int result = collection.arithmeticMean();
    // Assert
    EXPECT_EQ(6, result);
}
```

Java 대응 (JUnit 5 / Mockito)

```
// ScoreCollection.java
public class ScoreCollection {
    private List<Scoreable> scores =
        new ArrayList<>();

    public void add(Scoreable s) {
        scores.add(s);
    }
    public int arithmeticMean() {
        int total = scores.stream()
            .mapToInt(Scoreable::getScore)
            .sum();
        return total / scores.size();
    }
}

// ScoreCollectionTest.java
@Test
void arithmeticMeanOfTwo() {
    // Arrange
    ScoreCollection collection =
        new ScoreCollection();
    collection.add(() -> 5);
    collection.add(() -> 7);
    // Act
    int result = collection.arithmeticMean();
    // Assert
    assertEquals(6, result);
}
```

C++ lambda [] { return N; } ↔ Java lambda () -> N | EXPECT EQ(expected, actual) ↔ assertEquals(expected, actual) — 인수 순서 동일!

FIRST 원칙 — 좋은 단위 테스트의 5가지 조건

F

Fast

빠르게 — DB·네트워크 호출 최소화

I

Isolated

독립적 — 전역 상태·공유 변수 금지

R

Repeatable

반복 가능 — 시간·랜덤 등 환경 무관

S

Self-validating

자동 검증 — PASS/FAIL 자동 판단

T

Timely

적시 작성 — 코드와 동시 또는 이전

C++ (Google Test / GMock)

```
// ✅ Fast — 단순 연산
TEST(CalcTest, FastAdd) {
    Calculator c;
    EXPECT_EQ(10, c.add(5, 5)); // 순수 연산, 즉시
}

// ❌ Slow — 파일/DB 직접 접근
TEST(CalcTest, SlowFileRead) {
    std::ifstream f("/huge/data.csv"); // 느림!
    EXPECT_TRUE(f.is_open());
}

// ✅ Isolated — 전역 상태 없음
TEST(CounterTest, Increment) {
    Counter c; // 로컬 생성, 공유 없음
    c.increment();
    EXPECT_EQ(1, c.value());
}

// ❌ Not Isolated — 전역 공유
int g_counter = 0; // 위험! 순서 의존
```

Java 대응 (JUnit 5 / Mockito)

```
// ✅ Fast
@Test void fastAdd() {
    assertEquals(10, calc.add(5,5));
}

// ❌ Slow — DB 연결
@Test void slowDB() {
    db.connect(); // 느림!
    assertEquals("Hi", db.getData());
}

// ✅ Isolated
@Test void increment() {
    Counter c = new Counter(); // 로컬
    c.increment();
    assertEquals(1, c.value());
}

// ❌ Not Isolated
static int counter = 0; // 공유!
@Test void t1() {
```

Right-BICEP & CORRECT — 무엇을, 어떤 경계로 테스트할까?

R

Right

B

Boundary

I

Inverse

C

Cross

E

Error

P

Perf

C++ (Google Test / GMock)

```
// C++ - Right-BICEP + CORRECT 실전
// R: 정확한 결과
EXPECT_EQ(10, calc.add(5, 5));

// B: 경계 (Boundary) - INT 오버플로우
EXPECT_NO_THROW(calc.add(INT_MAX, 1));

// I: 역관계 - add 후 subtract
int r = calc.add(5, 5);
EXPECT_EQ(5, calc.subtract(r, 5));

// C: 교차 - 교환법칙
EXPECT_EQ(calc.add(2,3), calc.add(3,2));

// E: 에러 - 0 나누기
EXPECT_THROW(calc.divide(10, 0),
              std::invalid_argument);

// P: 성능
auto t0 = std::chrono::steady_clock::now();
calc.factorial(10000);
auto dur = std::chrono::steady_clock::now()-t0;
EXPECT_LT(std::chrono::duration_cast<
           std::chrono::microseconds>(dur).count(),
           1000); // 1ms 이내
```

// CORRECT - Range (범위)

Java 대응 (JUnit 5 / Mockito)

```
// Java 대응
// R
assertEquals(10, calc.add(5,5));

// B - overflow
assertDoesNotThrow(
    ()->calc.add(MAX_VALUE, 1));

// I - 역관계
assertEquals(5,
    calc.subtract(calc.add(5,5), 5));

// C - 교환법칙
assertEquals(calc.add(2,3),
    calc.add(3,2));

// E - 예외
assertThrows(ArithmeticException.class,
    () -> calc.divide(10, 0));

// P - 성능
long t0 = System.nanoTime();
calc.factorial(10000);
assertTrue(System.nanoTime()-t0
    < 1_000_000);
```

// CORRECT - Range

실습 — Largest 배열: 테스트로 버그 3개 잡기

C++ (Google Test / GMock)

```
// largest.h - 최초 버그 버전
int largest(const std::vector<int>& list) {
    int max = INT_MAX;           // 버그①: 항상 큰 값
    for (int i = 0; i < (int)list.size()-1; i++) // 버그②: 마지막 원소 누락
        if (list[i] > max) max = list[i];
    return max;
}

// — 버그 발견 테스트 작성 —————
TEST(LargestTest, Simple) {
    EXPECT_EQ(9, largest({7, 8, 9})); // FAIL ①
}
// 수정: max = INT_MIN (또는 list[0])

TEST(LargestTest, LastElement) {
    EXPECT_EQ(9, largest({8, 7, 9})); // FAIL ②
}
// 수정: i < list.size()

TEST(LargestTest, AllNegative) {
    EXPECT_EQ(-7, largest({-9, -8, -7})); // FAIL ③
}
// max=INT_MAX 수정으로 해결 (INT_MIN 사용)

// — 최종 수정 버전 —————
int largest(const std::vector<int>& list) {
    if (list.empty())
        throw std::runtime_error("EmptyList");
    int max = std::numeric_limits<int>::min();
    for (int v : list)
```

Java 대응 (JUnit 5 / Mockito)

```
// Java 대응 - 동일 버그 시나리오
// 버그① max=Integer.MAX_VALUE
// 버그② index < list.length-1
// 버그③ max=0 (음수 처리 실패)

// 최종 수정 버전
public class Largest {
    public static int largest(int[] list) {
        if (list.length == 0)
            throw new RuntimeException(
                "EmptyList");
        int max = Integer.MIN_VALUE;
        for (int v : list)
            if (v > max) max = v;
        return max;
    }
}

// 테스트
@Test void order() {
    assertEquals(9, Largest.largest(
        new int[]{7,8,9}));
    assertEquals(9, Largest.largest(
        new int[]{9,8,7}));
}
@Test void allNegative() {
    assertEquals(-7, Largest.largest(
        new int[]{-9,-8,-7}));
}
@Test void empty() {
```

🐉 버그①: INT_MAX → INT_MIN 버그②: size()-1 → size() 버그③: max=0 → min() — 테스트가 없었다면 전부 숨어있었을 버그!

TDD (TEST DRIVEN DEVELOPMENT)

1

Test the program before you write it.

-TDD by Example, Kent Beck

2

TDD의 기원

Agile 개발 방법론의 하나인 eXtreme Programming의 실천 방식 중 하나.

3

TDD의 정의

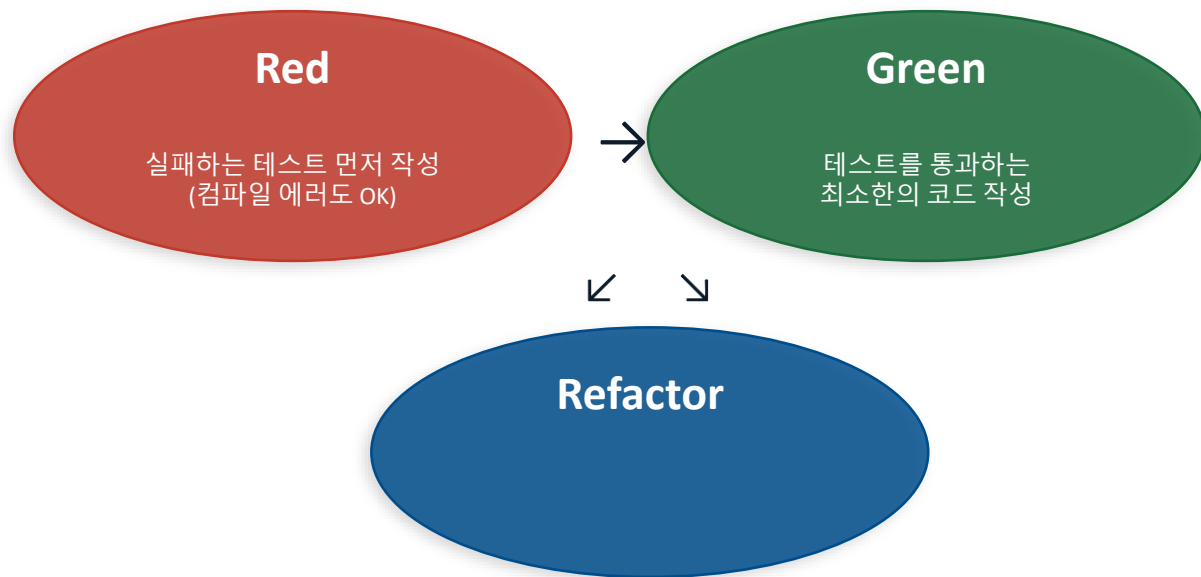
코드를 작성하기 전에 테스트 코드를 먼저 만드는 것
설계 문서를 만들어 생각하고 코딩하여 눈으로 확인하던 개발 방식
=> 예상 결과를 코드로 표현해 놓고 해당 코드가 자동으로 판단하게 하는 개발 방식

4

TDD의 목표

Clean code that works. – Ron Jeffries
소프트웨어의 품질 : 유지보수의 편의성, 가독성, 비용, 안정성 고려

TDD 사이클 — Red → Green → Refactor



Java 대응 (JUnit 5 / Mockito)

// Java TDD 대응 - 동일 사이클

// ① Red

```
@Test void isEmptyOnConstruct() {  
    MyStack<Integer> s =  
        new MyStack<>(); // 없으면 컴파일 에러  
    assertTrue(s.isEmpty());  
}
```

// ② Green (하드코딩)

```
class MyStack<T> {
```

C++ (Google Test / GMock)

// C++ TDD 예시 - Stack 구현 과정

// ① Red: 실패하는 테스트 먼저

```
TEST(StackTest, IsEmptyOnConstruct) {  
    MyStack<int> s; // 컴파일 에러!  
    EXPECT_TRUE(s.empty()); // 목표 명확  
}
```

// ② Green: 최소 구현으로 통과

```
template<typename T>  
class MyStack {  
public:  
    bool empty() const { return true;  
} // 하드코딩  
};
```

// ③ Red: 다음 테스트 추가

```
TEST(StackTest, SizeAfterPush) {  
    MyStack<int> s;  
    s.push(42); // 컴파일 에러!  
    EXPECT_EQ(1u, s.size());  
}
```

// ④ Green: push/size 구현

```
template<typename T>  
class MyStack {  
    std::vector<T> data;
```

TDD 및 Refactoring을 이용한 Clean Code 달성

1. 질문(Ask):테스트 작성을 통해 시스템에 질문한다. (테스트 수행 결과는 실패)
2. 응답(Respond):테스트를 통과하는 코드를 작성해서 질문에 대답한다. (테스트 성공)
3. 정제(Refine,Refactoring): 아이디어를 통합하고, 불필요한 것은 제거하고, 모호한 것은 명확히 해서 대답을 정제한다. (리팩토링)
4. 반복(Repeat): 다음 질문을 통해 대화를 계속 진행한다.

1일차 강의 자료 · 8시간

워크 북 실습

실습 #3 — Person & CalcSample TDD



CHAPTER 2

Google Test 고급

파라미터화 · Typed Test · EXPECT_THAT Matcher · ApprovalTests C++

TEST_P — 파라미터화 테스트 (Google Test)

C++ Values() = 정적 · ValuesIn() = 컨테이너 · Range(start,end) = 범위 ↔ Java @ValueSource / @CsvSource / @MethodSource / @EnumSource

C++ (Google Test / GMock)

```
// — 단일 값 파라미터 —————
class IsBlankTest
: public ::testing::TestWithParam<std::string> {};

TEST_P(IsBlankTest, ReturnsTrueForBlanks) {
    EXPECT_TRUE(isBlank(GetParam()));
}

INSTANTIATE_TEST_SUITE_P(
    BlankStrings,
    IsBlankTest,
    ::testing::Values("", " ", "  ")
);

// — 다중 값 (tuple) —————
class AddTest
: public ::testing::TestWithParam<
    std::tuple<int,int,int>> {};

TEST_P(AddTest, CorrectSum) {
    auto [a, b, expected] = GetParam(); // C++17
    Calculator calc;
    EXPECT_EQ(expected, calc.add(a, b));
}
```

Java 대응 (JUnit 5 / Mockito)

```
// Java - @ParameterizedTest 대응

// ① @ValueSource
@ParameterizedTest
@ValueSource(strings = {"", " "})
void isBlank(String input) {
    assertTrue(Strings.isBlank(input));
}

// ② @CsvSource (다중 값)
@ParameterizedTest(
    name="{index}: {0}+{1}={2}")
@CsvSource({"1,2,3", "5,5,10",
            "-1,1,0"})
void testAdd(int a,int b,int exp){
    assertEquals(exp, calc.add(a,b));
}

// ③ @MethodSource (복잡한 객체)
@ParameterizedTest
@MethodSource("provideArgs")
void testMulti(String s,
               int n, List<String> list){
    assertEquals(5, s.length());
}
```

TEST_P — 파라미터화 테스트 (Google Test)

C++ (Google Test / GMock)

```
INSTANTIATE_TEST_SUITE_P(
    AddCases,
    AddTest,
    ::testing::Values(
        std::make_tuple(1, 2, 3),
        std::make_tuple(5, 5, 10),
        std::make_tuple(-1, 1, 0)
    )
);

// — 이름 커스터마이징 —————
INSTANTIATE_TEST_SUITE_P(
    NamedCases, AddTest,
    ::testing::ValuesIn(std::vector<
        std::tuple<int,int,int>>{
            {1,2,3},{5,5,10}
        }),
    [](const auto& info) {
        return "case_" +
            std::to_string(info.index);
    }
);
```

Java 대응 (JUnit 5 / Mockito)

```
static Stream<Arguments> provideArgs(){
    return Stream.of(
        arguments("apple",1,
            List.of("a","b"))
    );
}
```


TYPED_TEST — 타입 파라미터화 (C++ 전용 강점)

실행 시 각 타입 조합 별 별도 테스트 케이스 생성

예: NumericStackTest/0 (int), /1 (float), /2 (double), /3 (long)

C++ (Google Test / GMock)

```
// — TYPED_TEST: 여러 타입에 동일 테스트 ——  
// (Java에는 없는 C++ 고유 기능)  
  
template <typename T>  
class NumericStackTest : public ::testing::Test {  
protected:  
    std::stack<T> stack_;  
};  
  
// 테스트할 타입 목록  
using NumericTypes =  
    ::testing::Types<int, float, double, long>;  
  
TYPED_TEST_SUITE(NumericStackTest, NumericTypes);  
  
TYPED_TEST(NumericStackTest, IsEmptyOnConstruct) {  
    EXPECT_TRUE(this->stack_.empty());  
}  
  
TYPED_TEST(NumericStackTest, PushAndPop) {  
    TypeParam val = static_cast<TypeParam>(42);  
    this->stack_.push(val);  
    EXPECT_FALSE(this->stack_.empty());  
    EXPECT_EQ(val, this->stack_.top());  
    this->stack_.pop();  
    EXPECT_TRUE(this->stack_.empty());  
}
```

TYPED_TEST — 타입 파라미터화 (C++ 전용 강점)

C++ (Google Test / GMock)

```
// — TYPED_TEST_P: 타입+파라미터 조합 —————
template <typename T>
class VectorTest : public ::testing::Test {};
TYPED_TEST_SUITE_P(VectorTest);

TYPED_TEST_P(VectorTest, DefaultConstructed) {
    std::vector<TypeParam> v;
    EXPECT_EQ(0u, v.size());
    EXPECT_TRUE(v.empty());
}

REGISTER_TYPED_TEST_SUITE_P(
    VectorTest,
    DefaultConstructed
);

using MyTypes = ::testing::Types<int, std::string>;
INSTANTIATE_TYPED_TEST_SUITE_P(My, VectorTest, MyTypes);
```

Java 대응 (제한적)

```
// Java는 Type erasure로 인해
// TYPED_TEST 직접 대응 없음
// 유사하게 제네릭 + @ParameterizedTest로 근사

@ParameterizedTest
@ValueSource(ints = {42})
<T extends Number> void stackTest(int v) {
    // 타입별 분기 처리 필요
}

// 실제로는 별도 클래스 작성이 일반적
```

TYPED_TEST 사용 시점

- 동일 로직, 다른 자료형 (int/float/double)
- 컨테이너 인터페이스 테스트 (stack/queue)
- 템플릿 클래스의 타입 안전성 검증
- 수치 알고리즘의 다형성 보장

C++만의 강점: 컴파일 타임 타입 안전성
+ 제로 오버헤드 테스트 다형성

EXPECT_THAT & Matchers — gmock의 강력한 검증 도구

C++ Matcher 기본 = gmock 내장 | 커스텀 Matcher: MATCHER(name, desc) { return arg > 0; } 매크로 사용 ↔ Java: extends BaseMatcher<T>

C++ (Google Test / GMock)

```
#include <gmock/gmock.h>
using namespace testing;

// — 기본 Matchers —
EXPECT_THAT(value, Eq(42));           // ==
EXPECT_THAT(value, Ne(0));            // !=
EXPECT_THAT(value, Gt(10));           // >
EXPECT_THAT(value, Ge(10));           // >=
EXPECT_THAT(value, Lt(100));          // <
EXPECT_THAT(value, Le(100));          // <=

// — 수치 —
double pi = 3.1415;
EXPECT_THAT(pi, DoubleNear(3.14, 0.01));
EXPECT_THAT(1, Ge(1));

// — 논리 조합 —
EXPECT_THAT(pi, AllOf(Gt(1.0), Lt(4.0)));
EXPECT_THAT(pi, AnyOf(Gt(2.0), Lt(4.0)));
EXPECT_THAT(pi, Not(Lt(0)));

// — 문자열 —
string s = "Hamcrest is awesome";
EXPECT_THAT(s, StartsWith("Ham"));
EXPECT_THAT(s, EndsWith("some"));
EXPECT_THAT(s, HasSubstr("is"));
```

Java 대응 (JUnit 5 / Mockito)

```
// Java Hamcrest 대응
import static org.hamcrest.MatcherAssert
    .assertThat;
import static org.hamcrest.Matchers.*;

// 기본
assertThat(val, equalTo(42));
assertThat(val, not(0));
assertThat(val, greaterThan(10));
assertThat(val, lessThan(100));

// 수치
assertThat(3.1, closeTo(3.0, 0.3));

// 논리
assertThat(pi,
    both(greaterThan(1.0)
        .and(lessThan(4.0)));
assertThat(pi,
    anyOf(greaterThan(2.0),
        lessThan(4.0)));

// 문자열
assertThat(s, startsWith("Ham"));
assertThat(s, endsWith("some"));
assertThat(s, containsString("is"));
assertThat(s,
    equalToIgnoringCase(
        "hamcrest is awesome"));
```

EXPECT_THAT & Matchers — gmock의 강력한 검증 도구

C++ (Google Test / GMock)

```
// — 컬렉션 —————  
vector<int> v{1, 2, 3};  
EXPECT_THAT(v, Contains(2));  
EXPECT_THAT(v, SizeIs(3));  
EXPECT_THAT(v, ElementsAre(1, 2, 3));  
EXPECT_THAT(v, UnorderedElementsAre(3,1,2));  
EXPECT_THAT(v, Each(Gt(0)));  
  
// Map  
map<string,int> m{"a",1},{"b",2};  
EXPECT_THAT(m, Contains(Pair("a", 1)));  
EXPECT_THAT(m, Contains(Key("b")));
```

Java 대응 (JUnit 5 / Mockito)

```
// 컬렉션  
assertThat(list, hasItem(2));  
assertThat(list, hasSize(3));  
assertThat(list,  
    containsInAnyOrder(3,1,2));  
assertThat(map,  
    hasEntry("a", 1));  
assertThat(map, hasKey("b"));
```

테스트 구조화 — Fixture 계층 · DisabledTest · 조건부 테스트

C++ 픽스처 계층 = 상속(ChildTest : ParentTest) ↔ Java @Nested 내부 클래스 | C++ 비활성화 = DISABLED_ 접두어 ↔ Java @Disabled 어노테이션

C++ (Google Test / GMock)

```
// — 픽스처 계층 (C++ Nested 대응) —————
class StackTest : public ::testing::Test {
protected:
    std::stack<std::string> stack_;
};

// "WhenNew" 그룹
TEST_F(StackTest, IsEmpty) {
    EXPECT_TRUE(stack_.empty());
}
TEST_F(StackTest, SizeIsZero) {
    EXPECT_EQ(0u, stack_.size());
}

// "AfterPush" 그룹 — 상속으로 픽스처 확장
class StackAfterPushTest : public StackTest {
protected:
    void SetUp() override {
        StackTest::SetUp();
        stack_.push("element");
    }
};
TEST_F(StackAfterPushTest, IsNotEmpty) {
    EXPECT_FALSE(stack_.empty());
}
```

Java 대응 (JUnit 5 / Mockito)

```
// Java — @Nested 계층 구조
class StackTest {

    @Nested
    class WhenNew {
        Stack<String> s = new Stack<>();

        @Test void isEmpty() {
            assertTrue(s.isEmpty());
        }
        @Test void sizeIsZero() {
            assertEquals(0, s.size());
        }
    }

    @Nested
    class AfterPush {
        Stack<String> s;
        @BeforeEach void push() {
            s = new Stack<>();
            s.push("element");
        }
        @Test void isNotEmpty() {
            assertFalse(s.isEmpty());
        }
        @Test void topIsElement() {
            assertEquals("element",
                          s.peek());
        }
    }
}
```

테스트 구조화 — Fixture 계층 · DisabledTest · 조건부 테스트

C++ (Google Test / GMock)

```
TEST_F(StackAfterPushTest, TopIsElement) {
    EXPECT_EQ("element", stack_.top());
}

// — 테스트 비활성화 —————
TEST(MyTest, DISABLED_NotYetImplemented) {
    // 접두어 DISABLED_ → 실행 제외, 경고 출력
    FAIL() << "not implemented";
}

// — 조건부 스킵 —————
TEST(MyTest, CiOnlyTest) {
    if (std::getenv("CI") == nullptr)
        GTEST_SKIP() << "CI only";
    // CI 환경에서만 실행
}

// — 테스트 순서 (랜덤화) —————
// cmake: --gtest_shuffle --gtest_random_seed=42
```

Java 대응 (JUnit 5 / Mockito)

```
// 비활성화: @Disabled
@Disabled
@Test void notYetImplemented() {}

// 조건부: @EnabledOnOs
@Test
@EnabledOnOs({LINUX, MAC})
void onLinuxOrMac() {}

// 조건부 스킵: assumeTrue
@Test void ciOnly() {
    assumeTrue("CI".equals(
        System.getenv("ENV")));
}
```

ApprovalTests C++ — 복잡한 출력 결과 스냅샷 검증

1

테스트 실행 → actual을 .received.txt 저장

2

.approved.txt 와 자동 비교

3

일치 → 통과 / 불일치 → FAIL + diff 뷰어

C++ (Google Test / GMock)

```
// — CMake - ApprovalTests 설치 —————
# FetchContent로 헤더 전용 라이브러리 추가
FetchContent_Declare(
  ApprovalTests
  GIT_REPOSITORY
    https://github.com/approvals/
    ApprovalTests.cpp.git
  GIT_TAG v.10.13.0
)
FetchContent_MakeAvailable(ApprovalTests)

target_link_libraries(all_tests
  ApprovalTests::ApprovalTests)

// — 기본 사용법 —————
#define APPROVALS_GOOGLETEST
#include "ApprovalTests.hpp"
```

Java 대응 (JUnit 5 / Mockito)

```
// Java Approval - Maven 의존성
// <dependency>
//   <groupId>com.approvaltests</groupId>
//   <artifactId>approvaltests</artifactId>
//   <version>9.5.0</version>
//   <scope>test</scope>
// </dependency>

import org.approvaltests.Approvals;

// 기본 사용법
@Test
public void testSortWords() {
    String input =
        "banana apple Orange Grape";
    String result =
        StringProcessor.process(input);

    Approvals.verify(result);
    // → ClassName.methodName
    //   .approved.txt 와 비교
}
```

ApprovalTests C++ — 복잡한 출력 결과 스냅샷 검증

C++ (Google Test / GMock)

```
// StringProcessor 테스트
TEST(StringProcessorTest, SortWords) {
    std::string input =
        "banana apple Orange Grape";
    std::string result =
        StringProcessor::process(input);

    ApprovalTests::Approvals::verify(result);
    // → StringProcessorTest.SortWords
    //   .approved.txt 와 비교
}
```

// — 컬렉션 Approval —————

```
TEST(PersonTest, ApproveList) {
    std::vector<std::string> names =
        {"Alice", "Bob", "Charlie"};

    ApprovalTests::Approvals::verifyAll(
        "Names", names);
    // approved.txt:
    // Names[0] = Alice
    // Names[1] = Bob
    // Names[2] = Charlie
}
```

Java 대응 (JUnit 5 / Mockito)

```
// 컬렉션 Approval
@Test
public void approveList() {
    List<String> names =
        List.of("Alice", "Bob", "Charlie");
    Approvals.verifyAll("Names", names);
}
```

approved.txt = 기준(Golden Master) 파일을 직접 생성하거나, received.txt 내용을 확인 후 rename하여 승인 | diff 뷰어: AraxisMerge / Beyond Compare / VSCode

CHAPTER 3

Google Mock

— Test Double 완전정복

MOCK_METHOD · ON_CALL · EXPECT_CALL · SaveArg · InSequence

Test Double 5종 — 언제 무엇을 쓸 것인가?

When we are writing a test in which we cannot (or chose not to) use a real depended-on component (DOC), we can replace it with a Test Double.

테스트 중 실제 의존성(DOC)을 사용할 수 없거나 사용하지 않기로 선택한 경우, 이를 대신하는 Test Double을 사용할 수 있음

The Test Double doesn't have to behave exactly like the real DOC; it merely has to provide the same API as the real one so that the SUT thinks it is the real one!

Test Double은 실제 DOC와 완전히 동일하게 동작할 필요는 없으며, SUT(System Under Test)가 실제 DOC처럼 인식하도록 동일한 API만 제공하면 됨.

By Gerad Meszaros(xUnit Test Patterns 저자 저라드 메스자로스) –

Test Double 5종 — 언제 무엇을 쓸 것인가?

유형	설명	실제행위	결과조작	행위검증
Dummy	인자 채우기용. 실제 사용 안 함.	X	X	X
Fake	단순화된 실제 구현 (인메모리 DB 등).	✓	X	△
Stub	미리 정의된 값 반환. 상태 설정용.	X	✓	X
Spy	실제 동작 + 호출 기록. 일부 override.	✓	△	✓
Mock	행위 검증 전용. EXPECT_CALL로 사전 선언.	X	✓	✓

C++ (Google Test / GMock)

```
// 공통 인터페이스 (C++) - 이후 예제 기반
class MessageService {
public:
    virtual std::string sendMessage(
        const std::string& to,
        const std::string& message) = 0;
    virtual ~MessageService() = default;
};

class NotificationService {
    MessageService* svc_;
public:
    explicit NotificationService(
        MessageService* s) : svc_(s) {}
    std::string notify(const std::string& user,
        const std::string& msg) {
        return svc_->sendMessage(user, msg);
    }
};
```

Java 대응 (JUnit 5 / Mockito)

```
// 공통 인터페이스 (Java)
public interface MessageService {
    String sendMessage(
        String to, String message);
}

public class NotificationService {
    private MessageService svc;
    public NotificationService(
        MessageService s) {
        this.svc = s;
    }
    public String notify(
        String user, String msg) {
        return svc.sendMessage(user, msg);
    }
}
```

Dummy & Fake — 직접 구현 Test Double

C++ (Google Test / GMock)

```
// — DUMMY —  
// 인터페이스 충족용, 실제로 호출 안 됨  
class DummyMessageService : public MessageService {  
public:  
    std::string sendMessage(  
        const std::string&  
        const std::string&) override {  
        return ""; // 아무 일도 안 함  
    }  
};  
  
TEST(NotificationTest, DummyExample) {  
    DummyMessageService dummy;  
    NotificationService svc(&dummy);  
    // sendMessage는 실제 호출 안 됨  
    EXPECT_EQ("OK", svc.getStatus());  
}
```

Java 대응 (JUnit 5 / Mockito)

```
// Java - Dummy  
MessageService dummy =  
    (to, msg) -> null;  
  
NotificationService svc =  
    new NotificationService(dummy);  
assertEquals("OK", svc.getStatus());
```

Dummy ≠ nullptr: nullptr 전달 시 역참조 크래시 위험 → 항상 빈 구현 클래스 제공

Dummy & Fake — 직접 구현 Test Double

C++ (Google Test / GMock)

```
// — FAKE —  
// 실제 동작하지만 단순화된 구현  
class FakeMessageService : public MessageService {  
    std::vector<std::string> messages_;  
public:  
    std::string sendMessage(  
        const std::string& to,  
        const std::string& msg) override {  
        messages_.push_back(to + ":" + msg);  
        return "sent";  
    }  
    const std::vector<std::string>&  
    getMessages() const { return messages_; }  
};  
  
TEST(FakeTest, StoresMessages) {  
    FakeMessageService fake;  
    fake.sendMessage("alice", "hi");  
    fake.sendMessage("bob", "hello");  
  
    EXPECT_EQ(2u, fake.getMessages().size());  
    EXPECT_THAT(fake.getMessages(),  
        Contains("alice:hi"));  
}
```

Java 대응 (JUnit 5 / Mockito)

```
// Java - Fake  
class FakeMessageService  
    implements MessageService {  
    List<String> messages =  
        new ArrayList<>();  
  
    @Override  
    public String sendMessage(  
        String to, String msg) {  
        messages.add(to + ":" + msg);  
        return "sent";  
    }  
}  
  
@Test void fakeStoresMessages() {  
    var fake = new FakeMessageService();  
    fake.sendMessage("alice", "hi");  
    fake.sendMessage("bob", "hello");  
  
    assertEquals(2,  
        fake.messages.size());  
    assertTrue(  
        fake.messages.contains(  
            "alice:hi"));  
}
```

Fake = 인메모리 DB, 파일 시스템 대체 등 실제 프로젝트에서 많이 사용

MOCK_METHOD & EXPECT_CALL — Google Mock 핵심

ON_CALL vs EXPECT_CALL: ON_CALL=결과만 지정(횟수 검증 없음), EXPECT_CALL=행위까지 검증 ↔ Java: when()=결과, verify()=행위검증 분리

C++ (Google Test / GMock)

```
#include <gmock/gmock.h>
using namespace testing;

// — Mock 클래스 정의 —————
class MockMessageService : public MessageService {
public:
    // MOCK_METHOD(반환타입, 메서드명, (인수타입...), (한정자))
    MOCK_METHOD(std::string, sendMessage,
        (const std::string& to,
         const std::string& msg),
        (override));
};

// — Stub (ON_CALL) - 반환값 설정 —————
TEST(StubTest, StubExample) {
    MockMessageService stub;

    // ON_CALL = 호출 시 반환값 지정 (검증 없음)
    ON_CALL(stub, sendMessage(_, _))
        .WillByDefault(Return("stubbed"));

    EXPECT_EQ("stubbed",
        stub.sendMessage("to", "msg"));
}
```

Java 대응 (JUnit 5 / Mockito)

```
// Java - Mockito 대응

// Mock 생성
MessageService mock =
    mock(MessageService.class);

// Stub (when...thenReturn)
when(mock.sendMessage(
    anyString(), anyString()))
    .thenReturn("stubbed");

// 반환값 확인
assertEquals("stubbed",
    mock.sendMessage("to", "msg"));
```

MOCK_METHOD & EXPECT_CALL — Google Mock 핵심

C++ (Google Test / GMock)

```
// — Mock (EXPECT_CALL) - 행위 검증 —————
TEST(MockTest, MockExample) {
    MockMessageService mock;

    // EXPECT_CALL = 호출 횟수·인수까지 검증
    EXPECT_CALL(mock, sendMessage("to", "hello"))
        .Times(1)
        .WillOnce(Return("mocked"));

    EXPECT_EQ("mocked",
        mock.sendMessage("to", "hello"));
    // 테스트 종료 시 EXPECT_CALL 자동 검증
}

// — 다양한 Times 지정 —————
EXPECT_CALL(mock, sendMessage(_, _))
    .Times(AnyNumber());    // 0회 이상
EXPECT_CALL(mock, sendMessage(_, _))
    .Times(AtLeast(1));    // 1회 이상
EXPECT_CALL(mock, sendMessage(_, _))
    .Times(Between(2, 5)); // 2~5회
```

Java 대응 (JUnit 5 / Mockito)

```
// Mock + 행위 검증 (verify)
MessageService m =
    mock(MessageService.class);
when(m.sendMessage("to", "hello"))
    .thenReturn("mocked");

String result =
    m.sendMessage("to", "hello");
assertEquals("mocked", result);

// 호출 횟수 검증
verify(m).sendMessage("to", "hello");
verify(m, times(1))
    .sendMessage(any(), any());
verify(m, never())
    .sendMessage("x", "hello");

// Times 지정
verify(m, atLeastOnce())
    .sendMessage(any(), any());
verify(m, atMost(3))
    .sendMessage(any(), any());
```

Stub · Spy · Argument Matchers — 고급 Mock 기법

NiceMock<T>: 예상치 못한 호출 경고 없음 | StrictMock<T>: 예상치 못한 호출 = 즉시 실패 | NaggyMock<T>: 기본값 (경고만)

C++ (Google Test / GMock)

```
// — STUB — 다양한 WillByDefault 패턴 —————
ON_CALL(mock, sendMessage(_, _))
    .WillByDefault(Return("default"));
ON_CALL(mock, sendMessage("vip", _))
    .WillByDefault(Return("priority")); // 우선 매칭

// 예외 던지기
ON_CALL(mock, sendMessage(_, _))
    .WillByDefault(Throw(
        std::runtime_error("network error")));
```

Java 대응 (JUnit 5 / Mockito)

```
// Java — Stub 패턴
// 기본 반환
when(mock.sendMessage(any(), any()))
    .thenReturn("default");

// 조건별 반환
when(mock.sendMessage(eq("vip"), any()))
    .thenReturn("priority");

// 예외
when(mock.sendMessage(any(), any()))
    .thenThrow(new RuntimeException(
        "network error"));
```


Stub · Spy · Argument Matchers — 고급 Mock 기법

C++ (Google Test / GMock)

```
// — SPY — 실제 객체 감싸기 —————
// (Google Mock: NiceMock/StrictMock으로 제어)
class RealMessageService : public MessageService {
public:
    std::string sendMessage(
        const std::string& to,
        const std::string& msg) override {
        return "real: " + msg;
    }
};

// 실제 구현 상속 후 일부만 override
class SpyMessageService
    : public RealMessageService {
public:
    MOCK_METHOD(std::string, sendMessage,
        (const std::string&,
         const std::string&),
        (override));
};
```

Java 대응 (JUnit 5 / Mockito)

```
// Java Spy
MessageService real = (to,msg) ->
    "real: " + msg;
MessageService spy = spy(real);

// 특정 호출만 가로채기
doReturn("fake").when(spy)
    .sendMessage("to", "msg");

assertEquals("fake",
    spy.sendMessage("to", "msg"));
assertEquals("real: other",
    spy.sendMessage("to", "other"));

// Argument Matchers
verify(mock).sendMessage(
    anyString(), anyString());
verify(mock).sendMessage(
    eq("to"), contains("hi"));
verify(mock).sendMessage(
    argThat(s -> s.startsWith("vip")),
    any());
```

Stub · Spy · Argument Matchers — 고급 Mock 기법

C++ (Google Test / GMock)

```
TEST(SpyTest, SpyExample) {
    SpyMessageService spy;
    // "to", "msg"만 가로채기
    EXPECT_CALL(spy, sendMessage("to", "msg"))
        .WillOnce(Return("fake"));
    // 나머지는 실제 호출로 위임
    ON_CALL(spy, sendMessage(Ne("to"), _))
        .WillByDefault([&](auto& a, auto& b){
            return RealMessageService::
                sendMessage(a, b);
        });
    EXPECT_EQ("fake",
        spy.sendMessage("to", "msg"));
}
```

```
// — Argument Matchers —————
// _ = 임의값, Eq(v), Ne(v), Gt, Lt
// StartsWith, HasSubstr, MatchesRegex
// ElementsAre, Contains, SizeIs
EXPECT_CALL(mock, sendMessage(
    StartsWith("vip"), HasSubstr("urgent")));
```

Google Mock 완전 정리 — C++ Cheatsheet

Mock 정의

`MOCK_METHOD(Ret, Name, (Args), (Override))`

`MOCK_METHOD(Ret, Name, (Args), (Const, Override))`

`NiceMock<T> / StrictMock<T>`

인터페이스를 흉내 내는 "가짜 객체(Mock)"를 만든다

ON_CALL (Stub)

`ON_CALL(m, f(_)).WillByDefault(Return(v))`

`.WillByDefault(Throw(ex))`

`.WillByDefault(Invoke(fn))`

이 함수는 기본적으로 이렇게 동작해라

EXPECT_CALL (검증)

`EXPECT_CALL(m, f(arg))`

`.Times(n) / AtLeast(n) / Between(a,b)`

`.WillOnce(Return(v)) /
.WillRepeatedly(...)`

이 함수는 반드시 이렇게 호출되어야 한다

Argument Matchers

`_, Eq(v), Ne(v), Gt(v), Lt(v), Ge, Le`

`StartsWith / EndsWith / HasSubstr /
MatchesRegex`

`Contains / ElementsAre / SizeIs / Each(m)`

함수 인자를 유연하게 검사

Actions

`Return(v) / ReturnRef(v) /
ReturnPointee(&v)`

`Throw(ex) / Invoke(fn) / DoAll(a1, a2)`

`SaveArg<N>(&v) / SetArgPointee<N>(v)`

함수가 호출됐을 때 무엇을 할지 정의

순서 검증

`InSequence seq; (블록 내 순서 강제)`

`::testing::Sequence s;`

`EXPECT_CALL(m, f()).InSequence(s)`

함수 호출 순서까지 검증

Google Mock 완전 정리 — C++ Cheatsheet

C++ (Google Test / GMock)

```
// Mockito → GMock 빠른 대응 표
// when(m,f()).thenReturn(v) → ON_CALL(m,f()).WillByDefault(Return(v))
// verify(m).f() → EXPECT_CALL(m,f()).Times(1)
// verify(m,times(n)) → EXPECT_CALL(m,f()).Times(n)
// verify(m,never()) → EXPECT_CALL(m,f()).Times(0)
// verify(m,atLeastOnce()) → EXPECT_CALL(m,f()).Times(AtLeast(1))
// ArgumentCaptor → SaveArg<N>(&var)
// InOrder → InSequence
```

CHAPTER 4

Write Better Tests

Red-Green-Refactor · 컬렉션 · operator== · 닭과달걀 · Test Smell

답과 달걀 메서드 — C++에서의 3가지 해결책

문제: `add()`와 `get()`이 서로를 필요로 함 → 어느 것을 먼저 독립적으로 테스트할 수 없음 (단위 테스트의 딜레마)

C++ (Google Test / GMock)

```
// — 해결책 1: 독립 테스트 분리 —————
TEST(AttendeeTest, TestAdd) {
    Attendee a;
    a.add("홍길동");
    // add의 결과를 get으로 확인 (불가피)
    EXPECT_EQ("홍길동", a.get(0));
}

TEST(AttendeeTest, TestGet) {
    Attendee a;
    a.add("임꺽정"); // 다른 데이터로 독립
    EXPECT_EQ("임꺽정", a.get(0));
}

// — 해결책 2: Mock으로 add() 분리 —————
class MockAttendee : public Attendee {
public:
    MOCK_METHOD(void, add,
                (const std::string&), (override));
};

TEST(AttendeeTest, MockAddAndGet) {
    MockAttendee mock;
    // add()는 Mock으로 분리 (항상 성공 가정)
    EXPECT_CALL(mock, add("홍길동"))
        .Times(1);
    mock.add("홍길동");
    // get()은 실제 구현 호출 (real 상속)
}
```

Java 대응 (JUnit 5 / Mockito)

```
// 해결책 1: 독립 테스트
@Test void testAdd() {
    Attendee a = new Attendee();
    a.add("홍길동");
    assertEquals("홍길동", a.get(0));
}

@Test void testGet() {
    Attendee a = new Attendee();
    a.add("임꺽정");
    assertEquals("임꺽정", a.get(0));
}

// 해결책 2: Mock으로 분리
Attendee mock =
    mock(Attendee.class);
doNothing().when(mock)
    .add(anyString());
when(mock.get(0))
    .thenReturn("홍길동");
mock.add("홍길동");
assertEquals("홍길동", mock.get(0));
```

답과 달걀 메서드 — C++에서의 3가지 해결책

문제: add()와 get()이 서로를 필요로 함 → 어느 것을 먼저 독립적으로 테스트할 수 없음 (단위 테스트의 딜레마)

C++ (Google Test / GMock)

```
// — 해결책 3: friend + 직접 필드 접근 ————
class Attendee {
    std::vector<std::string> attendees_;
    friend class AttendeeTest; // 테스트 클래스에 friend 선언
public:
    void add(const std::string& name) {
        attendees_.push_back(name);
    }
    std::string get(int i) const {
        return attendees_.at(i);
    }
};

TEST(AttendeeTest, AddWithDirectAccess) {
    Attendee a;
    a.add("홍길동");
    // private 필드 직접 접근 (friend 덕분)
    EXPECT_EQ(1u, a.attendees_.size());
    EXPECT_EQ("홍길동", a.attendees_[0]);
}

TEST(AttendeeTest, GetWithDirectSet) {
    Attendee a;
    a.attendees_.push_back("임꺽정"); // 직접 설정
    EXPECT_EQ("임꺽정", a.get(0));
}
```

Java 대응 (JUnit 5 / Mockito)

```
// 해결책 3: Reflection
@Test void addWithReflection()
    throws Exception {
    Attendee a = new Attendee();
    a.add("홍길동");
    // private 필드 직접 접근
    Field f = Attendee.class
        .getDeclaredField("attendees");
    f.setAccessible(true);
    List<?> list = (List<?>) f.get(a);
    assertEquals(1, list.size());
    assertEquals("홍길동", list.get(0));
}
```

C++ friend 클래스 선언 ↔ Java Reflection (setAccessible) | friend는 설계 냄새? → 테스트 목적이라면 허용 범위. private을 public으로 바꾸는 것보단 낫다

컬렉션 & 객체 동치성 — operator== / equals()

C++ (Google Test / GMock)

```
// — 기본형 컬렉션 비교 —————
std::vector<std::string> v1{"Alice","Bob","Carol"};
std::vector<std::string> v2{"Alice","Bob","Carol"};

EXPECT_EQ(v1, v2); // operator== 지원 ✓

// 순서 무시 비교
EXPECT_THAT(v1,
    UnorderedElementsAre("Bob","Carol","Alice")); // ✓

// — 객체 컬렉션 — operator== 없으면 실패 ———
struct Music {
    std::string song;
    std::string artist;
};

std::vector<Music> list1{
    {"Better in time","Leona Lewis"}};
std::vector<Music> list2{
    {"Better in time","Leona Lewis"}};

// EXPECT_EQ(list1, list2); // FAIL! operator== 없음
```

Java 대응 (JUnit 5 / Mockito)

```
// Java 기본형 컬렉션
List<String> a = List.of("A","B","C");
List<String> b = List.of("A","B","C");
assertEquals(a, b); // ✓ List.equals()

// 순서 무시
assertEquals(
    new HashSet<>(a),
    new HashSet<>(b)); // ✓

// Hamcrest
assertThat(a, containsInAnyOrder(
    "B","C","A")); // ✓

// 객체 컬렉션
class Music {
    String song, artist;
    // equals 미구현 → 실패!
}
List<Music> m1 = ...;
assertEquals(m1, m2); // FAIL
```


컬렉션 & 객체 동치성 — operator== / equals()

C++ (Google Test / GMock)

```
// — 해결책: operator== 정의 —————
struct Music {
    std::string song, artist;

    // C++20 기본 비교 연산자 (권장)
    bool operator==(const Music& const) = default;

    // C++17 이하: 직접 구현
    // bool operator==(const Music& o) const {
    //     return song==o.song && artist==o.artist;
    // }
};

// 이제 컬렉션 비교 가능
EXPECT_EQ(list1, list2);          // ✓

// GMock Matcher 활용
EXPECT_THAT(list1, ElementsAre(
    Field(&Music::song, "Better in time")));

EXPECT_THAT(list1, Contains(
    AllOf(Field(&Music::song, "Better in time"),
        Field(&Music::artist, "Leona Lewis"))));
```

Java 대응 (JUnit 5 / Mockito)

```
// 해결: equals/hashCode
@Override
public boolean equals(Object o) {
    if (this == o) return true;
    Music m = (Music) o;
    return song.equals(m.song) &&
        artist.equals(m.artist);
}

@Override
public int hashCode() {
    return Objects.hash(song, artist);
}

// Hamcrest 객체 프로퍼티 비교
assertThat(list, hasItem(
    hasProperty("song",
        equalTo("Better in time"))));
```

C++ operator== = default (C++20) — 자동 생성 | C++ Field(&Cls::member, matcher) → Java hasProperty("name", matcher) | 두 언어 모두 동치성 정의 필수!

TEST SMELL

테스트 코드에서 발생하는 나쁜 냄새

★ C++ 위주

Java (JUnit 5) 참고용 대응 코드 포함

1. 표지 & 5원칙
2. 6대 Smell 개요
3. Mystery Guest
4. Flaky Test
5. Assertion Roulette
6. Eager/Setup Bloat
7. 생성자·DTO 테스트
8. 닭과 달걀 + 해결책 3가지
9. 객체 동치성·컬렉션
10. Over-mocking & 정리

Write Better Tests — 5 Steps

1 Treat Test Code as Production Code

테스트 코드도 실제 코드처럼 다루기

2 Use Test Patterns for Readability

테스트 패턴으로 가독성 높이기

3 Avoid Unreliable Tests

불안정한 테스트 피하기

4 Test at The Appropriate Level

적절한 수준의 테스트 선택

5 Do Use Test Doubles

테스트 더블 적극 활용

Test Smell — 6가지 대표 유형 (C++ 중심)



Mystery Guest

❌ 외부 파일·DB 직접 의존 → 실패 원인 불명확

✅ 데이터 인라인 / Fake 객체로 격리



Flaky Test

❌ 시간·랜덤·전역 상태 의존 → 실행마다 결과 다름

✅ MockClock / MockRandom / SetUp 독립 보장



Assertion Roulette

❌ EXPECT 여러 개 → 실패 위치 불명확

✅ SCOPED_TRACE + 실패 메시지 추가



Eager Test

❌ 하나의 TEST에 기능 여럿 → SRP 위반

✅ 한 TEST = 한 관심사로 분리



Setup Bloat

❌ SetUp()이 비대 → 테스트 의도 파악 어려움

✅ 각 TEST_F에 필요한 셋업만 작성



Over-mocking

❌ 모든 것을 MOCK_METHOD → 실제 동작 미검증

✅ 핵심 외부 의존성만 Mock, 나머지는 실제

핵심 원칙: 테스트 코드도 Production 코드처럼 — 읽기 쉽고, 빠르고, 독립적이어야 한다

Mystery Guest — 외부 의존을 격리하라

파일·DB·네트워크를 직접 사용하면 테스트 실패 원인이 불명확해진다

Mystery Guest란?

테스트가 외부 리소스(파일, DB, 네트워크)에 의존할 때, 그 "손님"의 존재가 테스트 실패의 진짜 원인을 숨긴다.

해결책: 데이터를 테스트 내부에 인라인하거나, Fake 객체로 의존성을 격리한다.

C++ (Google Test / GMock)

```
// ❌ Bad - 외부 파일에 의존 (Mystery Guest)
TEST(UserTest, LoadFromFile_BAD) {
    // 파일 경로가 달라지거나 파일이 없으면 실패 원인 불명확!
    UserLoader loader("/data/test_users.csv");
    auto user = loader.load(1001);
    EXPECT_EQ("Alice", user.name);
    EXPECT_EQ(30,      user.age);
}

// ✅ Good - 데이터를 테스트 내부에 인라인
TEST(UserTest, LoadFromInlineData_GOOD) {
    // 테스트에 필요한 데이터를 직접 정의
    std::string csvData = "1001,Alice,30
1002,Bob,25";
    UserLoader loader(csvData); // 문자열 스트림으로 주입
    auto user = loader.load(1001);
    EXPECT_EQ("Alice", user.name);
    EXPECT_EQ(30,      user.age);
}

// ✅ Good - Fake 객체로 DB 의존 격리
class FakeUserRepository : public UserRepository {
    std::map<int, User> store_;
public:
    void save(User u) { store_[u.id] = u; }
    std::optional<User> find(int id) const override {
```

Java 대응 (JUnit 5 / Mockito)

```
// ❌ Bad
@Test void loadFromFile() {
    UserLoader loader =
        new UserLoader("/data/u.csv");
    assertEquals("Alice",
        loader.load(1001).name);
}

// ✅ Good - 인라인 데이터
@Test void loadFromInline() {
    String csv = "1001,Alice,30";
    UserLoader loader =
        new UserLoader(
            new StringReader(csv));
    assertEquals("Alice",
        loader.load(1001).name);
}

// ✅ Good - Fake Repository
@Test void findWithFake() {
    FakeUserRepository repo =
        new FakeUserRepository();
    repo.save(new User(1001,"Alice"));

    Optional<User> user =
        repo.find(1001);
```

Flaky Test — 시간.랜덤.전역 상태 의존을 제거하라

실행마다 결과가 달라지는 테스트는 신뢰할 수 없다

C++ (Google Test / GMock)

```
// ❌ Flaky - 시스템 시간 직접 사용
TEST(OrderTest, IsExpired_BAD) {
    Order order;
    order.createdAt = std::time(nullptr);
    std::this_thread::sleep_for(std::chrono::milliseconds(1));
    EXPECT_TRUE(order.isExpired(1)); // 실행 속도에 따라 결과 달라짐!
}

// ✅ Good - Clock 인터페이스 + Mock
class Clock {
public:
    virtual std::time_t now() const = 0;
    virtual ~Clock() = default;
};

class MockClock : public Clock {
public:
    MOCK_METHOD(std::time_t, now, (), (const, override));
};

TEST(OrderTest, IsExpired_GOOD) {
    MockClock clock;
    EXPECT_CALL(clock, now())
        .WillRepeatedly(Return(2000)); // 고정된 시간
    Order order(&clock);
    order.createdAt = 1000;           // 1000초 전 생성
    EXPECT_TRUE(order.isExpired(500)); // 500초 만료 → 확실히 만료
}
```

Java 대응 (JUnit 5 / Mockito)

```
// ❌ Flaky - 현재 시간 사용
@Test void isExpiredBad() {
    Order o = new Order();
    o.createdAt =
        System.currentTimeMillis();
    Thread.sleep(1);
    assertTrue(o.isExpired()); // 불안정
}

// ✅ Good - Clock Mock
Clock mock = mock(Clock.class);
when(mock.now()).thenReturn(2000L);
Order o = new Order(mock);
o.createdAt = 1000L;
assertTrue(o.isExpired(500)); // 안정
```

Flaky Test — 시간.랜덤.전역 상태 의존을 제거하라

원인 1: 시간 의존

`std::time()`, `std::chrono::now()` 직접 사용

원인 2: 랜덤 의존

`std::rand()`, `std::mt19937` 시드 고정 없이 사용

원인 3: 전역 상태

전역 변수, static 멤버 공유 → 테스트 순서 의존

C++ (Google Test / GMock)

```
// ❌ Flaky - 전역 상태 공유
int g_counter = 0;
TEST(CounterTest, Increment_BAD) {
    g_counter++;
    EXPECT_EQ(1, g_counter); // 다른 테스트 실행 후 실패할 수 있음!
}

// ✅ Good - 로컬 상태, 테스트 독립
TEST(CounterTest, Increment_GOOD) {
    Counter c; // 매 테스트마다 새로 생성
    c.increment();
    EXPECT_EQ(1, c.value()); // 항상 동일 결과
}
```

Java 대응 (JUnit 5 / Mockito)

```
// ❌ Flaky - 전역 상태
static int counter = 0;
@Test void t1() {
    counter++;
    assertEquals(1, counter); // 순서 의존
}

// ✅ Good - 로컬
@Test void t1Good() {
    Counter c = new Counter();
    c.increment();
    assertEquals(1, c.value());
}
```

Assertion Roulette — 실패 위치를 명확히 하라

여러 EXPECT 중 어느 것이 실패했는지 즉시 알 수 있어야 한다

C++ (Google Test / GMock)

```
// ❌ Bad – 어디서 실패했는지 모름 (Assertion Roulette)
TEST(PersonTest, CheckAll_BAD) {
    Person p{"Oliver", "Martin", 32};
    EXPECT_EQ("Oliver", p.firstName);
    EXPECT_TRUE(p.age < 40);
    EXPECT_EQ("Oliver Martin", p.fullName()); // 어느 줄에서 실패?
}

// ✅ Good – SCOPED_TRACE로 실패 위치 명확히 표시
TEST(PersonTest, CheckAll_GOOD) {
    Person p{"Oliver", "Martin", 32};

    {
        SCOPED_TRACE("Check firstName");
        EXPECT_EQ("Oliver", p.firstName)
            << "firstName mismatch: " << p.firstName;
    }

    {
        SCOPED_TRACE("Check age");
        EXPECT_LT(p.age, 40)
            << "age out of range: " << p.age;
    }

    {
        SCOPED_TRACE("Check fullName");
        EXPECT_EQ("Oliver Martin", p.fullName())
            << "fullName wrong: " << p.fullName();
    }
}
```

Java 대응 (JUnit 5 / Mockito)

```
// ❌ Bad – Assertion Roulette
@Test void checkAllBad() {
    Person p = new Person(
        "Oliver", "Martin", 32);
    assertEquals("Oliver",
        p.getFirstName());
    assertTrue(p.getAge() < 40);
    assertEquals("Oliver Martin",
        p.getFullName());
    // 어느 줄 실패인지 불명확
}

// ✅ Good – assertAll로 묶기
@Test void checkAllGood() {
    Person p = new Person(
        "Oliver", "Martin", 32);
    assertAll("person checks",
        () -> assertEquals("Oliver",
            p.getFirstName()),
        () -> assertTrue(
            p.getAge() < 40,
            "age out of range: "
            + p.getAge()),
        () -> assertEquals(
            "Oliver Martin",
            p.getFullName())
    );
    // 모든 실패를 한 번에 보고
}
```

Assertion Roulette — 실패 위치를 명확히 하라

EXPECT_* vs ASSERT_*

EXPECT_*: 실패해도 계속 실행 → 여러 실패를 한 번에 확인 가능 (권장)

ASSERT_*: 실패 즉시 테스트 중단 → 후속 검사 불가. 전제 조건 검사에만 사용

C++ (Google Test / GMock)

```
// ✅ Good – 파라미터화 테스트로 독립 검증 (각 케이스가 별도 결과)
class PersonParamTest
: public ::testing::TestWithParam<
    std::tuple<std::string,int,std::string>> {}

TEST_P(PersonParamTest, ValidatePerson) {
    auto [firstName, age, fullName] = GetParam();
    Person p{firstName, "Martin", age};
    EXPECT_EQ(firstName, p.firstName);
    EXPECT_EQ(fullName, p.fullName());
}

INSTANTIATE_TEST_SUITE_P(
    PersonCases, PersonParamTest,
    ::testing::Values(
        std::make_tuple("Oliver", 32, "Oliver Martin"),
        std::make_tuple("Alice", 25, "Alice Martin")
    )
);
```

Java 대응 (JUnit 5 / Mockito)

```
// ✅ Good – @ParameterizedTest
@ParameterizedTest
@CsvSource({"Oliver,32,Oliver Martin",
           "Alice,25,Alice Martin"})
void validatePerson(
    String first, int age, String full){
    Person p=new Person(first,"Martin",age);
    assertEquals(first, p.getFirstName());
    assertEquals(full, p.getFullName());
}
```


Eager Test & Setup Bloat — 한 테스트, 한 관심사

여러 기능을 한 테스트에 넣거나 setUp을 비대하게 만들지 말 것

❶ Eager Test — 한 TEST = 한 관심사 (SRP)

C++ (Google Test / GMock)

```
// ❌ Eager — 여러 기능을 한 TEST에서 검증 (SRP 위반)
TEST(CalculatorTest, DoEverything_BAD) {
    Calculator c;
    EXPECT_EQ(10, c.add(5, 5));           // 기능 ①
    EXPECT_EQ(0, c.subtract(5, 5));       // 기능 ②
    EXPECT_EQ(25, c.multiply(5, 5));       // 기능 ③
    EXPECT_THROW(c.divide(10, 0),         // 기능 ④
        std::invalid_argument);
}

// ✅ Good — 각 기능별 독립 TEST
TEST(CalculatorTest, Add) { Calculator c; EXPECT_EQ(10, c.add(5, 5)); }
TEST(CalculatorTest, Subtract) { Calculator c; EXPECT_EQ(0, c.subtract(5, 5)); }
TEST(CalculatorTest, Multiply) { Calculator c; EXPECT_EQ(25, c.multiply(5, 5)); }
TEST(CalculatorTest, DivByZero) {
    Calculator c;
    EXPECT_THROW(c.divide(10, 0), std::invalid_argument);
}
```

Java 대응 (JUnit 5 / Mockito)

```
// ❌ Eager — Bad
@Test void doEverything() {
    assertEquals(10, c.add(5,5));
    assertEquals(0, c.sub(5,5));
    assertThrows(ArithmeticException.class,
        ()->c.divide(10,0));
}

// ✅ Good — 각 기능 분리
@Test void add() { assertEquals(10, c.add(5,5)); }
@Test void subtract() { assertEquals(0, c.sub(5,5)); }
@Test void divByZero(){
    assertThrows(ArithmeticException.class,
        ()->c.divide(10,0));
}
```

Eager Test & Setup Bloat — 한 테스트, 한 관심사

Setup Bloat — SetUp은 간결하게, 필요한 것만

C++ (Google Test / GMock)

```
// ❌ Setup Bloat – 모든 것을 SetUp에 몰아넣음
class UserServiceTest : public ::testing::Test {
protected:
    void SetUp() override {
        db_.connect("localhost", 5432, "testdb");
        db_.createSchema();
        db_.seed("fixtures/all_users.sql");    // 수천 건 데이터
        cache_.init(db_);
        emailSvc_.configure("smtp://test");    // 이 테스트와 무관!
        smsAlertSvc_.enable();                // 이 테스트와 무관!
    }
};

// ✅ Good – 각 테스트에 필요한 것만
class UserServiceTest : public ::testing::Test {
protected:
    UserService svc_;                        // 공통 SUT만 초기화
    void SetUp() override { /* 최소화 */ }
};

TEST_F(UserServiceTest, FindUser) {
    FakeUserRepo repo;
    repo.save({1001, "Alice"});              // 이 테스트에만 필요한 데이터
    EXPECT_EQ("Alice", svc_.find(repo, 1001).name);
}
```

Java 대응 (JUnit 5 / Mockito)

```
// ❌ Setup Bloat
@BeforeEach void setUp() {
    db.connect("localhost");
    db.createSchema();
    db.seed("all_data.sql"); // 과도한 셋업
    emailSvc.configure("smtp://"); // 무관
}

// ✅ Good – 최소 셋업
@BeforeEach void setUp() {
    svc = new UserService(); // 최소화
}

@Test void findUser() {
    FakeUserRepo repo = new FakeUserRepo();
    repo.save(new User(1001, "Alice"));
    assertEquals("Alice",
        svc.find(repo, 1001).name);
}
```

원칙: 한 TEST가 실패하면 원인이 명확해야 한다. Eager는 '여러 원인의 가능성', Setup Bloat은 '원인 추적의 어려움'을 만든다.

TDD 체크리스트 ① — 생성자·구조체(DTO) 테스트

단순 초기화는 생략, 로직, 전제조건, 변환 포함 시 필수

① 생성자 테스트 — 언제 작성하는가?

```
// ① 단순 생성 → 테스트 생략
struct UserDTO { std::string name; int age; }; // 단순 데이터 → 테스트 불필요

// ② 전제조건 있는 생성자 → 필수 테스트
class Temperature {
    double celsius_;
public:
    explicit Temperature(double c) {
        if (c < -273.15)
            throw std::out_of_range("절대 영도 미만");
        celsius_ = c;
    }
    double get() const { return celsius_; }
};

TEST(TemperatureTest, ValidRange) {
    EXPECT_NO_THROW(Temperature(25.0));
    EXPECT_NO_THROW(Temperature(-273.15)); // 경계값
}

TEST(TemperatureTest, BelowAbsoluteZero) {
    EXPECT_THROW(Temperature(-300.0), std::out_of_range);
}
```

```
// ① 단순 → 생략
class UserDTO {
    String name; int age; // 생략

// ② 전제조건 → 필수
class Temperature {
    double c;
    Temperature(double c) {
        if (c < -273.15) throw
            new IllegalArgumentException();
        this.c = c;
    }
}

@Test void validRange() {
    assertDoesNotThrow(
        ()->new Temperature(25.0));
}

@Test void belowAbsZero() {
    assertThrows(
        IllegalArgumentException.class,
        ()->new Temperature(-300.0));
}
```

TDD 체크리스트 ① — 생성자·구조체(DTO) 테스트

단순 초기화는 생략, 로직·전제조건·변환 포함 시 필수

```
// ㉓ DB 커넥션 확보 → 필수 테스트 (Mock 활용)
class MockDBConnection : public DBConnection {
public:
    MOCK_METHOD(bool, connect, (), (override));
    MOCK_METHOD(bool, isConnected, (), (const, override));
};
TEST(EmployeeDaoTest, ConnectsOnConstruct) {
    MockDBConnection mockDb;
    EXPECT_CALL(mockDb, connect()).WillOnce(Return(true));
    EXPECT_CALL(mockDb, isConnected()).WillOnce(Return(true));
    EmployeeDao dao(&mockDb);
    EXPECT_TRUE(dao.isConnected());
}
```

```
// ㉓ DB 커넥션 → 필수
@Test void connectsOnConstruct() {
    DBConnection mockDb =
        mock(DBConnection.class);
    when(mockDb.connect())
        .thenReturn(true);
    EmployeeDao dao =
        new EmployeeDao(mockDb);
    assertTrue(dao.isConnected());
}
```

TDD 체크리스트 ② — 구조체(DTO) 변환 로직 테스트 예제

② 구조체/DTO 테스트 — 변환 로직이 있으면 필수!

단순 getter/setter → 생략 | 변환 로직(getEmailDomain, isExpensive 등) → 필수 | 불변 객체(const 멤버) → 상태 검증 메서드 테스트

```
struct UserDTO {
    std::string name;
    int age;
    // 단순 데이터 컨테이너 → 별도 테스트 불필요
};

// — Ex2. 변환 로직 포함 구조체 → 필수 테스트 —————
struct UserDTO {
    std::string name;
    std::string email;

    // 변환 로직: 이메일 도메인 추출
    std::string getEmailDomain() const {
        auto at = email.find('@');
        return at != std::string::npos
            ? email.substr(at + 1)
            : "";
    }
};

TEST(UserDTOTest, ValidEmail) {
    UserDTO u{"Alice", "alice@example.com"};
    EXPECT_EQ("example.com", u.getEmailDomain());
}

TEST(UserDTOTest, InvalidEmail) {
    UserDTO u{"Bob", "invalid-email"};
    EXPECT_EQ("", u.getEmailDomain()); // '@' 없으면 빈 문자열
}
```

```
String name;
int age;
// getter/setter만 → 생략
}

// Ex2. 변환 로직 → 필수
class UserDTO {
    String email;
    String getEmailDomain() {
        if(email!=null&&email.contains("@"))
            return email.split("@")[1];
        return null;
    }
}

@Test void validEmail() {
    UserDTO u = new UserDTO();
    u.email = "alice@example.com";
    assertEquals("example.com",
        u.getEmailDomain());
}

@Test void invalidEmail() {
    UserDTO u = new UserDTO();
    u.email = "invalid";
    assertNull(u.getEmailDomain());
}
```

TDD 체크리스트 ② — 구조체(DTO) 변환 로직 테스트 예제

단순 멤버는 생략, 로직 상태 메서드는 필수 테스트

C++ (Google Test / GMock)

```
// — Ex3. 불변 객체(const 멤버) + 상태 메서드 → 필수 테스트 —————
class ProductDTO {
    const std::string name_;
    const int price_;
public:
    ProductDTO(std::string n, int p)
        : name_(std::move(n)), price_(p) {}
    const std::string& getName() const { return name_; }
    int getPrice() const { return price_; }
    bool isExpensive() const { return price_ > 1000; } // 비즈니스
    로직!
};
TEST(ProductDTOTest, IsExpensive) {
    ProductDTO pen {"Pen", 500};
    ProductDTO laptop{"Laptop", 1500};
    EXPECT_FALSE(pen.isExpensive());
    EXPECT_TRUE(laptop.isExpensive());
}
TEST(ProductDTOTest, BoundaryPrice) {
    ProductDTO boundary{"Item", 1000}; // 경계값 1000
    EXPECT_FALSE(boundary.isExpensive()); // price > 1000, 1000은 아님
    ProductDTO overBound{"Item", 1001};
    EXPECT_TRUE(overBound.isExpensive());
}
```

Java 대응 (JUnit 5 / Mockito)

```
// Ex3. 불변 객체 + 상태 → 필수
class ProductDTO {
    final String name;
    final int price;
    ProductDTO(String n, int p){
        name=n; price=p; }
    boolean isExpensive(){
        return price > 1000; }
}
@Test void isExpensive() {
    assertFalse(
        new ProductDTO("Pen",500)
            .isExpensive());
    assertTrue(
        new ProductDTO("Laptop",1500)
            .isExpensive());
}
```

C++ tip: const 멤버 = 불변 보장 (Java의 final). isExpensive() 같은 비즈니스 로직은 경계값(1000, 1001)도 반드시 테스트

TDD 체크리스트 ③ — 닭과 달걀 & 객체 동치성 & 컬렉션

서로 의존하는 메서드 쌍, operator==, vector 비교 패턴

③ 닭과 달걀 메서드 — add()와 get()이 서로를 필요로 할 때

C++ (Google Test / GMock)

```
// 문제: add() 검증에 get() 필요, get() 검증에 add() 필요
class Attendee {
    std::vector<std::string> list_;
    friend class AttendeeTest; // ← 해결책 3: friend 선언
public:
    void add(const std::string& name) { list_.push_back(name); }
    std::string get(int i) const { return list_.at(i); }
};

// 해결책 1: 테스트 분리 (다른 데이터로 독립)
TEST(AttendeeTest, TestAdd) {
    Attendee a; a.add("홍길동");
    EXPECT_EQ("홍길동", a.get(0)); // add 결과를 get으로 확인 (불가피)
}
TEST(AttendeeTest, TestGet) {
    Attendee a; a.add("임꺽정"); // 다른 값으로 독립
    EXPECT_EQ("임꺽정", a.get(0));
}
```

Java 대응 (JUnit 5 / Mockito)

```
// 해결책 1: 분리
@Test void testAdd() {
    Attendee a = new Attendee();
    a.add("홍길동");
    assertEquals("홍길동", a.get(0));
}
@Test void testGet() {
    Attendee a = new Attendee();
    a.add("임꺽정"); // 다른 값
    assertEquals("임꺽정", a.get(0));
}
```

TDD 체크리스트 ③ — 닭과 달걀 & 객체 동치성 & 컬렉션

③ 닭과 달걀 메서드 — add()와 get()이 서로를 필요로 할 때

C++ (Google Test / GMock)

```
// 해결책 2: Mock으로 add() 분리
class MockAttendee : public Attendee {
public:
    MOCK_METHOD(void, add, (const std::string&), (override));
};

TEST(AttendeeTest, GetWithMockAdd) {
    MockAttendee mock;
    EXPECT_CALL(mock, add("홍길동")).Times(1);
    mock.add("홍길동");           // add = Mock (검증용)
}

// 해결책 3: friend로 private 필드 직접 접근
TEST(AttendeeTest, AddWithFriend) {
    Attendee a; a.add("홍길동");
    EXPECT_EQ(1u, a.list_.size()); // friend로 직접 접근 ✓
    EXPECT_EQ("홍길동", a.list_[0]);
}
```

Java 대응 (JUnit 5 / Mockito)

```
// 해결책 2: Mock
Attendee mock = mock(Attendee.class);
doNothing().when(mock).add(any());
when(mock.get(0)).thenReturn("홍");
// 해결책 3: Reflection
Field f = Attendee.class
    .getDeclaredField("attendees");
f.setAccessible(true);
List<?> list = (List<?>)f.get(mock);
assertEquals(1, list.size());
```


TDD 체크리스트 ③ — 닙과 달걀 & 객체 동치성 & 컬렉션

4 객체 동치성 & 컬렉션 비교

C++ (Google Test / GMock)

```
// operator== 없으면 컬렉션 비교 실패!
struct Music { std::string song, artist;
    bool operator==(const Music& o) const = default; // C++20
};
std::vector<Music> v1{{"Better", "Leona"}};
std::vector<Music> v2{{"Better", "Leona"}};
EXPECT_EQ(v1, v2); // ✓ operator== 정의 후 성공
EXPECT_THAT(v1, ElementsAre(Field(&Music::song, "Better")));
EXPECT_THAT(v1, UnorderedElementsAreArray(v2)); // 순서 무관 비교
```

Java 대응 (JUnit 5 / Mockito)

```
// equals/hashCode 미구현 → 실패!
// @Override equals + hashCode 구현 후:
List<Music> a = List.of(new Music("Better", "Leona"));
List<Music> b = List.of(new Music("Better", "Leona"));
assertEquals(a, b); // ✓
// 순서 무관: assertThat(a, containsInAnyOrder(...))
// 배열: assertEquals(arr1, arr2)
```

C++ friend 클래스 ↔ Java Reflection(setAccessible) | C++20 operator== =default ↔ Java equals()/hashCode() 오버라이드

Over-mocking & 종합 정리 — Test Smell 제거 원칙

꼭 필요한 것만 Mock, 그리고 6대 Smell 핵심 한 눈에 보기



Over-mocking — 핵심 외부 의존성에만 Mock을 사용하라

C++ (Google Test / GMock)

```
// ❌ Over-mocking - 모든 것을 Mock, 실제 로직 검증 불가
TEST(OrderTest, BadOverMock) {
    MockProduct    mockProd;
    MockInventory   mockInv;
    MockPricing     mockPrice;
    MockLogger      mockLog;
    MockNotifier    mockNotify;    // 과도한 Mock!
    ON_CALL(mockProd, getName()).WillByDefault(Return("Pen"));
    ON_CALL(mockInv,  hasStock(1)).WillByDefault(Return(true));
    ON_CALL(mockPrice, calc(1)).WillByDefault(Return(500));
    // 실제 주문 처리 로직은 아무것도 검증하지 않음!
}

// ✅ Good - 느린 외부 의존(결제 게이트웨이)만 Mock
TEST(OrderTest, GoodMock) {
    MockPaymentGateway mockGateway;
    EXPECT_CALL(mockGateway, charge(_, 1500))
        .WillOnce(Return(true));

    // 내부 로직은 실제 구현 사용
    Product product{"Laptop", 1500};
    Inventory inventory;    // 실제 인메모리 인벤토리
    Order order(product, inventory, &mockGateway);

    EXPECT_TRUE(order.process());
    EXPECT_EQ(OrderStatus::COMPLETED, order.status());
}
```

Java 대응 (JUnit 5 / Mockito)

```
// ❌ Over-mocking
@Test void badOverMock() {
    Product mock1 = mock(Product.class);
    Inventory mock2 = mock(Inventory.class);
    Pricing mock3 = mock(Pricing.class);
    Logger mock4 = mock(Logger.class);
    // 실제 검증 없음 - 의미없는 테스트
}

// ✅ Good - 결제만 Mock
@Test void goodMock() {
    PaymentGateway gateway =
        mock(PaymentGateway.class);
    when(gateway.charge(any(), eq(1500)))
        .thenReturn(true);

    Product product =
        new Product("Laptop", 1500);
    Order order = new Order(
        product,
        new Inventory(), // 실제 구현
        gateway);        // 핵심만 Mock

    assertTrue(order.process());
    assertEquals(COMPLETED,
        order.getStatus());
}
```

Over-mocking & 종합 정리 — Test Smell 제거 원칙

꼭 필요한 것만 Mock, 그리고 6대 Smell 핵심 한 눈에 보기

6대 Test Smell — 핵심 요약		
Test Smell	C++ 해결책 (Google Test)	Java 대응 (JUnit 5)
Mystery Guest	Fake 객체 / 인라인 데이터	Fake 구현체 / @TempDir
Flaky Test	MockClock / --gtest_shuffle	mock(Clock) / @Order 금지
Assertion Roulette	SCOPED_TRACE + << 메시지	assertAll("group", ...)
Eager Test	TEST별 단일 기능 분리	@Test별 단일 기능 분리
Setup Bloat	SetUp 최소화 / 픽스처 분리	@BeforeEach 최소화
Over-mocking	외부 I/O·네트워크만 Mock	외부 의존성만 mock()

4시간 강의 핵심 정리 — C++ 주, Java 참고

Ch.1

Google Test 기초 & 테스트 설계

CMake 설정 · TEST/TEST_F · AAA 패턴 · FIRST/Right-BICEP/CORRECT 원칙 · Largest 실습

Ch.2

Google Test 고급

TEST_P 파라미터화 · TYPED_TEST(타입 다형성) · EXPECT_THAT Matchers · ApprovalTests C++

Ch.3

Google Mock 완전정복

MOCK_METHOD · ON_CALL(Stub) · EXPECT_CALL(Mock) · SaveArg · NiceMock/StrictMock · InSequence

Ch.4

Write Better Tests — TDD 실전

Red-Green-Refactor · 닭과달걀(friend/Mock) · operator== · 컬렉션 · Test Smell 6종

4

마무리 & 실천 로드맵

C++ /Java 비교 정리 · 단계별 적용 전략

🕒 07:20 ~ 07:40 (20분)

Java vs C++ 테스트 도구 전체 비교

구분	☕ Java	⚙️ C++
테스트 프레임워크	JUnit 5 (+ Vintage for JUnit 4)	GoogleTest (GTest)
모킹 프레임워크	Mockito	Google Mock (GMock)
Matcher 라이브러리	Hamcrest	GMock Matchers
빌드 통합	Maven Surefire Plugin / Gradle	CMake + ctest
커버리지	JaCoCo (mvn verify)	gcov / lcov / llvm-cov
VSCode UI	Test Runner for Java (Testing 패널)	C++ TestMate (Testing 패널)
정적 분석	Checkstyle · SpotBugs · PMD	clang-tidy · cppcheck
CI/CD	GitHub Actions: mvn verify	GitHub Actions: cmake + ctest
Parameterized	@ParameterizedTest + @ValueSource 등	INstantiate_TEST_SUITE_P + Values
Test Fixture	@BeforeEach / @AfterEach	SetUp() / TearDown() override
Test Double 생성	@Mock / @Spy / @InjectMocks	MOCK_METHOD / ON_CALL / EXPECT_CALL

핵심 요약 & 참고 자료

파트	핵심 원칙	☕ Java	⚙️ C++
SW 테스트	결함은 일찍 발견할수록 비용 감소	JUnit 5 + JaCoCo	GoogleTest + gcov
테스트 케이스 설계	AAA · FIRST · Right-BICEP · CORRECT	@ParameterizedTest	INstantiate_Test_Suite_P
JUnit 5 / GTest	어노테이션 · 라이프사이클 · 검증	@Test @BeforeEach assertAll()	TEST_F SetUp() EXPECT_EQ
Test Double	외부 의존성 격리 → 단위 테스트	Mockito: @Mock @Spy	GMock: MOCK_METHOD EXPECT_CALL
TDD 사이클	RED → GREEN → REFACTOR 반복	mvn test → 초록 막대	ctest → PASS
Test Smell	Flaky·Fragile·Roulette 제거	PMD + SpotBugs 자동 탐지	clang-tidy + cppcheck

참고 문헌 & 자료

TDD by Example — Kent Beck
(Addison-Wesley, 2002)

JUnit 5 공식 문서
junit.org/junit5/docs/current/user-guide

Mockito 공식 문서
site.mockito.org

xUnit Test Patterns — Gerard Meszaros
xunitpatterns.com

GoogleTest 공식 문서
google.github.io/googletest

VSCode Java Testing
code.visualstudio.com/docs/java/java-testing

수고하셨습니다! 🎉

Test the program before you write it.

🔬 SW 테스트 — 결함 비용은 발견 시점에 따라 100배 차이

⚙️ AAA · FIRST · Right-BICEP · CORRECT — 좋은 테스트의 설계 원칙

☕ JUnit 5 / GoogleTest — @Test · @ParameterizedTest · Assertions

🐱 Test Double — Dummy · Fake · Stub · Spy · Mock + Mockito / GMock

🔴 TDD — RED → GREEN → REFACTOR · 3법칙 · Ask-Respond-Refine

🔍 Test Smell — Flaky · Fragile · Roulette · 정적 분석으로 자동 탐지